

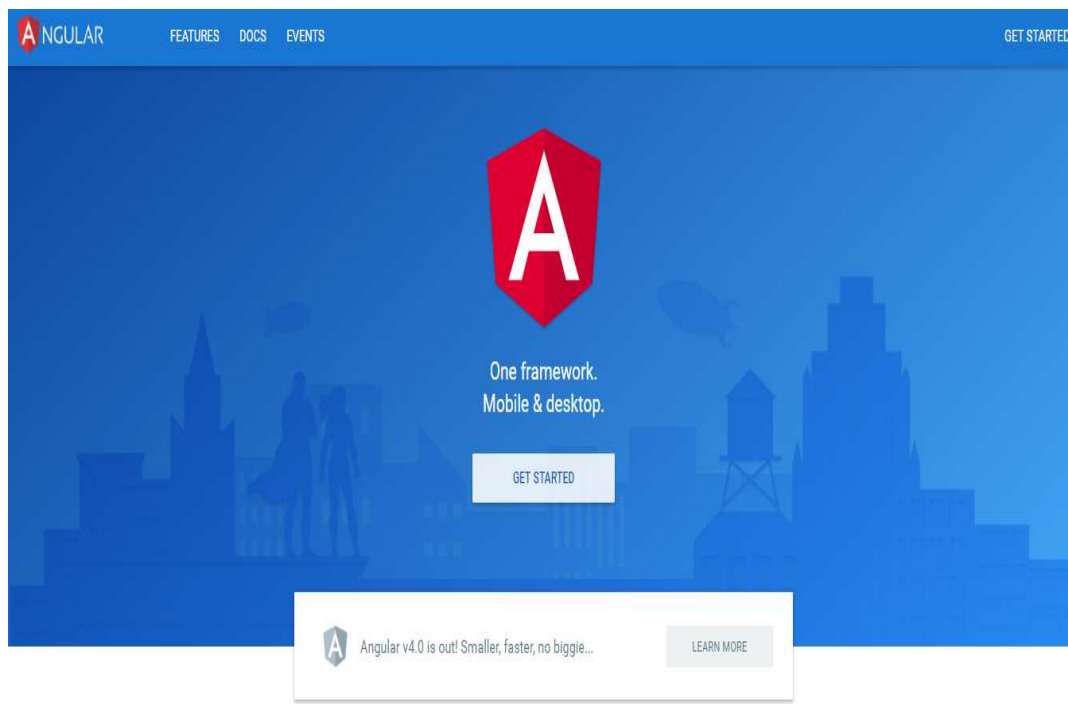
Angular 20 et versions antérieures - Développement d'applications Web

Aymen sellaouti





Références



Plan du Cours

1. Introduction
2. Les composants Standalone, la nouvelle architecture d'angular
 1. Le Binding
 2. Les signaux
 3. Le nouveau control Flow
 4. Interaction entre composants
 5. Directives
 6. Pipes
 7. Service et Injection de dépendances
11. Routing
12. Template Driven Form
13. HTTP
14. Form réactifs
15. HttpResources
16. Tests unitaires
17. Tests E2E



PRÉ-REQUIS

- ▶ HTML 5 / CSS 3 / Bootstrap



- ▶ JavaScript



- ▶ Programmation Orientée Objet

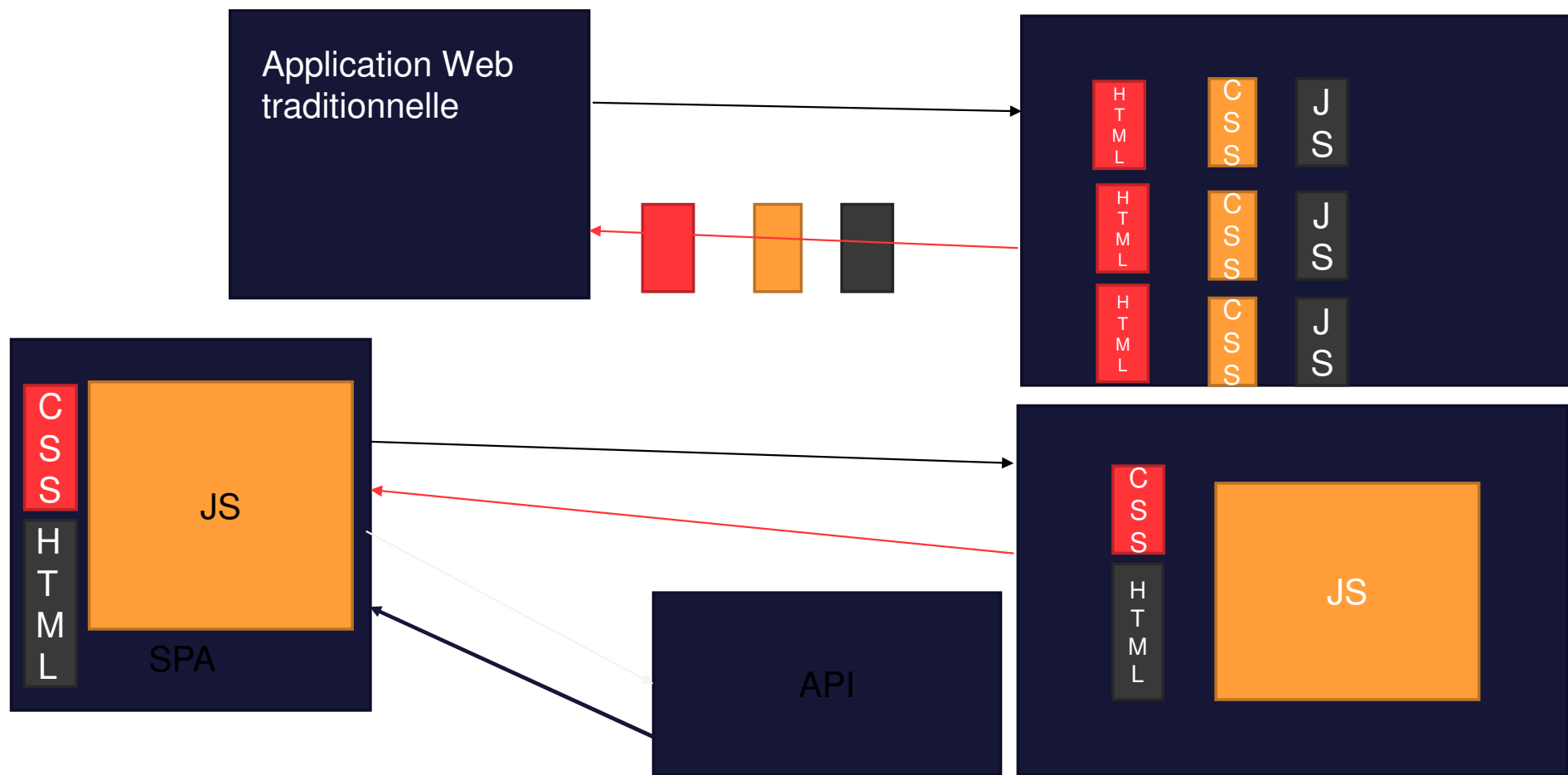
POO



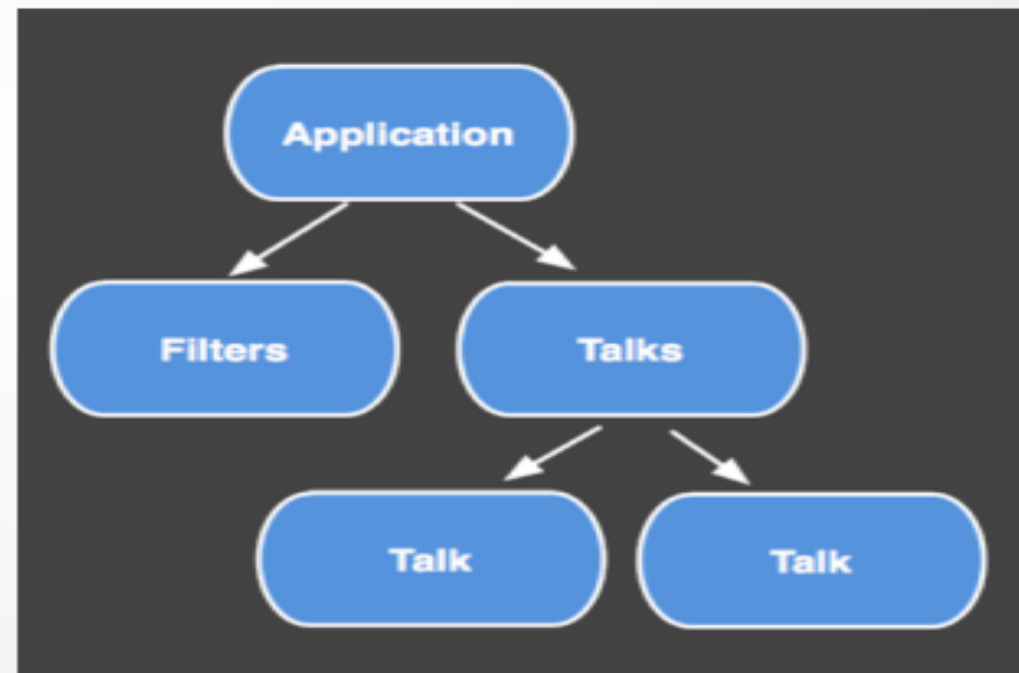
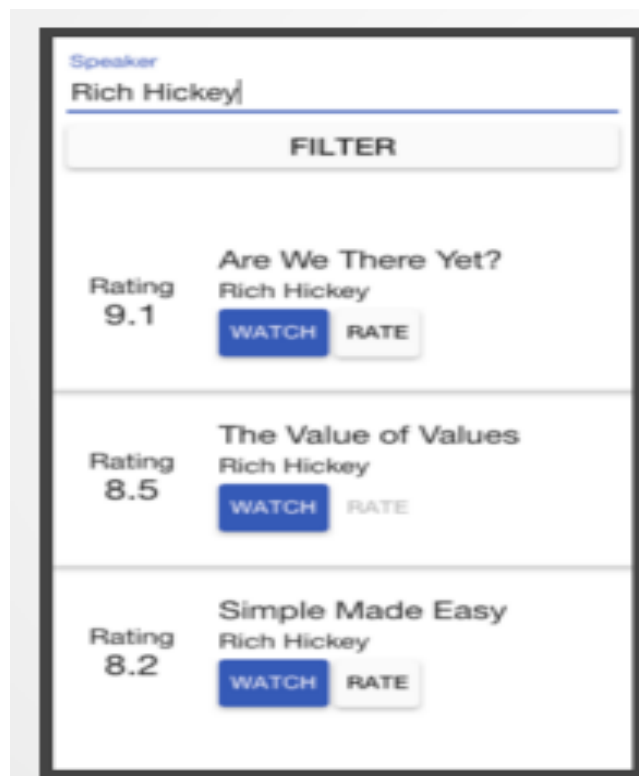
C'est quoi Angular?

- Framework JS
- SPA
- Supporte plusieurs langages : ES5, EC6, TypeScript, Dart
- Rapide
- Orienté Composant

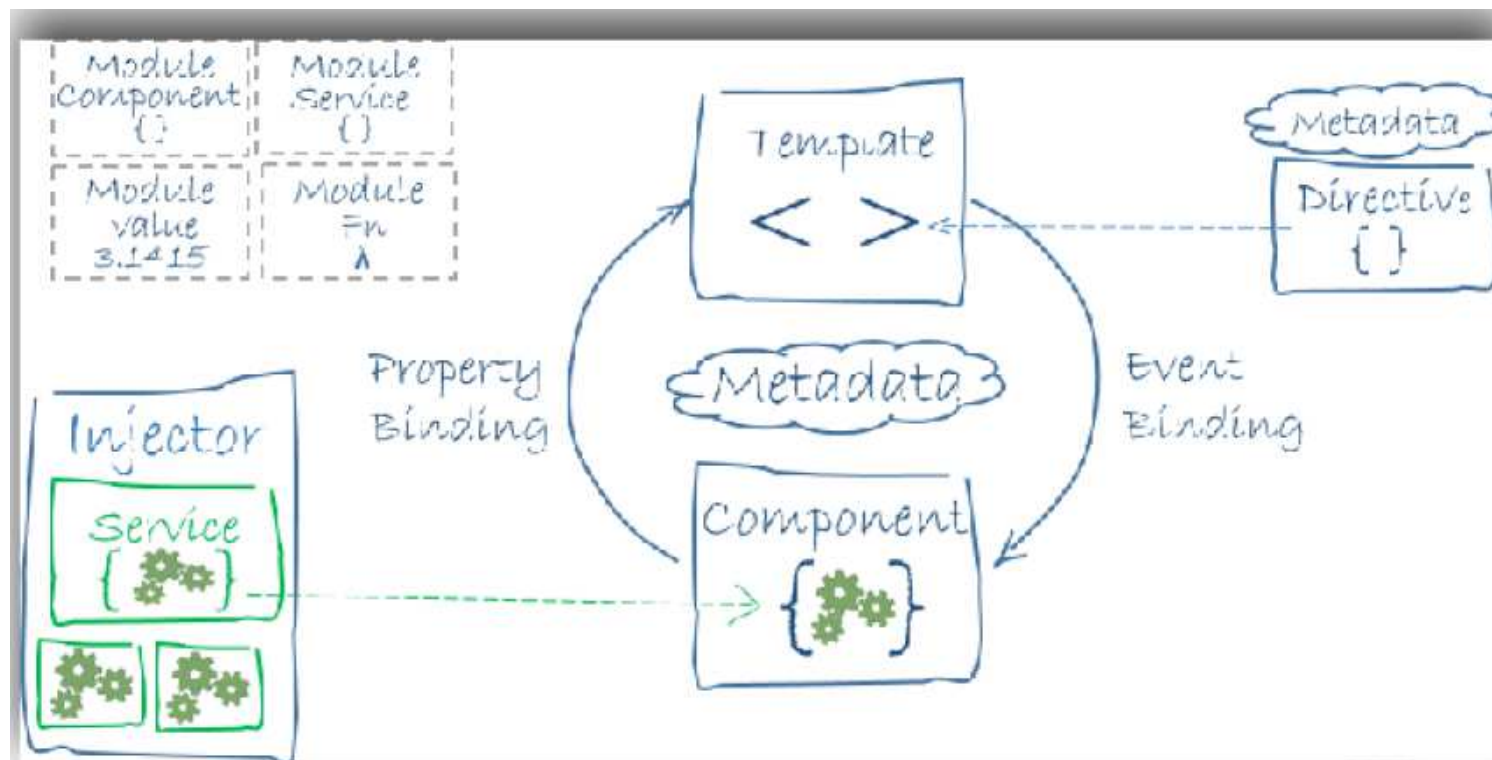




Angular : Arbre de composants

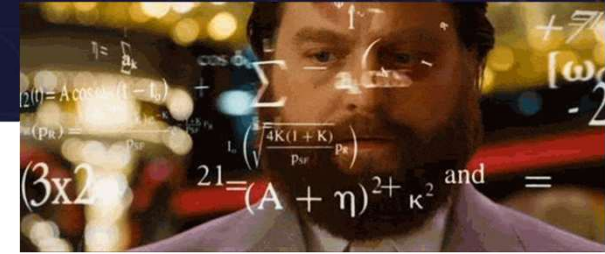


Architecture Angular





Installation d'Angular Angular Cli



- Nous allons installer notre première application en utilisant **angular Cli**.
- Si vous avez Node c'est bon, sinon, installer **NodeJs** sur votre machine. Vous devez avoir une version de **node nécessaire pour la version Angular que vous installez**.
- Une fois installé vous disposez de npm qui est le **Node Package Manager**. Afin de vérifier si vous avez NodeJs installé, tapez **npm -v**.
- Installer maintenant le Cli en tapant la : **npm install -g @angular/cli**
 - **npm install -g @angular/cli@20.3.14** installe la version **20.3.14**
 - **npm view @angular/cli** affiche la liste des versions de la cli
- Installer un nouveau projet à l'aide de la commande **ng new nomProjet**
- **npx @angular/cli@19.2.15** new projectName
- Afin d'avoir du help pour le cli tapez **ng help**
- Lancer le projet en utilisant la commande **ng serve**



Repo Github du Projet

- Pour notre formation, cloner ce projet dans lequel vous allez trouver les exemples et corrections des exercices:
 - `git clone https://github.com/aymensellaouti/ngM2i10626`



Ajouter Bootstrap

- On peut ajouter Bootstrap de plusieurs façons :
- 1- Via le CDN à ajouter dans le fichier index.html
- 2- En le téléchargeant du site officiel
- 3- Via npm
- npm install bootstrap –save
- Pour la majorité des bibliothèques, vous avez un fichier `package.json` avec les différents paths, vous permettant de ne pas spécifier le path complet, c'est le cas de bootstrap.
- `@import 'bootstrap';`
- Essayer la même chose avec font-awesome.

```
"main": "dist/js/bootstrap.js",  
"module": "dist/js/bootstrap.esm.js",  
"sass": "scss/bootstrap.scss",  
"style": "dist/css/bootstrap.css",
```





Angular

Les standalone composants, la nouvelle architecture d'angular

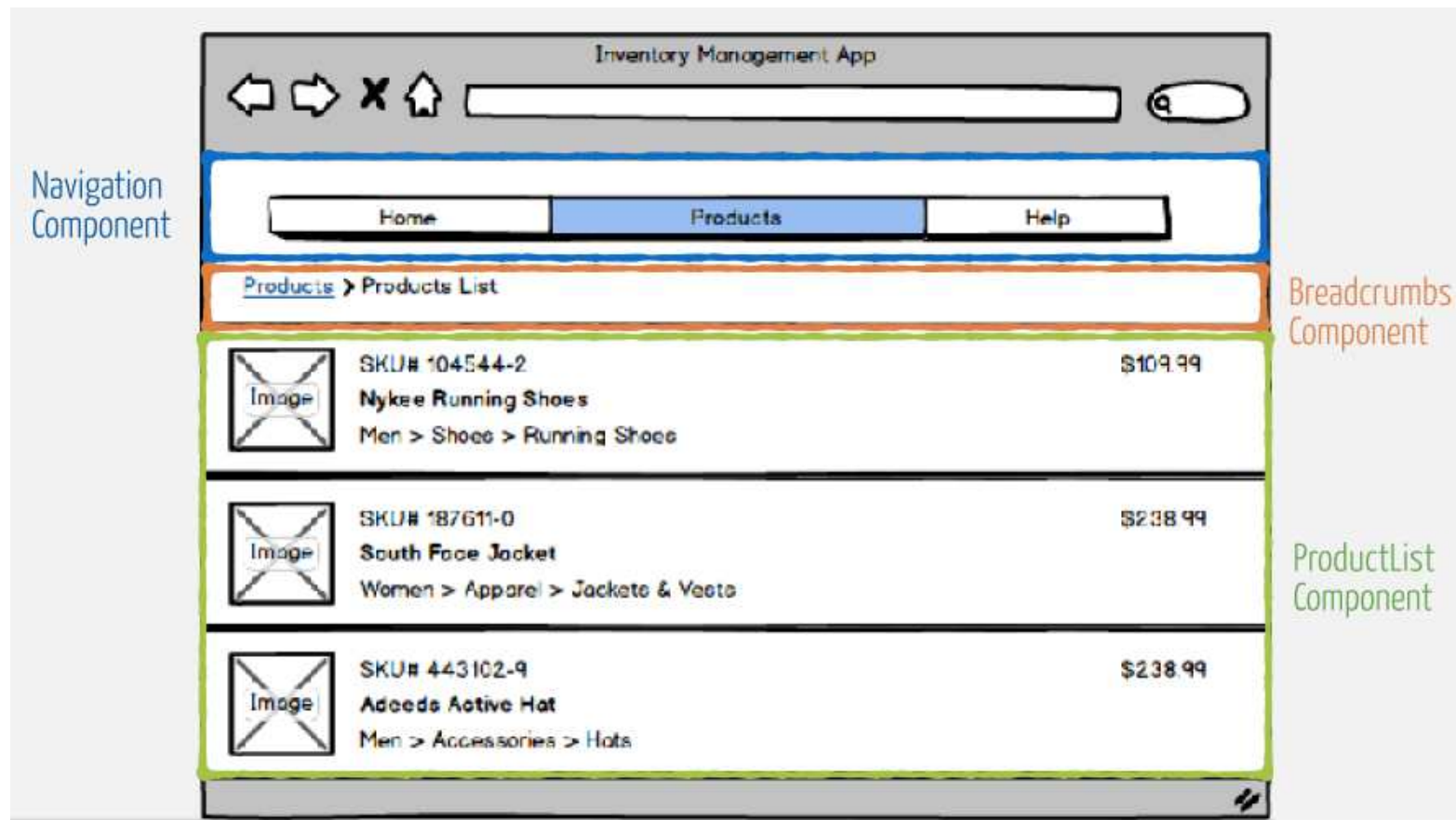


Qu'est ce qu'un composant (Component)

- Un **composant** est une **classe** qui permet de **gérer une vue**. Il se charge **uniquement** de cette vue là.
- Plus simplement, un composant est un fragment HTML géré par une classe JS (component en angular et controller en angularJS)
- Une application Angular est un **arbre de composants**
- La racine de cet arbre est l'application lancée par le navigateur au lancement.
- Un composant est :
 - **Composable** (normal c'est un composant)
 - **Réutilisable**
 - **Hiérarchique** (n'oublier pas c'est un arbre)
- NB : Dans le reste du cours les mots composant et component représentent toujours un composant Angular.



Quelques exemples



Quelques exemples

Product Row
Component



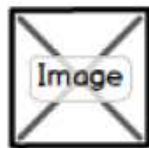
SKU# 104544-2
Nykee Running Shoes
Men > Shoes > Running Shoes

\$109.99



SKU# 187611-0
South Face Jacket
Women > Apparel > Jackets & Vests

\$238.99



SKU# 443102-9
Adeeds Active Hat
Men > Accessories > Hats

\$238.99



Quelques exemples

Product Image Component	Product Department Component	Price Display Component
	SKU# 104544-2 Nykee Running Shoes Men > Shoes > Running Shoes	\$109.99



Standalone Component

Le modèle mental

- Jusqu'à Angular14, un composant été toujours associé à un Module
- Même si **ce n'est pas géré de la même manière par le compilateur d'Angular**, vous pouvez voir un **standalone component comme un NgModule avec un seul composant**

```
@NgModule({
  declarations: [AppComponent],
  imports: [
    BrowserModule,
    AppRoutingModule,
    FormsModule,
  ],
})
export class AppModule {}
```

```
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css'],
})
export class AppComponent {}
```

```
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css'],
  imports: [
    RouterOutlet,
    HighlightDirective,
    CardComponent
  ],
  standalone: true,
})
export class AppComponent {}
```



Création d'un composant

➤ Pour créer un composant, lancer la commande :

`ng generate component my-new-component` ou son raccourci `ng g c my-new-component`





Standalone Component

Les composants autonomes

- Si vous avez travaillé avec une [version antérieure à Angular18](#). Afin de créer un *standalone* component, procéder comme vous faite d'habitude avec un composant et ajouter l'option `-standalone` (à partir d'Angular 18 c'est devenu le comportement par défaut).

```
@Component({  
  selector: 'app-standalone',  
  imports: [],  
  templateUrl: './standalone.component.html',  
  styleUrls: ['./standalone.component.css']  
})  
export class StandaloneComponent {}
```





Angular Le Binding



Que contient le composant ?

- Le composant dispose de deux parties :
 - La partie HTML qui représente la Vue
 - La partie TS décrivant l'état et le comportement du composant



L'état et le comportement d'un composant

- Et si on voulait afficher quelque chose dans notre template ?
- Et si on voulait contrôler un attribut de notre template ?
- Et si on voulait réagir à un événement qui survient dans notre template ?

```
export class FirstComponent {  
  //Propriétés : état State  
  name = 'First Component';  
  isHidden = false;  
  message = '';  
  //Méthodes : comportement Behaviour  
  showHide() {  
    this.isHidden = !this.isHidden;  
  }  
  changeMessage(newMessage: string) {  
    this.message = newMessage;  
  }  
}
```

Classe

```
<!-- attributs: L'état de la balise -->  
<p hidden> First Component works!</p>  
<div class="alert alert-info">  
  Je test le binding  
</div>  
<p>Le contenu de l'input est : </p>  
<input  
  type="text"  
  class="form-control"  
>  

```

Template

First Component works!

Je test le binding

Le contenu de l'input est :





Afficher des propriétés dans le Template

Interpolation

- L'interpolation permet de projeter des valeurs de propriétés dans votre template.
- Angular utilise la syntaxe "double accolades `{{ }}`" pour l'interpolation

```
export class FirstComponent {  
  //Propriétés : état State  
  
  name = 'First Component';  
  isHidden = false;  
  message = '';  
  //Méthodes : comportement Behaviour  
  showHide() {  
    this.isHidden = !this.isHidden;  
  }  
  changeMessage(newMessage: string) {  
    this.message = newMessage;  
  }  
}
```

```
<!-- attributs: l'état de la balise -->  
<p hidden>First Component works!</p>  
<p hidden>{{name}} works!</p>  
<div class="alert alert-info">  
  Je test le binding  
</div>  
<p>Le contenu de l'input est : </p>  
<input  
  type="text"  
  class="form-control"  
>  

```

First Component works!

Je test le binding

Le contenu de l'input est :





Contrôler un attribut de notre template Property Binding

- Afin de **contrôler un attribut d'une des balises de notre Template**, angular nous fournit le concept de **Binding de propriété** (**Property Binding**).
- C'est un Binding unidirectionnel.
- La propriété liée au composant est interprétée avant d'être ajoutée au Template.
- syntaxe: **[propriété]="varOuCte"**

```
<div [style.backgroundColor]="color">  
  Color  
</div>
```



Contrôler un attribut de notre template Property Binding

```
export class FirstComponent {  
  //Propriétés : état State  
  name = 'First Component';  
  isHidden = false;  
  message = '';  
  imagePath = 'assets/images/as.jpg';  
  //Méthodes : comportement Behaviour  
  showHide() {  
    this.isHidden = !this.isHidden;  
  }  
  changeMessage(newMessage: string) {  
    this.message = newMessage;  
  }  
}
```

```
<!-- attributs: L'état de la balise -->  
<p hidden>{{name}} works!</p>  
<p [hidden]="isHidden"> {{name}} works!</p>  
  
<div class="alert alert-info">  
  Je test le binding  
</div>  
<p>Le contenu de l'input est : {{ message}}</p>  
<input  
  type="text"  
  class="form-control"  
>  
  
<img [src]="imagePath" alt="aymen">
```

First Component works!

Je test le binding

Le contenu de l'input est :





Réagir à un événement qui survient dans notre template : Event Binding

- Afin **d'écouter et de réagir** à **un événement déclenché dans notre Template**, angular nous fournit le concept de **Binding d'événement** (**Event Binding**).
- C'est un **binding unidirectionnel**, Il permet d'interagir du DOM vers le composant. L'interaction se fait à travers les **événements**.
- Syntaxe : **(evenement)**="methodeAExecuter() ">

```
<a (click)="goToCv()" >Go to Cv</a>
```





Réagir à un événement qui survient dans notre template : Event Binding

```
export class FirstComponent {  
  //Propriétés : état State  
  name = 'First Component';  
  isHidden = false;  
  message = '';  
  //Méthodes : comportement Behaviour  
  showHide() {  
    this.isHidden = !this.isHidden;  
  }  
  changeMessage(newMessage: string) {  
    this.message = newMessage;  
  }  
}
```

```
<!-- attributs: l'état de la balise -->  
<p [hidden]="isHidden"> {{name}} works!</p>  
<!-- Au click appelle la fonction showHide -->  
<div (click)="showHide()" class="alert alert-info">  
  Je test le binding  
</div>  
<p>Le contenu de l'input est : {{ message}}</p>  
<input  
  type="text"  
  class="form-control"  
>
```





TEMPLATE REFERENCE

- Dans certains cas d'utilisation, vous avez besoin de récupérer **la référence d'un objet du DOM dans votre template**. Par exemple la référence d'un input pour accéder à sa valeur.
- Pour ce faire, on utilise le symbole # pour la création d'une variable de référence.

```
export class FirstComponent {  
  //Propriétés : état State  
  name = 'First Component';  
  isHidden = false;  
  message = '';  
  //Méthodes : comportement Behaviour  
  showHide() {  
    this.isHidden = !this.isHidden;  
  }  
  changeMessage(newMessage: string) {  
    this.message = newMessage;  
  }  
}
```

Classe

```
<!-- attributs: L'état de la balise -->  
<p hidden> First Component works!</p>  
<div class="alert alert-info">  
  Je test le binding  
</div>  
<p>Le contenu de l'input est : </p>  
<input  
  #myInput  
  (change)="changeMessage(myInput.value)"  
  type="text"  
  class="form-control"  
>  

```

Template

First Component works!

Je test le binding

Le contenu de l'input est :





Exercice

- Créer un nouveau composant. Ajouter y une Div et un input de type texte.
- Fait en sorte que lorsqu'on écrit une couleur dans l'input, ça devienne la couleur du Div.
- Ajouter un bouton. Au click sur le bouton, il faudra que le Div reprenne sa couleur par défaut.
- Ps : pour accéder au contenu d'un élément du Dom utiliser `#nom` dans la balise et accéder ensuite à cette propriété via le `nom`.
- Pour accéder à une propriété de style d'un élément on peut binder la propriété `[style.nomPropriété]` exemple `[style.backgroundColor]`



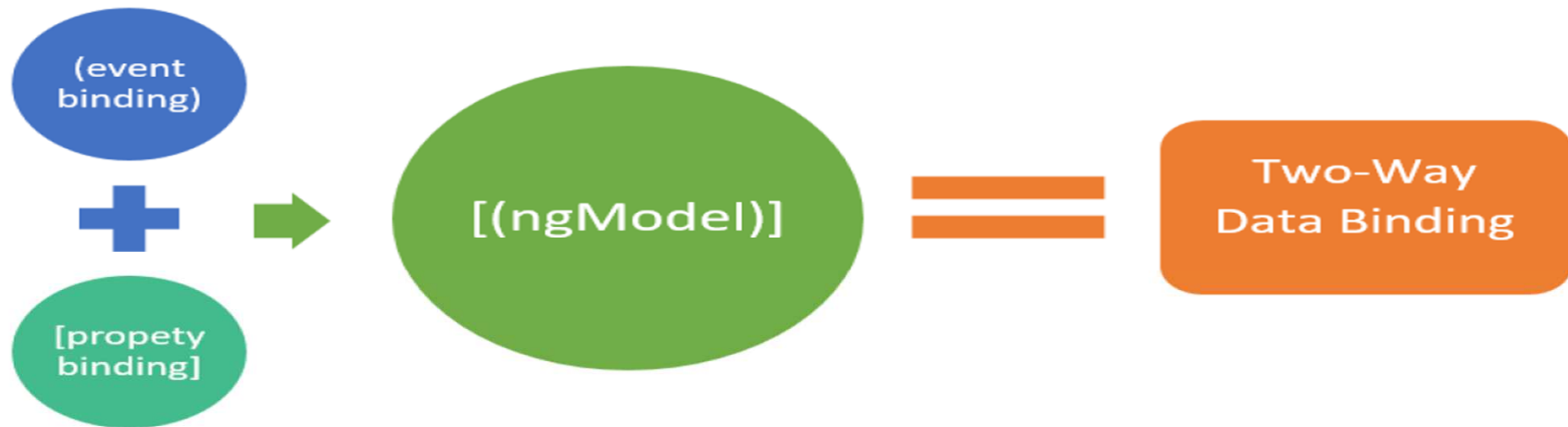
Two way Binding

- Binding Bi-directionnel
- Permet d'interagir du Dom vers le composant et du composant vers le DOM.
- Se fait à travers la directive `ngModel` (on reviendra sur le concept de directive plus en détail)
- Syntaxe :
 - `[(ngModel)]=property`
 - Afin de pouvoir utiliser `ngModel` vous devez importer le `FormsModule`



Two way Binding

PROPERTY BINDING + EVENT BINDING



```
<hr>  
Change me <input [(ngModel)]="nom">  
<br>My new name is {{nom}}
```

Template





Property Binding et Event Binding

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-two-way',
  templateUrl: './two-way.component.html',
  styleUrls: ['./two-way.component.css']
})
export class TwoWayComponent {
  two:string="myInitValue";
}
```

Component

```
<hr>
Change me if u can
<input
  [(ngModel)]="two">
<br>
it's always me :d
{{two}}
```

Template



Exercice

- Le but de cet exercice est de créer un aperçu de carte visite.
- Créer un composant
- Préparer une interface permettant de saisir d'un côté les données à insérer dans une carte visite. De l'autre côté et instantanément les données de la carte seront mises à jour.
- Préparer l'affichage de la carte visite. Vous pouvez utiliser ce thème gratuit :
 - <https://www.creative-tim.com/product/rotating-css-card>





Aymen Sellaouti
trainer

"I'm the new Sinatra, and since I made it here I can make it anywhere, yeah,
they love me everywhere"

 Auto Rotation

name :

firstname :

job :

path :

Two Way Binding



A user profile card for Sellaouti Aymen. It features a circular profile picture of a man with a beard. Below the picture, the name 'Sellaouti Aymen' is displayed, followed by the title 'Enseignant'. A quote 'tant qu'il y a de la vie il y a de l'espoir' is shown in a light blue box. At the bottom, there is a button labeled 'Auto Rotation'.

Nom :
Sellaouti

Prénom :
aymen

Job :
Enseignant

image :
as.jpg

Citation Favorite :
tant qu'il y a de la vie il y a de l'espoir

Décrivez nous votre travail :
J enseigne aux étudiants les technos du Web

Mots clé de votre travail :
HTML CSS JS PHP Symfony Angular

Two Way Binding



A user profile card for Sellaouti Aymen, similar to the first one but with additional social media statistics. It features a quote 'To be or not to be, this is my awesome motto!' at the top. The main text 'J enseigne aux étudiants les technos du Web' is followed by 'HTML CSS JS PHP Symfony Angular'. Below this, three statistics are shown: 235 Followers, 114 Following, and 35 Projects. At the bottom, there are social media icons for Facebook, Google+, and Twitter.

Nom :
Sellaouti

Prénom :
aymen

Job :
Enseignant

image :
as.jpg

Citation Favorite :
tant qu'il y a de la vie il y a de l'espoir

Décrivez nous votre travail :
J enseigne aux étudiants les technos du Web

Mots clé de votre travail :
HTML CSS JS PHP Symfony Angular

Résumé : Property Binding



DOM

`{{expression}}`

Interpolation

`[property] = "expression"`

One Way Binding

`(event) = "statement"`

Event Binding

`[(ngModel)] = "property"`

Two Way Binding



Component



Les Signaux



Angular Signals





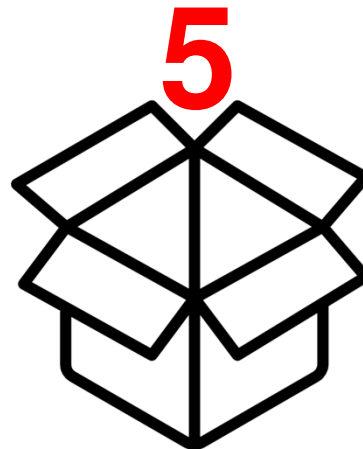
Qu'est ce qu'un Signal ?



- Un signal agit comme une **enveloppe** (wrapper) autour d'une valeur, ayant la capacité **d'avertir les consommateurs lorsque la valeur change**.
- On peut modéliser ça en disant qu'un signal c'est une donnée qui a le pouvoir de notifier ses changements.



`value = signal(5)`



`value()`



Ma nouvelle valeur est 7

`value.set(7)`





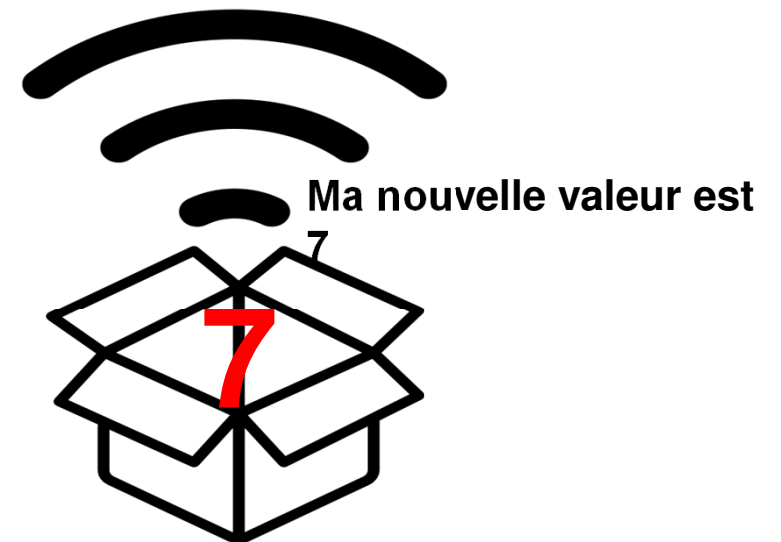
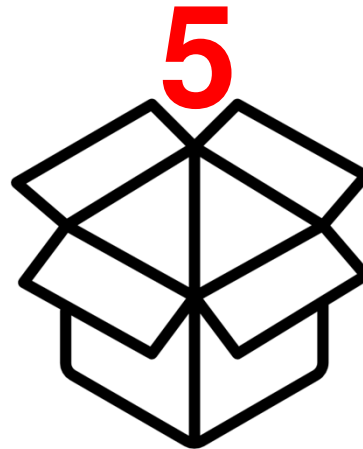
Qu'est ce qu'un Signal ?



- Un signal est une **primitive réactive** qui représente une valeur et qui nous permet de
 - **suivre ses changements** au fil du temps.
 - **modifier** cette même valeur en **notifiant tous ceux qui en dépendent**.
- Elle permet de définir l'état réactif de votre application et d'avoir une **identification précise de quels composants sont impactés par un changement**.



- Angular propose trois principales primitives pour utiliser les signaux :
 - signal
 - computed
 - effect

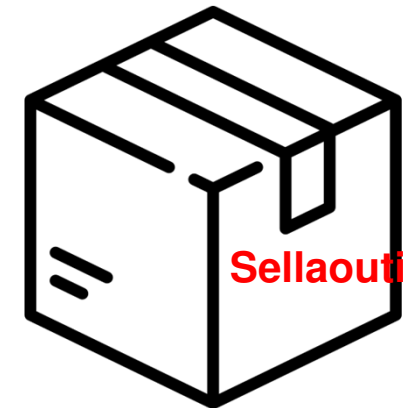




Créer un signal via l'api signal()

- La fonction **signal** permet d'initialiser une variable et d'informer Angular et son contexte à chaque fois que sa valeur change.
- Elle retourne un objet de type **WritableSignal**.
- **Un signal doit avoir une valeur initiale**

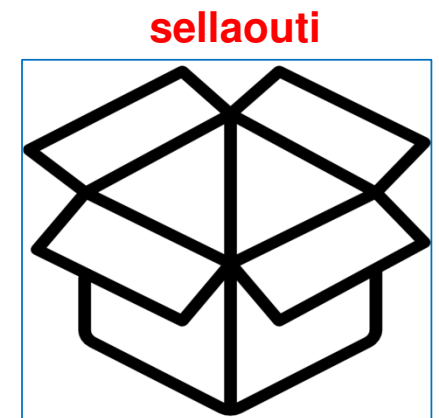
```
@Component({
  selector: 'app-signal-api',
  standalone: true,
  imports: [],
  styleUrls: ['./signal-api.component.css'],
  template: ` <h1>Hello World</h1> `,
})
export class SignalApiComponent {
  lastname: WritableSignal<string> = signal('sellaouti');
}
```



Récupérer la valeur d'un signal

- Afin de récupérer la valeur d'un signal il suffit de l'appeler comme une fonction.

```
@Component({
  selector: 'app-signal-api',
  standalone: true,
  imports: [],
  styleUrls: ['./signal-api.component.css'],
  template: `<h1>Hello {{ lastname() }}</h1>`,
})
export class SignalApiComponent {
  lastname: WritableSignal<string> = signal('sellaouti');
}
```





Les méthodes de modifications de signal : set() et update()

- Pour modifier la valeur d'un **signal**, on peut passer par :
- La Méthode **set**, qui permet d'affecter une nouvelle valeur au signal.
- La méthode **update**, qui permet de calculer une nouvelle valeur d'un signal en fonction de sa valeur précédente.

```
@Component({
  selector: 'app-signal-api',
  standalone: true,
  imports: [],
  template: `
    <h1>Hello {{ lastname() }}</h1>
    <input type="number" #input
      (change)="setCounter(+input.value)"
    />
    <h2 (click)="increment()">
      Click here. You clicked {{ counter() }} times
    </h2>
  `,
})
export class SignalApiComponent {
  lastname = signal('aymen');
  counter = signal(0);
  increment() {
    this.counter.update((currentValue) => currentValue + 1);
  }
  setCounter(val: number) {
    this.counter.set(val);
  }
}
```



Cas d'utilisation

- Soit le composant SumComponent. Que se passe-t-il si on incrémente x ou y ?

```
export class SumComponent {  
  x = 3;  
  y = 5;  
  z = this.x + this.y;  
}
```

```
<input type="number" [(ngModel)]="x" class="form-control">  
<input type="number" [(ngModel)]="y" class="form-control">  
{{ x }} + {{ y }} = {{ z }}
```



computed()

- L'api `computed()` permet de créer un **nouveau signal** dont la valeur **dépend d'autres signaux**.
- Lorsqu'un **signal est mis à jour**, tous ses signaux dépendants seront alors **automatiquement mis à jour**.
- On note que `computed()` retourne un objet de type `Signal` et non `WritableSignal`.
- Vous **ne pouvez pas modifier un computed**, il n'est pas `Writable`

```
lastname = signal('aymen');  
firstname = signal('sellaouti');  
fullname = computed(() => `${this.firstname()} ${this.lastname()}`)
```





computed()

Modifier un signal dans un computed

- computed() s'attend à ce que le calcul soit léger et rapide, et n'entraîne aucun effet secondaire.
- Vous ne pouvez pas modifier un signal dans un computed, ceci provoquera une erreur.
- Ne modifiez aucune variable dans les computed ; pas seulement les signaux, mais aussi les variables extérieures à la fonction.
- Ne modifiez pas le DOM et n'exécutez pas de tâches asynchrones.

```
fullname = computed(() => {  
  console.log('i am computing....');  
  this.firstname.set('ccc');  
  return `${this.firstname()} ${this.lastname()}`;  
});
```

```
✖ ▶ ERROR Error: NG0600: Writing to VM1270:1  
signals is not allowed in a `computed` or  
an `effect` by default. Use  
`allowSignalWrites` in the  
`CreateEffectOptions` to enable this inside  
effects.
```



- Pour identifier un signal qui a changé, et donc si on doit exécuter un computed, Angular utilise [Object.is](#).
- Object.is() permet de déterminer si deux valeurs sont identiques. Deux valeurs sont considérées identiques si :
 - elles sont toutes les deux undefined
 - elles sont toutes les deux null
 - elles sont toutes les deux true ou toutes les deux false
 - elles sont des chaînes de caractères de la même longueur et avec les mêmes caractères (dans le même ordre)
 - **elles sont toutes les deux le même objet (même référence)**
 - elles sont des nombres et
 - sont toutes les deux égales à +0
 - sont toutes les deux égales à -0
 - sont toutes les deux égales à NaN

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/is





computed()

Exclure un signal de l'arbre de dépendance

- Si, pour une raison ou une autre, vous voulez lire une valeur d'un signal dans un computed mais sans l'intégrer dans l'arbre de dépendance, vous pouvez utiliser l'Api [untracked](#).
- Dans cet exemple le computed fullname ne sera mis à jour que si le signal firstname change

```
lastname = signal('sellaouti');  
firstname = signal('aymen');  
fullname = computed(() => `${this.firstname()} ${untracked(this.lastname)}`);
```





computed()

Les computed, autres propriétés

- Les computed sont paresseux (*lazy*), ce qui signifie que computed *n'est invoquée que lorsque quelqu'un s'intéresse (lit) à sa valeur*. Cela permet *d'optimiser les performances* en évitant les calculs inutiles.
- Les computed sont *automatiquement supprimés* lorsque la référence du signal calculée devient hors de portée.
- Cela garantit que les ressources *sont libérées et qu'aucune opération de nettoyage explicite n'est requise*.
- Ceci est due à l'utilisation des `weakReference` avec les signaux



Exercice

- Créer un composant TTC qui permet de calculer le prix TTC d'un produit selon le nombre de pièces et la TVA. Sachant que l'utilisateur peut changer toutes les valeurs et que par défaut la quantité est de 1 le prix est de 0 et la tva est de 18.
- Si le nombre de pièces est entre 10 et 15 une remise de 20% est appliquée.
- Si le nombre de pièces est supérieur à 15 une remise de 30% est appliquée.

TTC Calculator

100	10	18
Prix HT	Quantité	TVA
Prix unitaire TTC : \$118.00	Prix total TTC : \$1,180.00	Discount : \$0.00



L'API effect()

- Dans certains cas d'utilisation, vous avez besoin d'être notifié par le changement d'un signal pour **effectuer un effet de bord**, donc faire un **traitement sans pour autant créer un nouveau signal ou en modifier d'autres**.
- Pensez à un log, update du localStorage,...
- L'effect va être réexécuté si l'un des signaux qu'il utilise émet une nouvelle valeur.



L'API effect()

- Vous pouvez **créer un effet** via la fonction **effect**.
- Les effets s'exécutent toujours **au moins une fois**.
- Lorsqu'un effet est exécuté, il **suit tous les signaux qu'il contient**. Chaque fois que l'une de ces valeurs de **signal change**, l'effet se reproduit.
- L'effet est **semblable aux computed**, les effets gardent une **trace de leurs dépendances de manière dynamique** et ne suivent que les **signaux** qui ont été **lus lors de l'exécution la plus récente**.

```
constructor() {  
  effect(() => {  
    console.log(`count:${this.counter()}`);  
  });  
}
```

```
private logEffect = effect(() => {  
  console.log(`  
    The current count is:${this.counter()}`  
  );  
});
```





Quand utiliser un signal et non une variable standard ?

- Utilisez les signals
 - pour chaque **variable d'état** de votre composant dont la valeur change.
 - lorsque vous voulez **suivre les changements** de votre variable
 - quand la **valeur de votre variable** doit **changer quand d'autres variables changent**
- N'utilisez pas les signals
 - quand la **valeur ne change pas**
 - Quand c'est une **variable locale** que vous **n'utilisez pas dans votre ui** ou dans des **calculs réactifs**
 - pour les **événements**
 - pour les **opérations asynchrones**





Angular

Le nouveau Control Flow





Angular 17

Le nouveau flux de control

- Pour pouvoir utiliser des structures conditionnelles et itératives dans les vues de vos composants, Angular propose depuis sa version 17 un nouveau Control Flow.
- Les flux suivants ont été proposés :
 - @If
 - @for
 - @switch





Control flow

@if

- `@if` est associé à une expression booléenne. Si cette expression est fausse (false) alors l'élément et son contenu sont retirés du DOM, ou jamais ajoutés.
- `@if(condition)`
 - Si le booléen est true alors l'élément host est visible.
 - Si le booléen est false alors l'élément host est caché.

```
@if (authService.isAuthenticated()) {  
  <button  
    (click)="deleteCv(cv)"  
    class="btn btn-danger">  
    Delete  
  </button>  
}
```





Control flow

@if, @else if et @else

- @if peut également être utilisé avec @else if et/ou @else selon le besoin.
- La syntaxe est très intuitive, demandé vous comment vous aurez fait en Javascript et ajoutez un @ :D.

```
@if (connectedUser) {  
    Hello {{ connectedUser.name }}  
} @else {  
<div>Merci de vous connectez</div>  
}
```





Control flow

@if, @else if et @else

```
@if (!connectedUser) {  
  <div class="alert alert-danger">Merci de vous connectez</div>  
} @else if (!connectedUser.activated) {  
  <div class="alert alert-warning">  
    Hello {{ connectedUser.name }}, merci d'activer votre compte  
  </div>  
} @else {  
  <div class="alert alert-success">Hello {{ connectedUser.name }}</div>  
}
```





Control flow

@for

- La directive structurelle `@for` permet de boucler sur un itérable et d'injecter les éléments dans le DOM.

```
<ul>
  @for (episode of episodes; track episode.id) {
    <li>{{ episode.title }}</li>
  }
</ul>
```





Control flow

@For

- **@for** fournit certaines informations sur la boucle en cours :
 - **\$index**: position de l'élément.
 - **\$odd**: true si l'élément est à une position impaire.
 - **\$even**: true si l'élément est à une position paire.
 - **\$first**: true si l'élément est à la première position.
 - **\$last**: true si l'élément est à la dernière position.

```
<ul>
  @for (episode of episodes; track episode.id) {
    <li>
      Episode {{ $index + 1 }} : {{ episode.title }}
    </li>
  }
</ul>
```





Control flow

@For : track

- Avec @For la fonction de tracking est devenue obligatoire
- La fonction de suivi créée via l'instruction `track` est utilisée pour permettre au mécanisme de détection des changements d'Angular de savoir exactement quels éléments mettre à jour dans le DOM après les modifications de l'itérable d'entrée.
- La fonction de suivi indique à Angular comment identifier de manière unique un élément de la liste.

```
@for (episode of episodes; track episode.id) {  
  <li>{{ episode.title }}</li>  
}
```





Control flow

@For : track

- En principe, il devrait toujours y avoir quelque chose d'unique dans les éléments sur lesquels vous itérer.
- Dans le pire des cas, s'il n'y a rien d'unique dans les éléments du tableau, vous pouvez utiliser `$index` de l'élément, c'est-à-dire la position de l'élément dans le tableau.

```
@for (episode of episodes; track $index) {  
  <li>{{ episode.title }}</li>  
}
```





Control flow

@For, la gestion d'un itérable vide

- Afin de gérer le cas où votre **itérable est vide**, nous avons le block **@empty**.
- Ce bloque **n'est activé** que si **l'itérable** sur lequel vous bouclez est **vide**.

```
<ol class="list-group">
  @for (player of players; track player.id) {
    <li class="list-group-item">{{player.name}}</li>
  }
  @empty {
    <li class="list-group-item list-group-item-danger">La liste ne nous est
pas encore parvenue</li>
  }
</ol>
```





Control flow

@switch

- Avec `@switch`, vous pouvez créer des switch très simplement.
- Vous avez les trois opérateurs : `@switch`, `@case`, `@default`
 - `@switch` : définir l'élément sur lequel switcher
 - `@case` : pour identifier le cas
 - `@default` : pour les valeurs par défaut

```
@switch(streamingService) {  
  @case ('Disney+') {  
    <div>'Mandalorian'</div>  
  } @case ('AppleTV') {  
    <div>'Ted Lasso'</div>  
  } @default {  
    <div>'Peaky Blinders'</div>  
  }  
}
```



Exercice

- Créer un nouveau composant names.
- Définissez un tableau de noms
- Affichez la liste des noms et faites en sorte que les noms, dont la taille dépasse 10, soit suivie d'une icône de votre choix

sellaouti

lewandowski 

dupont

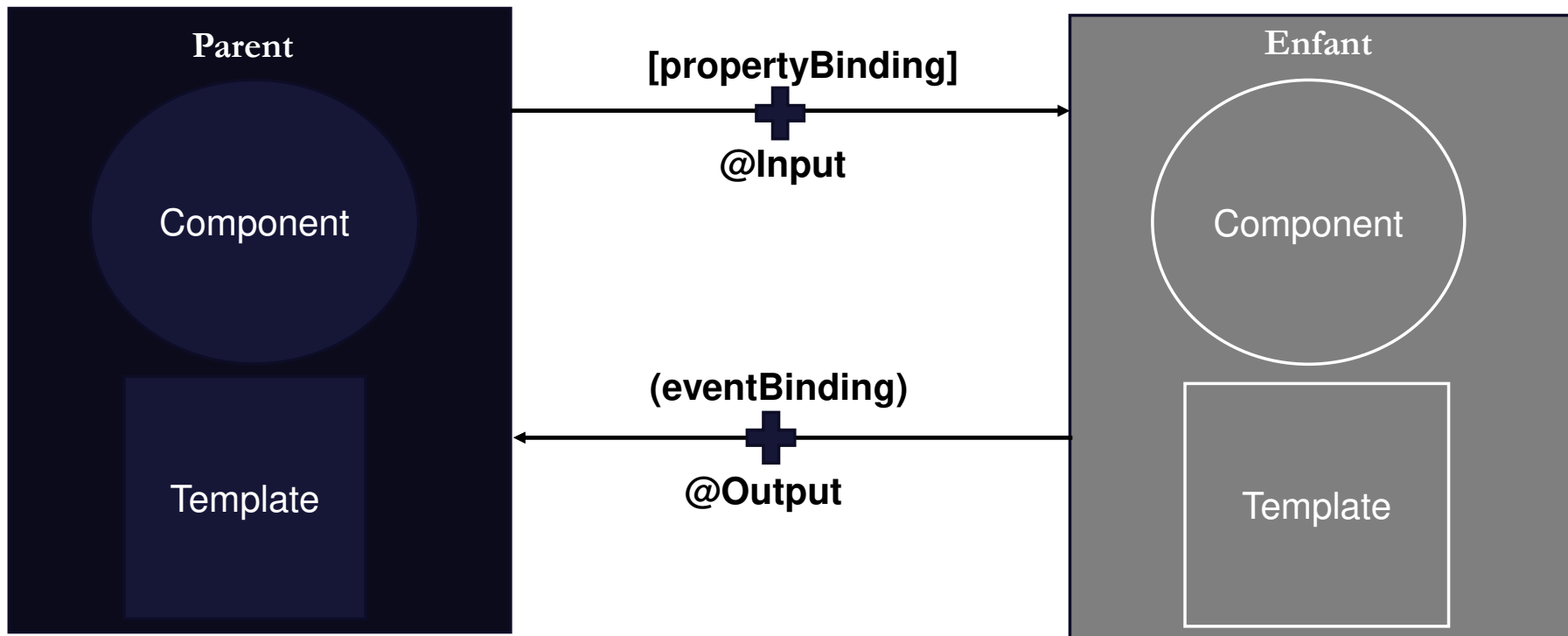




Interaction entre composants

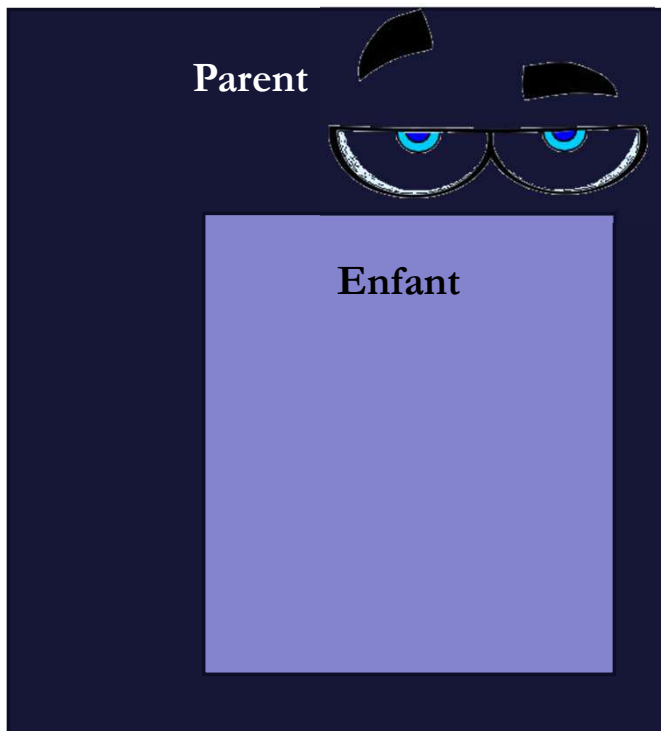


Interaction entre composants



Pourquoi ?

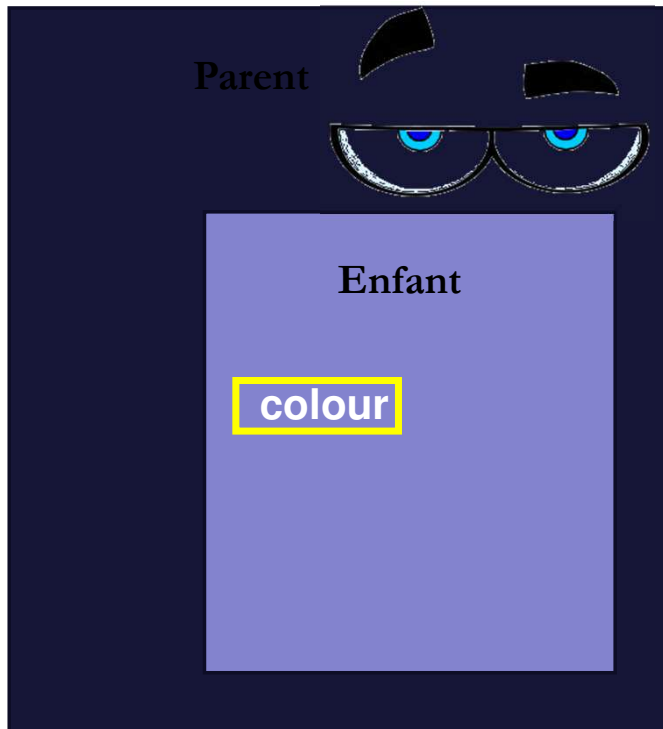
Le père voit le fils, le fils ne voit pas le père



```
import { Component } from '@angular/core';

@Component({
  selector: 'app-pere',
  template: `
    <p>Je suis le composant père </p>
    <forma-fils></forma-fils>
  `,
  styles:
})
export class PereComponent {
  color = 'green';
}
```

Interaction du père vers le fils



- Le parent voit l'enfant mais ne peut pas voir ses propriétés.
- Solution : Faire du property binding avec @input, qui peut prendre un objet d'options en paramètre (pour spécifier que l'envoi d'une valeur est required par exemple).

```
import { Component, OnInit, Input, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-child-first',
  templateUrl: './child-first.component.html',
  styleUrls: ['./child-first.component.css']
})
export class ChildFirstComponent implements OnInit {
  colour:string;
```

```
<app-fils [colour]="color"/>
```

Template du pere

Ts du fils



Signal Input

- La version [17.1](#) a vu Angular introduire le concept de *Signal Input* pour améliorer l'interaction entre les composants.
- Les Signal Input permettent de lier les valeurs des composants parents. Ces valeurs sont exposées à l'aide d'un signal

```
import { Input, input } from '@angular/core';  
isEvenn!: boolean;  
// Sans Signal Input  
@Input() isEven: number;  
// Avec les signal Input  
counter = input<number>();
```



Signal Input

- Comme les `@Input`, le Signal Input peut être optionnel ou `required`
- Il peut avoir une `valeur par défaut`
- Il peut avoir une `Alias`
- Et vous pouvez le `transformer`

```
// optional
counter = input<number>();
// Valeur par défaut = 0
counter = input<number>(0);
// Required
counter = input.required<number>();
```

```
counter = input(0,{
  alias: 'counter',
  transform: (value: number) => value * 100,
});
```

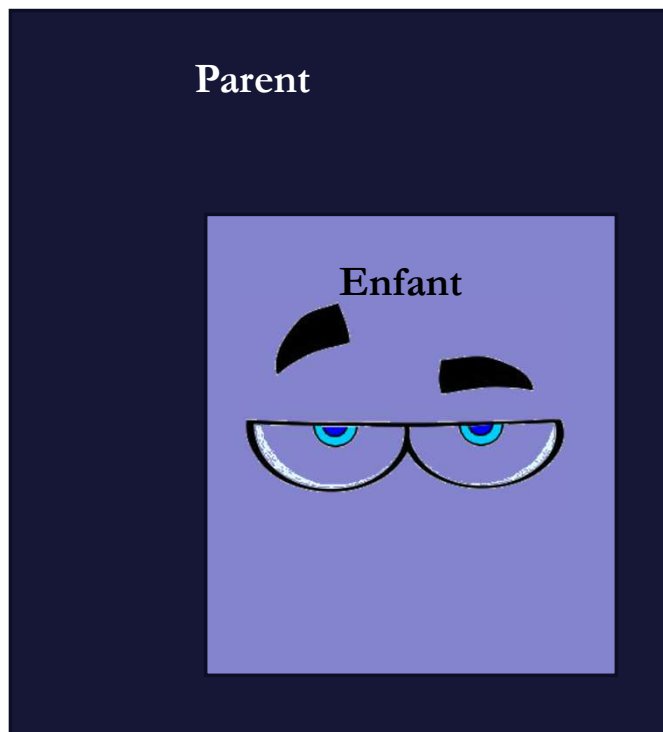
```
counter = input.required({
  alias: 'counter',
  transform: (value: number) => value * 100,
});
```

```
// avant
@Input({required: true})
Counter!:number;
// après
counter = input.required<number>();
```



Interaction du fils vers le père

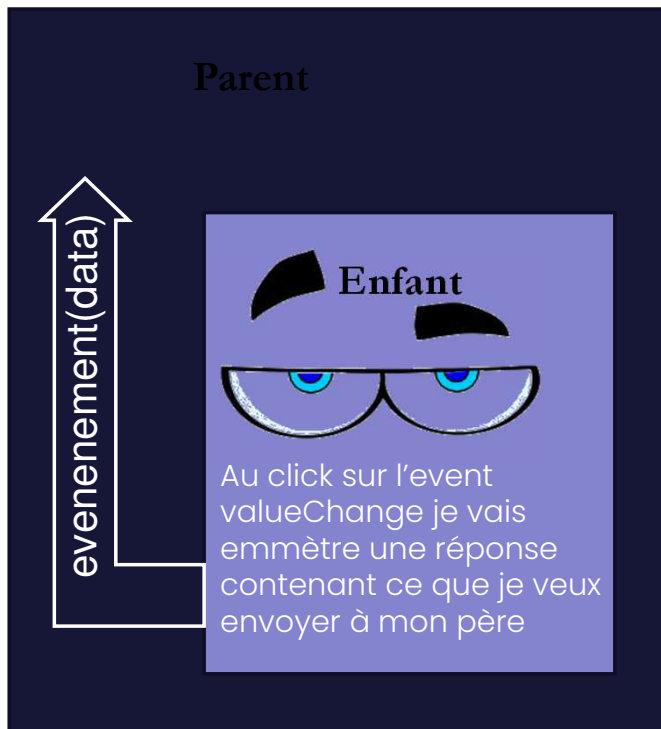
L'enfant ne peut pas voir le parent. Appel de l'enfant vers le parent. Que faire ?



```
import {Component} from '@angular/core';
@Component({
  selector: 'bind-output',
  template: `Bonjour je suis le fils`,
  styleUrls: ['./output.component.css']
})
export class OutputComponent {
  valeur:number=0;
}
```



Interaction du fils vers le père



- L'enfant ne voit pas le parent car il ne sait simplement pas par quel composant il a été appelé.
- Solution : 1 Faire du event binding avec @output

```
import { Component, OnInit, Input, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-child-first',
  templateUrl: './child-first.component.html',
  styleUrls: ['./child-first.component.css']
})
export class ChildFirstComponent implements OnInit {
  @Output() sendRequest = new EventEmitter();
}
```

Template du fils

<app-fils [colour]="color"/>

Template du père



Interaction du fils vers le père

2. Configurer l'événement

```
sendEvent() {  
  this.sendRequest.emit('Accuse la réception de la couleur ' + this.color);  
}
```

3. Récupérer l'évènement dans le composant parent

```
<app-child-first (sendRequest)="ReceivedEvent($event)"></app-child-first>
```

4. Traiter le message reçu de la part du composant enfant

```
ReceivedEvent(msg : any)  
{  
  alert(msg);  
}
```



Signal Output

- L'API `output()` remplace directement le décorateur `@Output()` traditionnel.
- Le décorateur `@Output` n'est pas obsolète
- Angular a donc ajouté `output()` comme nouvelle façon de définir les sorties des composants dans Angular, d'une manière plus sûre et mieux intégrée à RxJs que l'approche traditionnelle `@Output` et `EventEmitter`.
- La syntaxe a été simplifiée.

```
<app-item (selectCv)="onSelectCv($event)" [cv]="cv" />
```

Pere

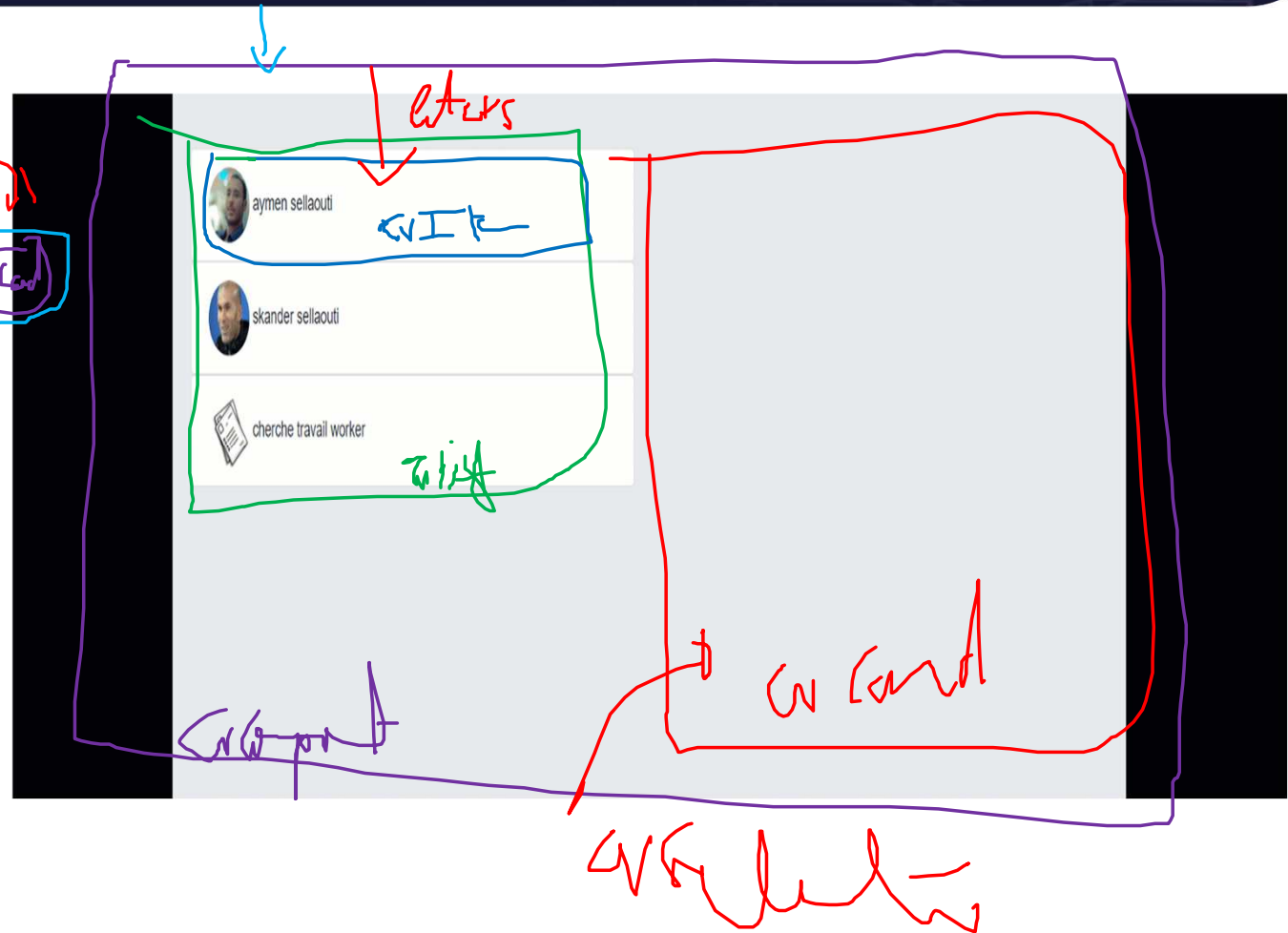
```
/* Sans les signal Output */  
@Output() oldSelectCv = new EventEmitter<Cv>();  
/* Avec les signal Output */  
selectCv = output<Cv>();  
onClick() {  
  if (this.cv) {  
    /* Cette partie du code reste la même */  
    this.selectCv.emit(this.cv);  
  }  
}
```

Fils



Exercice

- Le but de cet exercice est de créer une mini plateforme de recrutement.
- La première étape est de créer la structure suivante avec une vue contenant deux parties :
 - Liste des Cvs inscrits
 - Détail du Cv qui apparaîtra au click
- Il vous est demandé juste d'afficher un seul Cv et de lui afficher les détails au click.

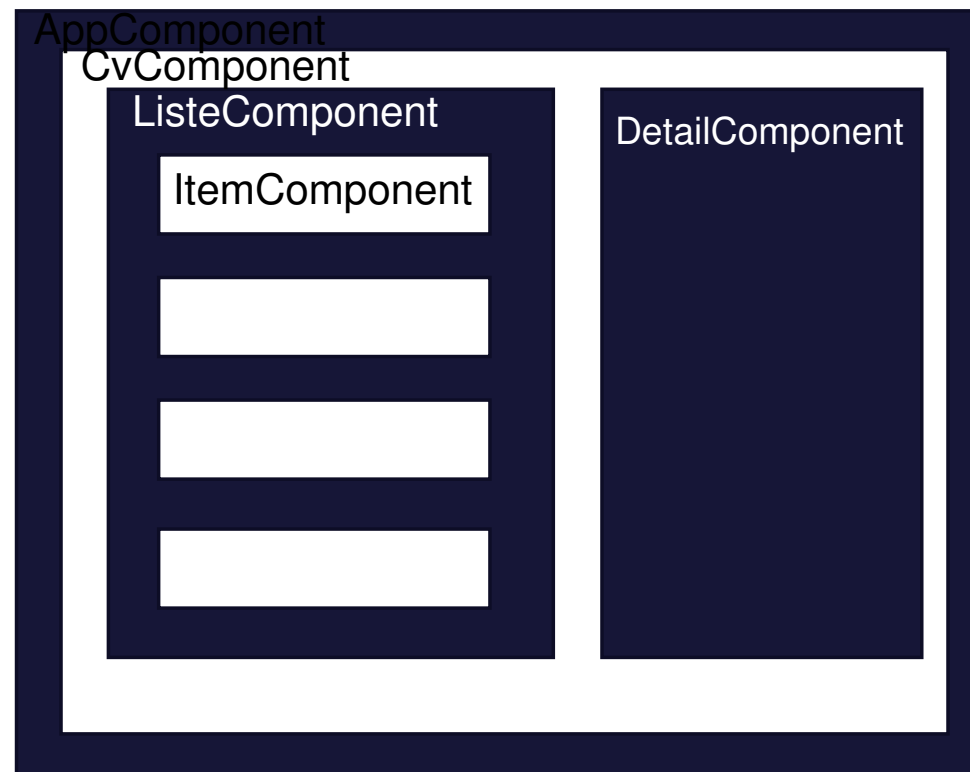


Exercice

- Voici la décomposition de l'interface en composant

Un cv est caractérisé par :

id
name
firstname
Age
Cin
Job
path





Angular

Les directives



1. Comprendre la définition et l'intérêt des directives.
2. Voir quelques directives d'attributs offertes par angular et savoir les utiliser
3. Créer votre propre directive d'attributs
4. Voir quelques directives structurelles offertes par angular et savoir les utiliser



Qu'est ce qu'une directive

- Imaginons qu'au survol d'un élément de votre composant vous voulez qu'il grandisse, qu'il change de couleur ou n'importe quelle réaction.
- La première idée est d'aller implémenter la logique dans le composant et ça marche.
- Deux jours plus tard le même besoin apparait dans un autre composant, on doit refaire la même chose.
- Encore le même besoin...
- Est-ce que ça pose un problème ?



Qu'est ce qu'une directive

- Une directive est une **classe annotée avec le décorateur @Directive()** qui permet à Angular **d'attacher un comportement spécifique à un élément HTML**.
- Les directives permettent :
 - **Réutilisabilité du code : en centralisant** un comportement ou un style **réutilisable** dans toute l'application.
 - **Séparation des préoccupations (Separation of concerns) :**
La logique **métier est séparée du design** ou de l'affichage.
 - **Clarté et lisibilité :**
Plutôt qu'un code lourd dans le composant, la directive gère une fonctionnalité spécifique (exemple : gestion de permissions d'un bouton).
 - **Personnalisation facile :**
Tu peux créer des directives sur mesure pour enrichir le comportement de tes composants

Qu'est ce qu'une directive

- La documentation officielle d'Angular identifie trois types de directives :
 - Les **composants** qui sont des directives avec des templates.
 - Les **directives structurelles** qui changent **l'apparence** du **DOM** en ajoutant et supprimant des éléments.
 - Les **directives d'attributs** (attribute directives) qui permettent de changer **l'apparence** ou le **comportement** d'un **élément**.
- Il existe des **directives fournies par Angular** mais vous pouvez **créer vos propres directives**



Les directives structurales

- Une **directive** structurale permet de modifier le DOM.
- Elles sont appliquées sur **l'élément HOST**.
- Elles sont généralement précédées par le **préfix ***.
- Les directives les plus connues (**dépréciées**) sont :
 - *ngIf
 - *ngFor





Les directives d'attribut (ngStyle) soft deprecation à partir d'NG19

- Cette directive permet de modifier l'apparence de l'élément cible.
- Elle est placée entre [] [ngStyle] qu'on ajoute dans la balise cible.
- Elle prend en paramètre un attribut représentant un objet décrivant le style à appliquer.
- Dans cet objet, la clé va représenter l'attribut de style que vous voulez gérer, e.g. color, backgroundColor, fontSize, fontFamily, border.
- La valeur représente la valeur associée à cette propriété de style et qui peut être une constante, dans ce cas elle ne change pas ou une variable et donc elle suivra toujours la valeur de cette variable.
- Dans cet exemple, l'attribut de style color prend sa valeur de la variable color, il sera donc 'lightblue'
- Pour l'attribut de style fontFamily on lui a associé une constante qui est 'garamond'.

```
@Component({  
  selector: 'app-ngstyle-exemple',  
  templateUrl: './ngstyle-exemple.component.html',  
  styleUrls: ['./ngstyle-exemple.component.css']  
})  
export class NgstyleExempleComponent {  
  color = 'lightblue';  
}
```

```
<p  
  [ngStyle]="{  
    backgroundColor: 'red',  
    color: color,  
    fontFamily: 'garamond'  
  }"  
>ngstyle-exemple works!</p>
```





Les directives d'attribut (ngStyle) soft deprecation à partir d'NG19



```
<div [style]="{'width': '200px'}">
```

OR

```
<div [style]="width: 200px">
```

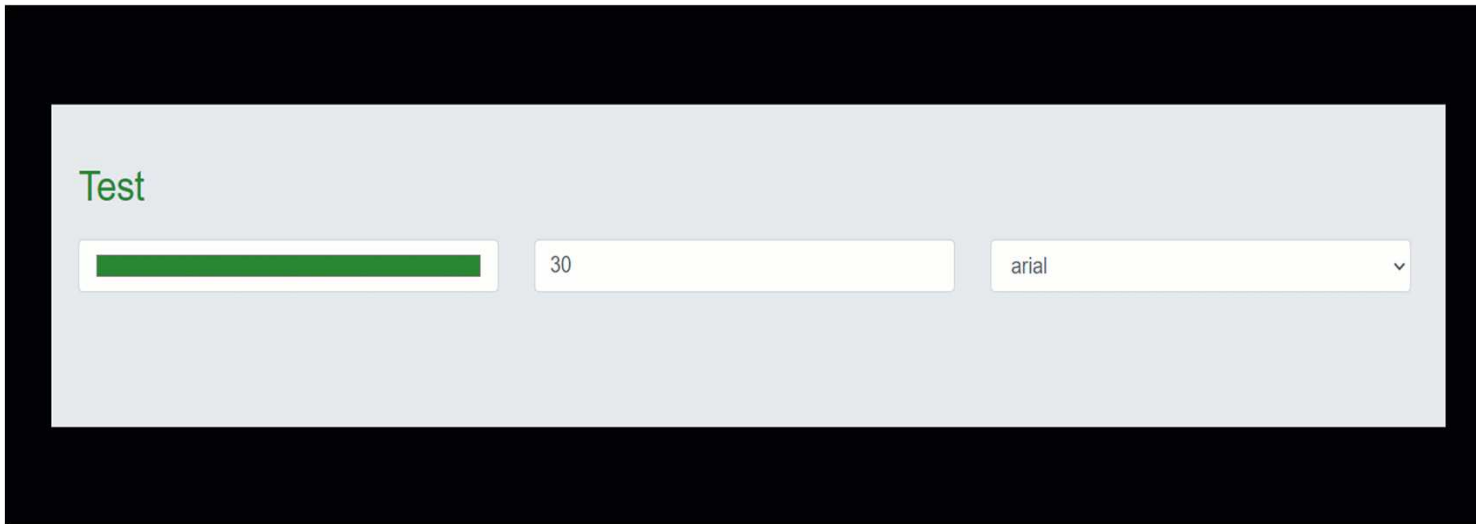


style object



Exercice

- Nous voulons simuler un Mini Word pour gérer un paragraphe en utilisant ngStyle.
- Préparer un input de type color, un input de type number, et un select box.
- Faire en sorte que lorsqu'on change la couleur du color input, ça devienne la couleur du paragraphe. Et que lorsque on change le nombre dans le number input la taille de l'écriture.
- Finalement ajouter une liste et mettez-y un ensemble de police. Lorsque le user sélectionne une police dans la liste, la police dans le paragraphe change.



The screenshot shows a light blue rectangular area containing a text editor. Inside the editor, the word "Test" is written in a green font. Below the text, there are three controls: a color picker showing a green bar, a number input field containing the value "30", and a dropdown menu showing "arial" with a downward arrow.





Les directives d'attribut (ngClass) soft deprecation à partir d'NG19

- Cette directive permet de modifier l'attribut `class`.
- Elle cohabite avec l'attribut `class`.
- Elle prend en paramètre
 - Une chaîne (string)
 - Un tableau (dans ce cas il faut ajouter les `[ ]` donc `[ngClass]`)
 - Un objet (dans ce cas il faut ajouter les `[ ]` donc `[ngClass]`)
- Elle utilise le **property Binding**.
- Dans cet exemple la classe CSS `off` sera appliqué à la Div

```
@Component({  
  selector: 'app-ngclass-exemple',  
  templateUrl: './ngclass-exemple.component.html',  
  styleUrls: ['./ngclass-exemple.component.css'],  
})  
export class NgclassExempleComponent {  
  isOn = false;  
}
```

```
<div  
  [ngClass]="{  
    on: isOn,  
    off: !isOn  
  }"  
  class="ampoule">lampe</div>
```





Les directives d'attribut (ngClass) soft deprecation à partir d'NG19

		class expression
<pre><div [class]="(true) ? 'my-class' : 'blank'"></pre>		
		class string
<pre><div [class]=" 'my-class' "></pre>		
		class array
<pre><div [class]="['user', 'phone', 'email']"></pre>		
	OR	
<pre><div [class]="{ 'user': getStatus() === 'active', 'phone': true, 'email': false, }"></pre> <pre><div [class.user]="getStatus() === 'active' [class.phone]="true" [class.email]="false" ></pre>		



Exercice

- Afin d'améliorer l'affichage de la liste des cvs dans votre CvTech, reprenez le composant cvList et faites-en sortes d'avoir les éléments successifs colorés d'une manière permutée entre deux couleurs.



Customiser une directive d'attribut

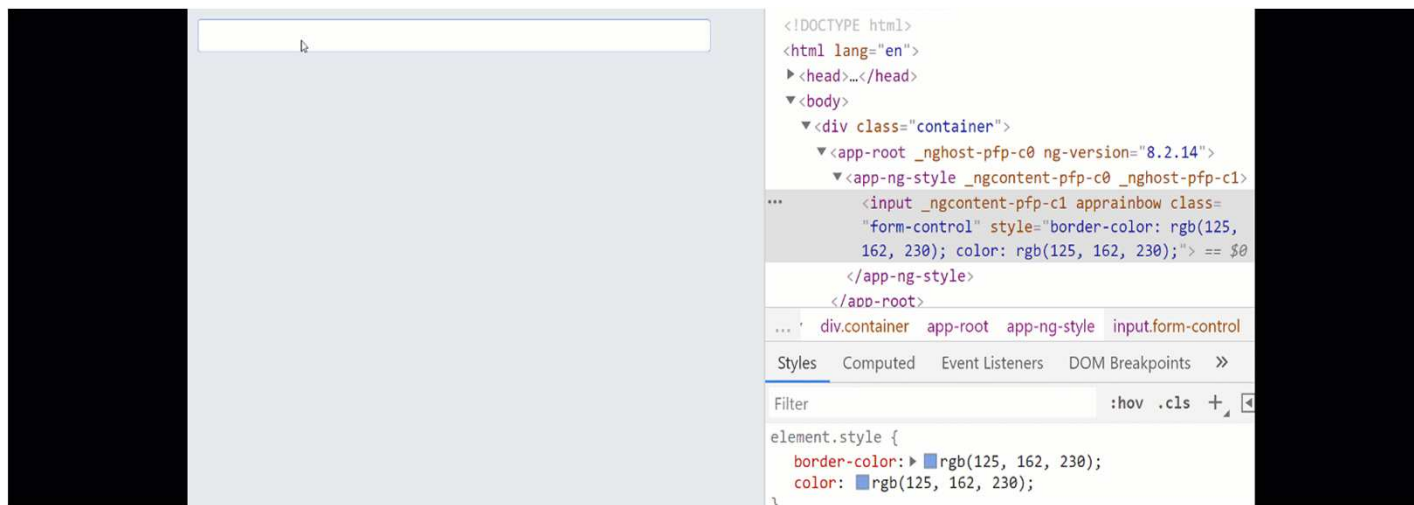
- Afin de créer sa propre « attribut directive » il faut utiliser un `HostBinding` sur la **propriété** que vous voulez **bind**er.
 - **Exemple** : `@HostBinding('style.backgroundColor') bg:string="red";`
- Dans cet exemple on lie (bind) l'attribut **bg** de la directive à l'attribut `style.backgroundColor` de l'élément hôte (celui qui est tagué) de la directive. A chaque fois qu'on change **bg** le `style.backgroundColor` de l'élément hôte change.
- Si on veut associer un **événement** à notre directive on utilise un `HostListener` qu'on associe à un **événement** déclenchant une **méthode**.
 - **Exemple** : `@HostListener('mouseenter') mouseover(){
 this.bg =this.highlightColor;
}`
- Afin d'utiliser le `HostBinding` et le `HostListener` il faut les importer du `core d'angular`



Exercice

Un truc plus sympa, on va créer un simulateur d'écriture arc en ciel.

- Créer une directive
- Créer un hostbinding sur la couleur et la couleur de la bordure.
- Préparer une méthode qui permet de générer aléatoirement une couleur dans votre directive.
- Faire en sorte qu'en appliquant votre directive à un input, à chaque écriture d'une lettre (event keyup) la couleur change en prenant aléatoirement l'une des couleurs de votre tableau. Pensez à utiliser `Math.random()` qui vous retourne une valeur entre 0 et 1.



HostBinding et HostListener

- La solution la plus appropriée est de ne plus utiliser les décorateurs @HostBinding et @HostListener et de plutôt **passer par l'attribut host de votre décorateur @Directive ou @Component**.
- Cet attribut prend en paramètre un objet dans lequel vous pouvez utiliser le binding standard

```
@Directive({
  selector: '[appHighlight]',
  standalone: true,
  host: {
    '[style.backgroundColor]': 'this.color()',
    '(mouseenter)': 'this.onMouseEnter()',
    '(mouseleave)': 'this.onMouseLeave()',
  }
})
export class HighlightDirective {//...}
```





Angular

Les pipes



1. Définir les pipes et l'intérêt de les utiliser
2. Vue globale des pipes prédéfinies
3. Créer un pipe personnalisé

Qu'est ce qu'un pipe

- Un **pipe** est une fonctionnalité qui permet de formater et de transformer vos données avant de les afficher dans vos Templates.
- Exemple l'affichage d'une date selon un certain format.
- Il existe des pipes **offerts par Angular** et prêt à l'emploi.
- Vous pouvez créer vos **propres pipes**.



Avec le pipe uppercase :

Sans aucun pipe :

```
<input type="text" [(ngModel)]="pipeVar" class="form-control">  
<br>Avec le pipe uppercase : {{pipeVar|uppercase}}<br>  
Sans aucun pipe : {{pipeVar}}
```



- Afin d'utiliser un pipe vous utilisez la syntaxe suivante :
 - {{ variable | nomDuPipe }}
- Exemple : {{ maDate | date }}
- Afin d'utiliser plusieurs pipes combinés vous utilisez la syntaxe suivante :
 - {{ variable | nomDuPipe1 | nomDuPipe2 | nomDuPipe3 }}
- Exemple : {{ maDate | date | uppercase }}

Paramétrer un pipe

- Afin de paramétrer les pipes ajouter `'.'` après le pipe suivi de votre paramètre.
 - `{{ maDate | date:".MM/dd/yy" }}`
- Si vous avez plusieurs paramètres c'est une suite de `'.'`
 - `{{ nom | slice:1:4 }}`

Les pipes disponibles par défaut (Built-in pipes)

➤ La documentation d'angular vous offre la liste des pipes prêt à l'emploi.

➤ <https://angular.io/api?type=pipe>

- uppercase
- lowercase
- titlecase
- currency
- date
- json
- percent
- ...



Pipe personnalisé

- Un pipe personnalisé est une **classe** décorée avec le **décorateur @Pipe**.
- Elle **implémente l'interface PipeTransform**
- Elle doit implémenter la méthode **transform** qui prend en **paramètre** la **valeur cible** ainsi qu'un **ensemble d'options**.
- La méthode **transform** doit **retourner la valeur transformée**
- Le pipe doit être déclaré au niveau de votre module de la même manière qu'une directive ou un composant.
- **Pout créer un pipe avec le cli : `ng g p nomPipe`**



Exemple de pipe

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'team',
})
export class TeamPipe implements PipeTransform {
  transform(team: any): string {
    switch (team) {
      case 'barca':
        return 'blaugrana';
      case 'roma':
        return 'giallorossa';
      case 'milan':
        return 'rossoneri';
    }
  }
}
```

```
<ol>
  @for ( team of teams() ; track team ) {
    <li>
      {{team | team}}
    </li>
  }
</ol>
```

```
@Component({
  selector: 'app-root',
  imports: [RouterOutlet, TeamPipe],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  teams = signal(['milan', 'barca', 'roma']);
}
```

1. rossoneri
2. blaugrana
3. giallorossa



Exercice

- Créer un pipe appelé `defaultImage` qui retourne le nom d'une image par défaut que vous stockerez dans vos assets au cas où la valeur fournie au pipe est une chaîne vide ou ne contient que des espaces.





```
Component service  
{Constructor(service)}
```

Angular

Service et injection de dépendances



1. Définir un service
2. Définir ce qu'est l'injection de dépendance
3. Injecter un service
4. Réordonner son code en utilisant les services

Comment fonctionne un restaurant ?



Injection de dépendance (DI)



Component *Service*
{Constructor(service)}

- L'injection de dépendance est un patron de conception.

```
Classe A1{  
  ClasseB b;  
  ClasseC c;  
  ...  
}
```

```
Classe A2{  
  ClasseB b;  
  ...  
}
```

```
Classe A3{  
  ClasseC c;  
  ...  
}
```

Que se passera t-il si on change quelque chose dans le constructeur de B ou C ?

Qui va modifier l'instanciation de ces classes dans les différentes classes qui en dépendent?



Injection de dépendance (DI)

Component *Service*
{Constructor(service)}

- Déléguer cette tâche à une entité tierce.

```
Classe A1{  
  Constructor(B b, C c)  
  ...  
}
```

```
Classe A2{  
  Constructor(B b)  
  ...  
}
```

```
Classe A3{  
  Constructor(C c)  
  ...  
}
```

INJECTOR





Comment fonctionne un restaurant (système d'injection de dépendances) ?

Providers

Injector

IOC

Injection



Injection de dépendance (DI)

- Comment les injecter ?
- Comment spécifier à l'injecteur quel service et ou est-il visible ?

Injection de dépendance (DI)

- L'injection de dépendance utilise les étapes suivantes :
 - **Provider les dépendances** (Préparer notre menu, définir quels plats nous pouvons fournir) via l'annotation **@Injectable** et son attribut **providedIn** (ici les services), dans le **provider du module** ou du **composant** (C'est la préparation de votre menu).
 - **Injecter le service** (commander le plat souhaité) en le passant comme **paramètre du constructeur** de la **classe** (composant ou service) qui en a besoin ou à partir de la version 14 utiliser la fonction **inject** (c'est la bonne pratique).





Injection de dépendance (DI)

Provider les dépendances et les injecter (Préparer notre menu, et commander)

```
@NgModule({  
  ancienne  
  providers: [CvService],  
  bootstrap: [AppComponent],  
})  
export class AppModule {}
```

```
@Component({  
  selector: 'app-cv',  
  templateUrl: './cv.component.html',  
  styleUrls: ['./cv.component.css'],  
  providers: [CvService]  
})  
export class CvComponent {}
```

```
export const appConfig: ApplicationConfig = {  
  providers: [CvService],  
  nouvelle  
};
```

```
@Injectable({  
  providedIn: 'root',  
})  
export class CvService {}
```

Provider

Injecter

```
export class CvComponent {  
  constructor(  
    ancienne  
    private cvService: CvService  
  ) {}  
}
```

```
export class CvComponent {  
  nouvelle  
  private cvService = inject(CvService);  
}
```





Les providers personnalisés la fonction inject (après Angular 14)

- La fonction inject vous permet d'injecter un injectable.
- Avant Angular 14 et à partir d'Angular 9, la fonction inject pouvait être utilisé uniquement dans la factory de l'InjectionToken ou dans le factory du @Injectable.
- A partir d'Angular 14, vous pouvez l'utiliser dans tout le contexte d'injection de dépendances comme vos composants, directives et pipes.
- Le premier intérêt est le type safety avec le décorateur @Inject.
- Il facilite aussi l'héritage en externalisant la dépendance du composant.
- Faites attention quand vous utilisez inject elle doit être dans un contexte d'injection

```
export class CvComponent {  
  // Donne moi le sayHelloService  
  sayHelloService = inject(SayHelloService);  
  cvService = inject(CvService);  
}
```





Chargement automatique du service

- A partir de Angular 6 vous pouvez ne plus utiliser le provider du module afin de charger votre service mais le faire directement au niveau du service à travers l'annotation `@Injectable` et sa propriété `providedIn`. Vous pouvez charger le service dans toute l'application via le mot clé `root`. Ceci permet d'optimiser votre code via :
- **Lazy loading** : N'instancie un service que s'il est injecté au moins une fois
- Permettre le **Tree-Shaking** des services non utilisés : Si le service n'est jamais utilisé, son code sera entièrement retiré du build final.

```
@Injectable({  
  providedIn: 'root',  
})  
export class CvService {}
```



@Injectable

- C'est un décorateur permettant de rendre une classe injectable
- Une classe est dite injectable si on peut y injecter des dépendances
- `@Component`, `@Pipe`, et `@Directive` sont des sous classes de `@Injectable()`, ceci explique le fait qu'on peut y injecter directement des dépendances.
- Si vous n'aller injecter aucun service dans votre service, cette annotation n'est plus nécessaire.
- **Remarque** : Angular conseille de toujours mettre cette annotation.



Créer un service

- Afin de créer un service, vous pouvez passer par la commande

`ng g s nomService`

- Créer manuellement une classe et de la fournir
 - en ajoutant `@Injectable({'providedIn': 'root'})`
 - ou en la providant via un provider





Configurez l'injection de dépendance

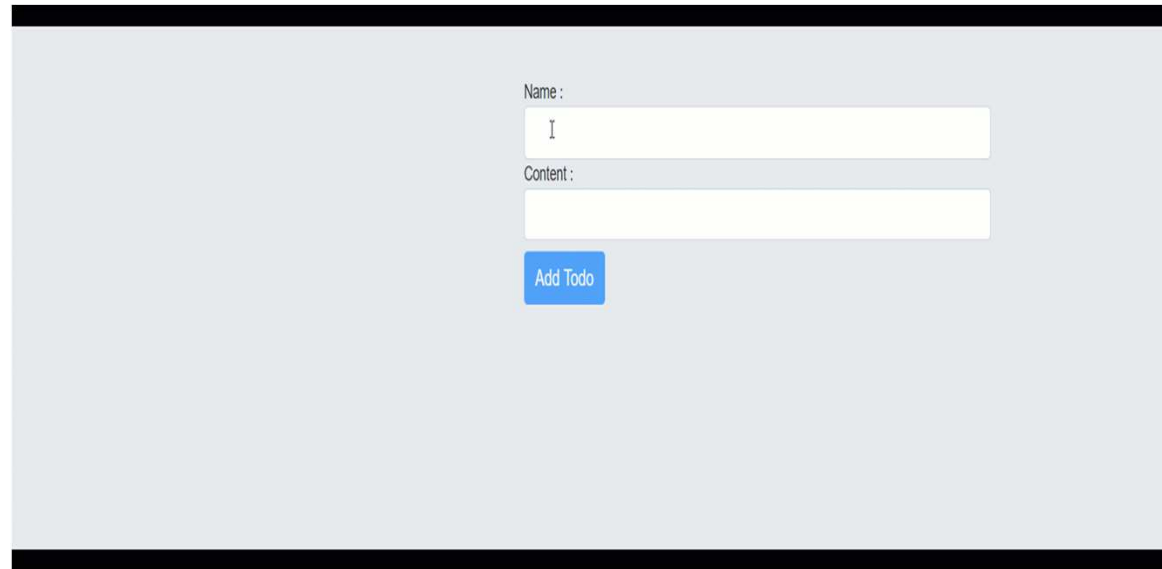
Provider des services externes

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(
      routes,
      withComponentInputBinding()
    ),
    provideHttpClient(
      withInterceptorsFromDi()
    ),
    importProvidersFrom(
      TranslateModule.forRoot({
        loader: {
          provide: TranslateLoader, useFactory: HttpLoaderFactory, deps: [HttpClient],
        },
      })
    ),
  ],
};
```



Exercice

- Créons ensemble un service de Todo, le Model et le composant qui va avec. Un Todo est caractérisé par un nom et un contenu.
- Ce service permettra de faire les fonctionnalités suivantes :
 - Logger les todos
 - Ajouter un Todo
 - Récupérer la liste des Todos
 - Supprimer un Todo

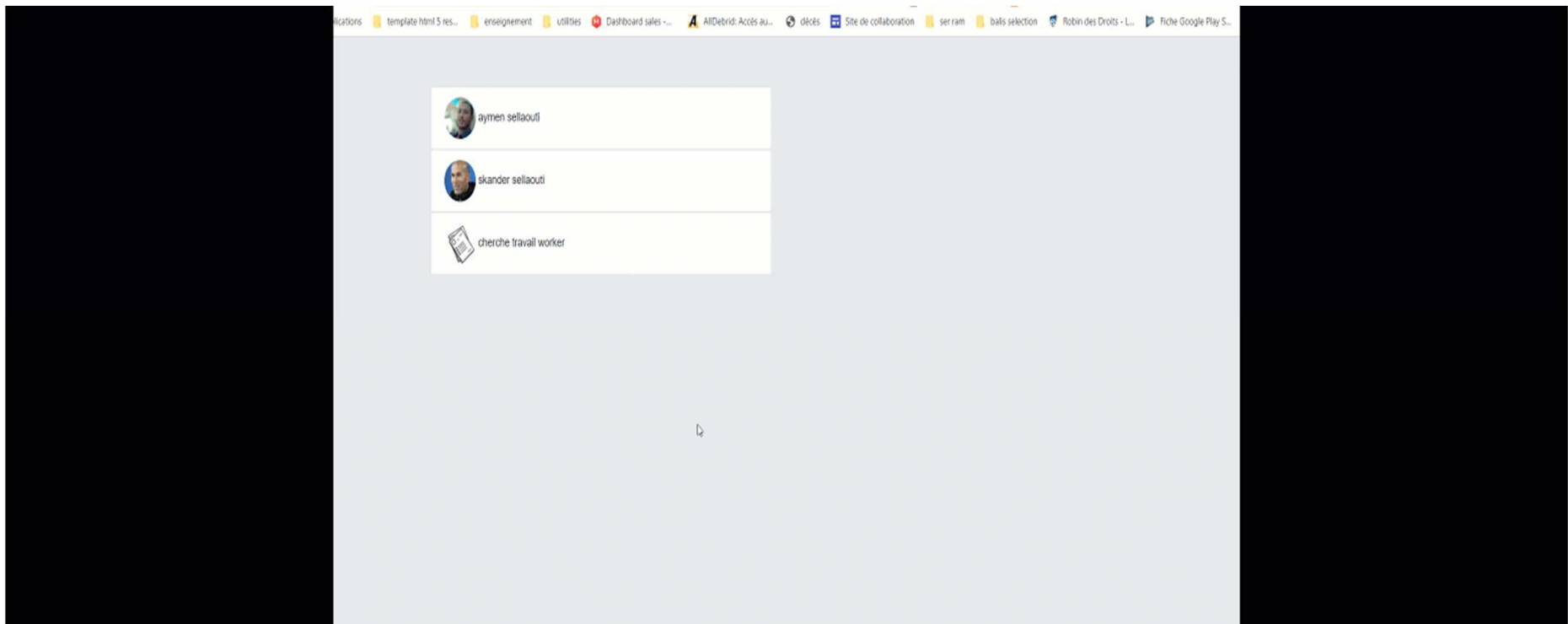


A screenshot of a web form for adding a todo item. The form is set against a light blue background. It features two input fields: the first is labeled 'Name :' and contains the letter 'I'; the second is labeled 'Content :'. Below these fields is a blue button with the text 'Add Todo' in white. The entire form is enclosed in a thin black border.



- Ajouter les services suivants afin d'améliorer la gestion de notre plateforme d'embauche.
 - Un premier **service CvService** qui gérera les Cvs. Pour le moment c'est lui qui contiendra la liste des cvs que nous avons.
 - Faite en sorte que le CvService gère le selectedCv à afficher dans le CvCardComponent
 - Ajouter aussi un **composant** pour afficher la liste des cvs embauchées ainsi qu'un **service EmbaucheService** qui gérer les embauches.
 - Au click sur le bouton embaucher d'un Cv, le cv est ajouté à la liste des personnes embauchées et une liste des embauchées apparaît.

Exercice



Angular Routing

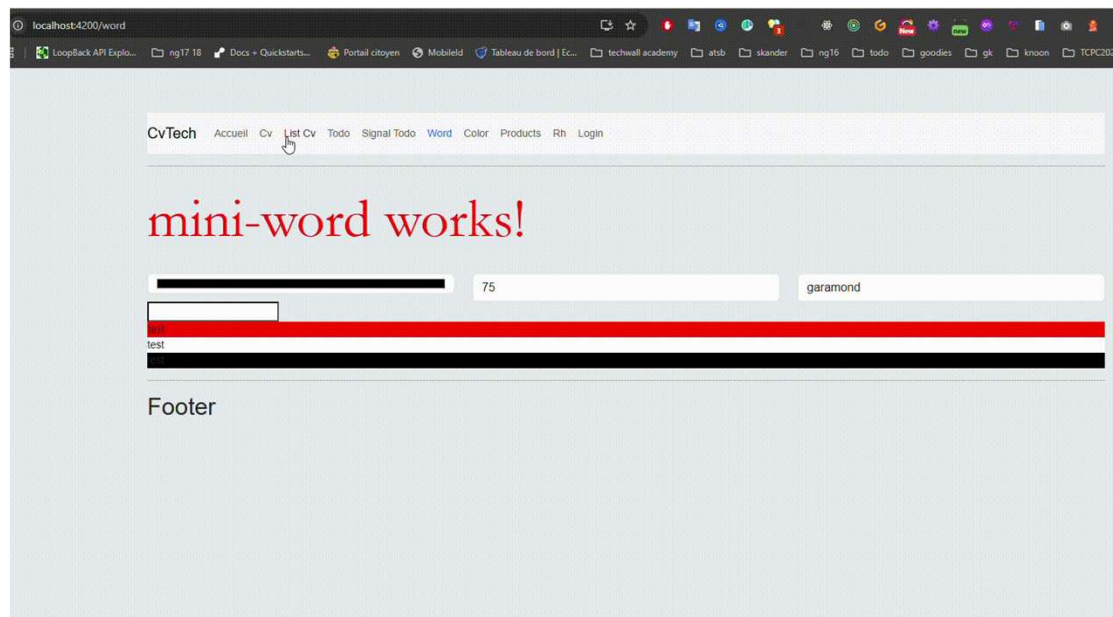


1. Définir le routeur d'Angular
2. Définir une route
3. Déclencher une route à partir d'un composant
4. Ajouter des paramètres à une route
5. Récupérer les paramètres d'une route à partir du composant.
6. Préfixer un ensemble de routes
7. Gérer les routes inexistantes



Comment fonctionne le système de routing d'Angular et quels sont ses composantes

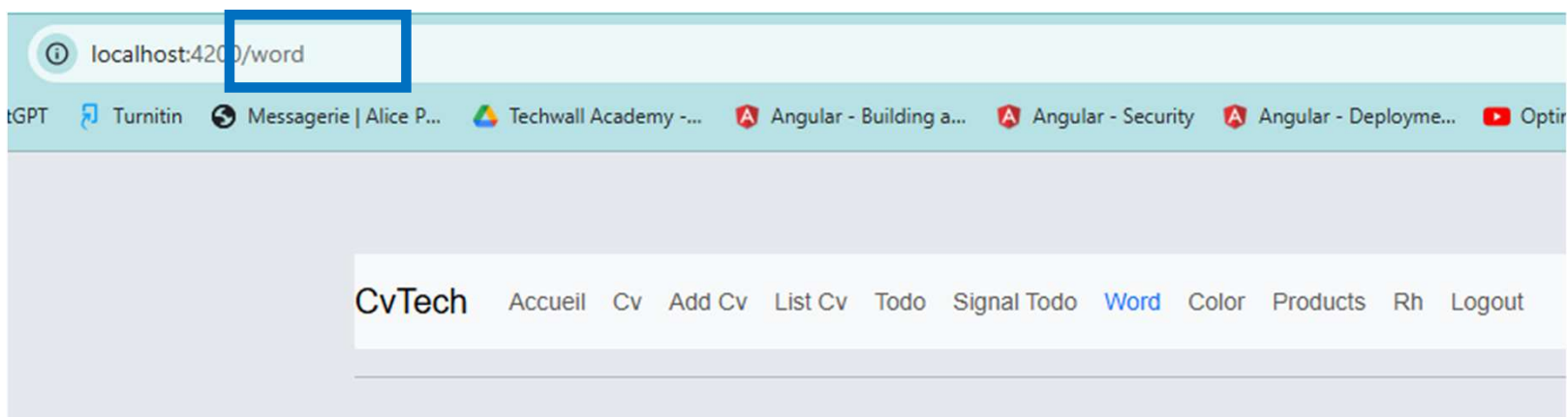
- Quand vous **naviguez dans un site web**, c'est votre URI qui vous permet de demander **quelle page (ressource) afficher**.
- A chaque fois que vous changez d'URI, Angular détecte ce changement et essaye de détecter à quoi correspond l'URI demandée. Pour ce faire nous avons besoin de deux choses :
 - Un **responsable** de cette tâche : c'est le **RouterModule**
 - De définir cette **relation URI, Composant** pour que le RouterModule sache quoi faire : c'est votre **route**.





Création d'un système de Routing

Application modulaire



```
export const appConfig: ApplicationConfig = {  
  providers: [  
    provideRouter(routes)  
  ]  
}
```

```
const routes: Routes = [  
  { path: word, component: FirstComponent },  
  { path: 'word', component: MiniWordComponent },  
  { path: 'color', component: ColorComponent },  
];
```



Syntaxe minimaliste d'une route

- Une route est un objet.
- Les deux propriétés essentielles sont `path` et `component`.
- `Path` représente l'URI
- `component` permet de spécifier le composant à exécuter.

```
const routes: Routes = [  
  { path: '', component: FirstComponent },  
  { path: 'word', component: MiniWordComponent },  
  { path: 'color', component: ColorComponent },  
];
```





Création d'un système de Routing

Application Standalone

- Afin de créer un système de routing, il suffit de provider vos routes avec la fonction `provideRouter` au niveau de la fonction `bootstrapApplication`.
- Vous ne devez plus importer le `RouterModule`, c'est la fonction `provideRouter` qui s'en charge.

```
bootstrapApplication(AppComponent, appConfig)  
  .catch((err) => console.error(err));
```

```
export const appConfig: ApplicationConfig = {  
  providers: [  
    provideRouter(routes)  
  ]  
}
```

```
export const routes: Routes = [  
  { path: '', component: HomeComponent },  
]
```





Préparer l'emplacement d'affichage des vues correspondantes aux routes

- Généralement dans vos sites ou applications Web vous avez un template que vous suivez.
- Ce template contient une **partie statique** et une **partie variable**.
- La partie variable sera celle ou Angular devra affiché le composant associé à la route.
- Pour indiquer cette partie variable vous devez utiliser la **directive** `<router-outlet/>`.

```
<app-navbar/>
<hr>
<router-outlet/>
<hr>
<h2>Footer</h2>
```





Création d'un système de Routing

Application Standalone

- Maintenant et dans les **différents composants** qui vont utiliser les **éléments du module de routing**, vous avez deux choix.
- **Importer le RouterModule**
- Ceci fonctionne **vous pouvez utiliser le router-outlet**, mais **ce n'est pas la meilleure façon de faire**, ça n'explique pas de quoi dépend exactement votre AppComponent.

```
@Component({
  selector: 'app-root',
  template: `
    <router-outlet></router-outlet>
  `,
  styleUrls: ['./app.component.css'],
  imports: [RouterModule],
  standalone: true,
})
export class AppComponent {
```





Création d'un système de Routing

Application Standalone

- Importer uniquement ce dont vous dépendez, ce que vous avez besoin d'utiliser, n'oubliez pas il n'y a pas que les Standalone Components, on parle aussi de **Standalone Directive et Pipe** et c'est le même modèle mental.

```
@Component({  
  selector: 'app-root',  
  template: `  
    <router-outlet></router-outlet>  
  `,  
  styleUrls: ['./app.component.css'],  
  imports: [RouterOutlet],  
  standalone: true,  
})  
export class AppComponent {
```





Déclencher une route routerLink

routerLink : DIRECTIVE VS PROPERTY

- RouterLink a deux variantes : une directive et une property de la directive.
 1. On opte pour la directive quand la route cible est statique (sans paramètres).
 2. On fait du property binding quand la route cible est dynamique.

```
@Component({
  selector: 'app-root',
  template: `
    <a [routerLink]="['cv', cv.id]">route
dynamique</a>
    <a routerLink="cv">route statique</a>
    <router-outlet></router-outlet>`,
  styleUrls: ['./app.component.css'],
  imports: [RouterOutlet, RouterLink],
  standalone: true,
})
export class AppComponent {
```



Déclencher une route routerLink

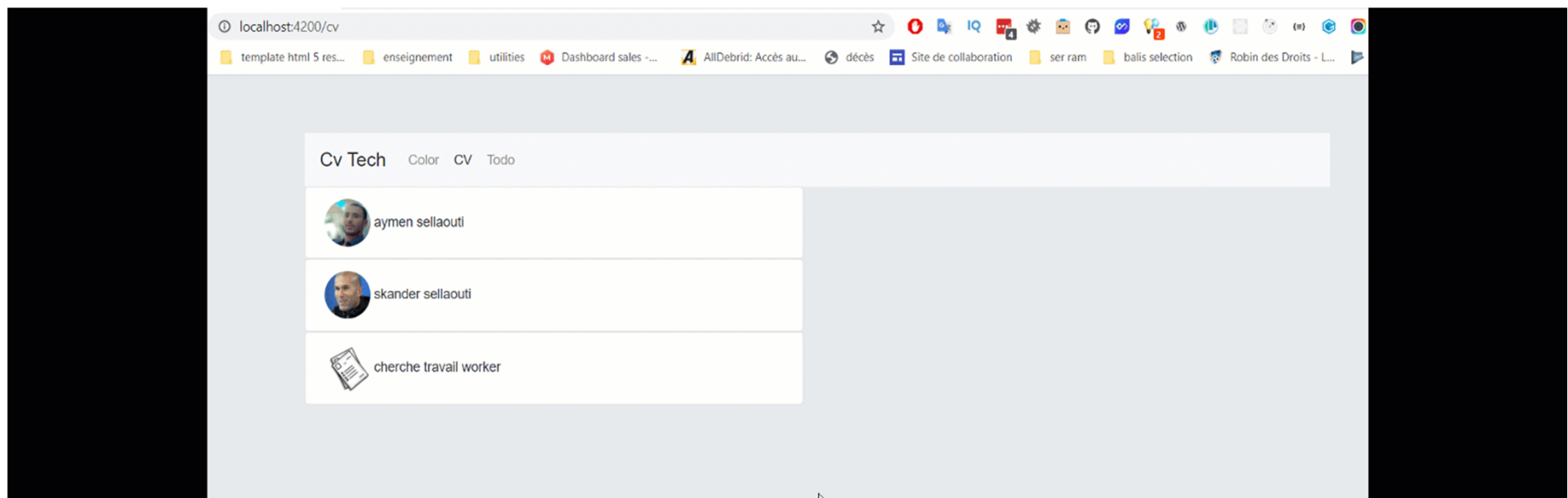
- Afin de **mettre en avant la route sélectionnée par l'utilisateur** dans votre menu, vous pouvez utiliser la directive **routerLinkActive**
- **routerLinkActive** prend en paramètre la **liste des classes CSS** que vous voulez **appliquez** à la **route sélectionnée ainsi qu'à tous ses ancêtres**.
- Par exemple si on a l'URI **/cv/liste** les classes de **routerLinkActive** seront ajoutées à cet URI ainsi qu'à l'URI **/cv** et **/**.
- Pour identifier uniquement l'uri cible, ajouter la directive **routerLinkActive** avec la valeur **{exact: true}**

```
<a class="nav-item nav-link"
  routerLinkActive="active text-primary"
  [routerLinkActiveOptions]="{exact: true}"
  [routerLink]="['/cv/liste']">
  >Accueil
</a>
```



Exercice

- Faites en sorte d'avoir un composant Header dans votre application qui permet d'afficher l'ensemble de vos liens.
- En cliquant sur un lien, le composant qui lui est associé doit être affiché.



Déclencher une route à partir du composant

- Afin de déclencher une route à travers le composant on utilise le service `Router` et sa méthode `navigate`.
- Cette méthode prend le même paramètre que le `routerLink`, à savoir un tableau contenant la description de la route.
- Afin d'utiliser le `Router`, il faut l'importer de l'`@angular/router` et l'injecter dans votre composant.

```
import { ActivatedRoute, Router } from "@angular/router";
router = inject(Router);
onLoadCv(id: number) {
  // méthode conseillée
  this.router.navigate(['cv', id]);
  //alternative
  this.router.navigate([`cv/${id}`]);
}
```



Les paramètres d'une route

- Afin de spécifier à notre router qu'un segment d'une route est un paramètre, il suffit d'y ajouter ':' devant le nom de ce segment.
- Exemple
 - `/cv/:id` permet de dire que la route contient **au début cv** ensuite un **paramètre de route appelé id**.

```
const routes: Routes = [  
  { path: '', component: FirstComponent },  
  { path: 'todo/:id', component: TodoDetailsComponent },  
  { path: 'color', component: ColorComponent },  
];
```



Création d'un système de Routing

localhost:4200/todo/5

La deuxième route ne contient qu'un seul segment et moi j'en cherche 2 ça ne peut pas être ça

```
@NgModule({
  imports: [
    RouterModule.forRoot(routes),
  ],
  exports: [RouterModule],
})
export class AppRoutingModule {}
```

Je cherche une route avec deux segments

- 1- Le premier segment si c'est une constante il doit être todo sinon si c'est une variable peut contenir n'importe quoi
- 2- Le deuxième segment si c'est une constante il doit être 5, sinon si c'est une variable, il peut contenir n'importe quoi

```
const routes: Routes = [
  { path: 'todo/5', component: FirstComponent },
  { path: 'color', component: ColorComponent },
  { path: 'todo/:id', component: TodoDetailsComponent },
];
```



Récupérer les paramètres d'une route

- Afin de récupérer les paramètres d'une route au niveau d'un composant on doit procéder comme suit :
 1. `Importer ActivatedRoute` qui nous permettra de récupérer les paramètres de la route de "`@angular/router`".
 2. `Injecter ActivatedRoute` au niveau du composant.
 3. Utilisez l'objet `snapshot`

```
import { ActivatedRoute } from "@angular/router";

export class DetailsCvComponent {
  acr = inject(ActivatedRoute);
  constructor() {
    this.acr.snapshot.params;
  }
}
```





Récupérer les paramètres d'une route

ActivatedRoute / snapshot

- La propriété **snapshot** de l'objet **ActivatedRoute** contient un instantané de l'état de la route actuelle.
- Elle contient des **informations** telles que l'URL actuelle, les paramètres de route actuels,...
- Elle est généralement utilisée pour accéder aux informations de route dans un composant lors de son initialisation.
- Il est important de noter que l'instantané de la route ne change pas lorsque la route change. Il représente un état figé de la route lors de son instanciation.





Récupérer les paramètres d'une route

ActivatedRoute / snapshot

- Voici quelques propriétés courantes de l'API snapshot :
 - **url**: Retourne l'URL de la route actuelle sous forme de tableau de segments d'URL.
 - **params**: Retourne un objet qui contient les paramètres de route actuels.
 - **queryParams**: Retourne un objet qui contient les paramètres de requête actuels.
 - **fragment**: Retourne la partie de l'URL après le symbole "#".
 - **data**: Retourne les données de route associées à la route actuelle.
 - **component**: Retourne le composant de route actuel.
 - **routeConfig**: Retourne la configuration de la route actuelle.

```
▼ snapshot: ActivatedRouteSnapshot
  ▶ component: Class DetailsCvComponent
  ▶ data: {cv: {...}}
  fragment: null
  outlet: "primary"
  ▶ params: {id: '27'}
  ▶ queryParams: {}
  ▼ routeConfig:
    ▶ component: Class DetailsCvComponent
    path: ":id"
    ▶ resolve: {cv: f}
    ▶ [[Prototype]]: Object
  ▶ url: [UrlSegment]
    _lastPathIndex: 1
  ▶ _paramMap: ParamsAsMap {params: {...}}
  ▶ _resolve: {cv: f}
  ▶ _resolvedData: {cv: {...}}
  ▶ _routerState: RouterStateSnapshot {_root: TreeNode, url: '/cv/2'}
  ▶ _urlSegment: UrlSegmentGroup {segments: Array(2), children: {...}}
  ▶ children: Array(0)
  firstChild: null
  ▼ paramMap: ParamsAsMap
    ▶ params: {id: '27'}
    keys: (...)
    ▶ [[Prototype]]: Object
    parent: (...)
```





Récupérer les paramètres d'une route

ActivatedRoute / snapshot

- Donc pour accéder à votre propriété, passez par l'objet `snapshot`
- Avec `snapshot`, vous avez deux méthodes pour récupérer les paramètres:
- Via la `propriété params` qui retourne un tableau d'objet des paramètres
- Via la propriété `paramMap`
 - Appeler sa méthode `get`
 - Passez lui le nom de la propriété souhaitée.

```
import { ActivatedRoute } from "@angular/router";

export class DetailsCvComponent {
  acr = inject(ActivatedRoute);
  constructor() {
    console.log({'snapshot.params': this.acr.snapshot.params});
  }
}
```

```
▼ snapshot.params:
  id: "662"
```



Passer le paramètre à travers le tableau de routerLink

- Une autre méthode permet de passer le paramètre de la route est en l'ajoutant comme un autre attribut du tableau associé au routerLink

```
import { Component, OnInit } from '@angular/core';
import { Router } from '@angular/router';
@Component({
  selector: 'app-home',
  templateUrl: './home.component.html',
  styleUrls: ['./home.component.css'],
})
export class HomeComponent {
  constructor(private router: Router) {}
  id: number = 10;
  onNaviger() {
    this.router.navigate(['/about', this.id]);
  }
}
```

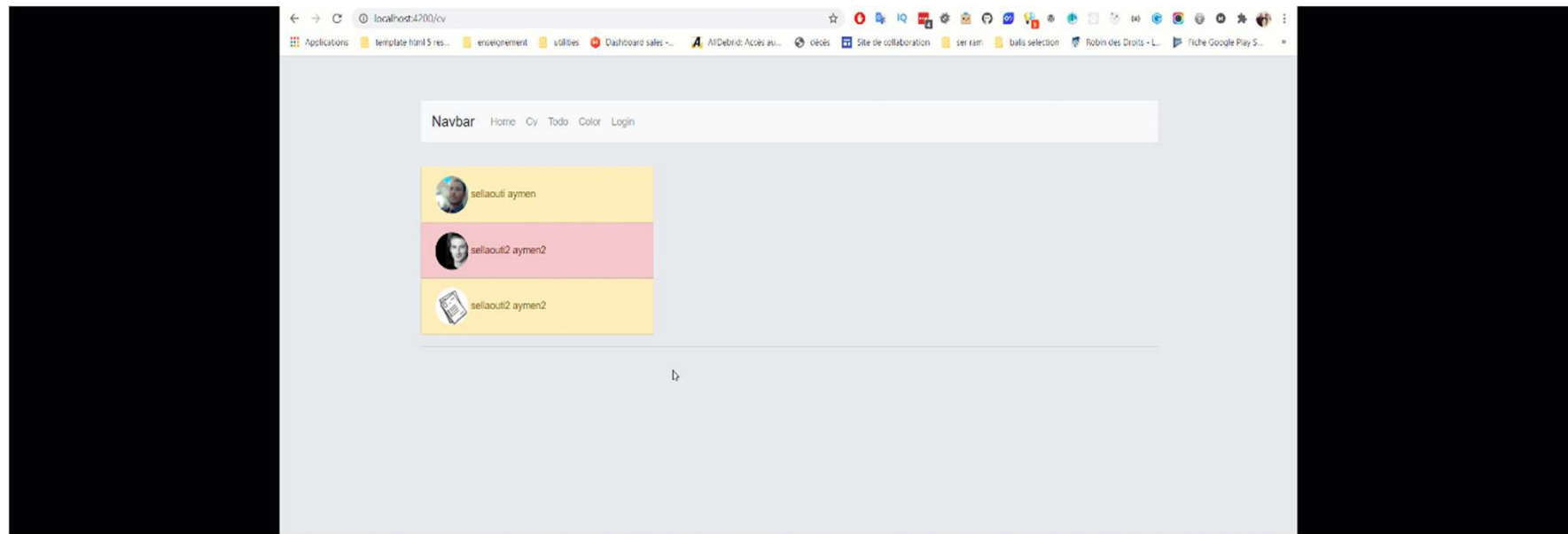


- Il existe une route **joker** qui **matche n'importe quelle autre route**. C'est la route **'**'**.

```
const APP_ROUTE: Routes = [  
  { path: 'cv', component: CvComponent },  
  { path: 'lampe', component: ColorComponent },  
  { path: 'login', component: LoginComponent },  
  { path: '**', component: NFComponent },  
];
```

Exercice

- Ajouter les fonctionnalités suivantes à votre cvTech:
 - Une page détail qui va afficher les détails d'un cv.
 - Un bouton dans chaque cv qui au click vous envoie vers la page détails.
 - Dans la page détail, un bouton delete qui au click supprime ce cv et vous renvoie à la liste des cvs.





Angular Form



Approche de gestion de FORM

1. Approche basée Template
2. Approche réactive



1. Créer un formulaire
2. Ajouter des validateurs
3. Appréhender les classes Css générées par le formulaire
4. Manipuler l'objet ngForm
5. Manipuler les controles du formulaire



Approche basée Template/ Template Driven Approach

1

Importer le module `FormsModule` dans `app.module.ts` si vous travaillez avec un **application modulaire**, sinon **dans votre composant** si vous travaillez avec les **standalone component**

2

Angular détecte automatiquement un objet form à l'aide de la balise FORM. Cependant, il ne détecte aucun des éléments (inputs).

3

Spécifier à Angular quel sont les éléments (contrôles) à gérer.

- Pour chaque élément ajouter la directive angular `ngModel`.
- Identifier l'élément avec un nom permettant de le détecter et de l'identifier dans le composant.

```
<input
  type="text"
  id="username"
  class="form-control"
  ngModel
  name="username"
>
```

4

Associer l'objet représentant le formulaire à une variable et la passer à votre fonction en utilisant le référencement interne `#` et la directive `ngForm`



Approche basée Template/ Template Driven Approach

```
<form
  (ngSubmit)="onSubmit(formulaire)"
  #formulaire="ngForm">
```

Template

```
export class
TmplateDrivenComponent{
  onSubmit(formulaire: NgForm) {
    console.log(formulaire);
  }
}
```

Component.ts





Approche basée Template Validation

- Afin de valider les propriétés des différents contrôles, Angular utilise des attributs et des directives
 - required
 - email
- La propriété valid de ngForm permet de vérifier si le formulaire est valid ou non en se basant sur les validateurs qu'ils contiennent.





Approche basée Template NgForm

En détectant le formulaire, Angular décore les différents éléments du formulaire avec des classes qui informe sur leur état :

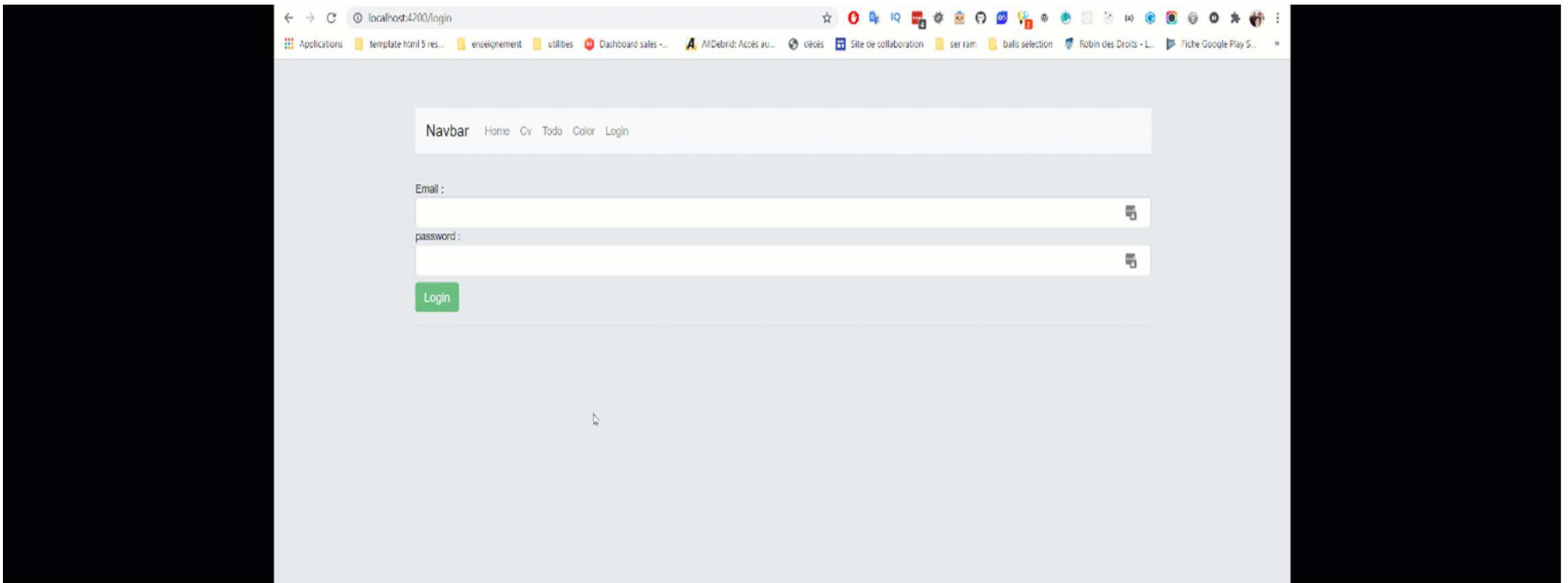
- **dirty** : informe sur le fait que l'une des propriétés du formulaire a été modifié ou non
- **Valid / invalid** : informe si le formulaire est valide ou non
- **Untouched / touched** : informe si le formulaire est touché ou non
- **pristine** : le formulaire n'a pas été touché, c'est l'opposé du dirty



Exercice

- Créer un formulaire d'authentification contenant les champs suivants :
 - Email
 - Password
 - Envoyer
- Si un champ est invalide alors il devra avoir une bordure rouge.
- Les deux champs sont obligatoires
- Le password doit avoir au moins 4 caractères.
- Un champ vide et non encore modifié ne peut avoir de bordure rouge que s'il a été touché.
- Le bouton « envoyer » ne doit être cliquable que si le formulaire est valide.
- Utiliser le binding sur la propriété *disabled*.





localhost:4200/login

Applications template.html 5 res... enseignement utilites Dashboard sales -... AlDebrid: Accès au... clés Site de collaboration ser ram balls selection Robin des Droits - L... Fiche Google Play S...

Navbar Home Cv Todo Color Login

Email :

password :

Login



Approche basée Template

Accéder aux propriétés d'un champ (contrôle) du formulaire

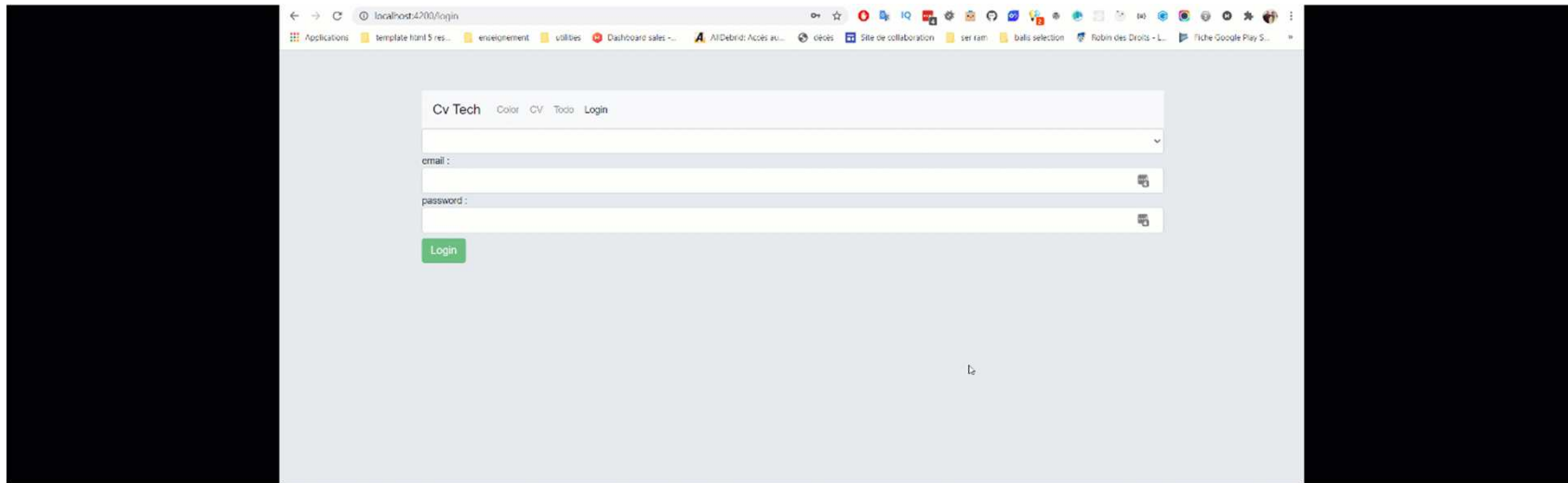
- Pour accéder à l'objet form et ces propriétés nous avons utilisé `#notreForm=«ngForm »`
- Pour les champs du formulaire c'est la même chose mais au lieu du ngForm c'est un `ngModel`

`#notreChamp=« ngModel »`



Exercice

- Ajouter un petit message d'erreur qui devra s'afficher sous le champ de l'email s'il est invalide. Ce champ ne devra apparaître que si l'utilisateur accède ou modifie le champ email.
- Ajouter un champ d'erreur pour signaler l'erreur à l'utilisateur.





Angular
HTTP



1. Comprendre le design pattern Observable et son implémentation avec RxJs
2. Appréhender le Module HTTPClientModule d'Angular
3. Utiliser les différents services du module HTTPClientModule
4. Comprendre le principe d'authentification via les tokens
5. Utiliser les protecteurs de routes (les guards)
6. Utiliser les Interceptors afin d'intercepter les requêtes Http
7. Déployer votre application en production

- Angular est un Framework FrontEnd
- Pas d'accès à la BD
- Pas de possibilité de persistance des données
- Pas de puissance permettant des traitements lourds.

Le HTTPClient

Installation de HTTP

- A partir de la version 18, le `HttpClientModule` est devenu **deprecated**.
- Afin d'utiliser le service `HttpClient`, vous devez le configurer en utilisant la méthode `provideHttpClient`, que la plupart des applications incluent dans la clé `providers` de votre `app.config.ts`.

```
export const appConfig: ApplicationConfig = {  
  providers: [  
    provideHttpClient(),  
  ],  
};
```

- Une fois configuré, vous pouvez injecter votre service là où vous en avez besoin.

```
http = inject(HttpClient);
```



- En injectant le HttpClient il vous offre plusieurs méthodes qui représentent les verbes HTTP GET, POST, PUT, PATCH et DELETE
- Chacune retourne un observable

```
this.httpClient.  
delete  
get  
head  
jsonp  
options  
patch  
post  
put  
request
```

Qu'est ce que la programmation réactive

1. Nouvelle manière d'appréhender les appels asynchrones
2. Programmation avec des flux de données asynchrones

Programmation reactive =
Flux de données (observable) + **écouteurs d'événements**(observer).

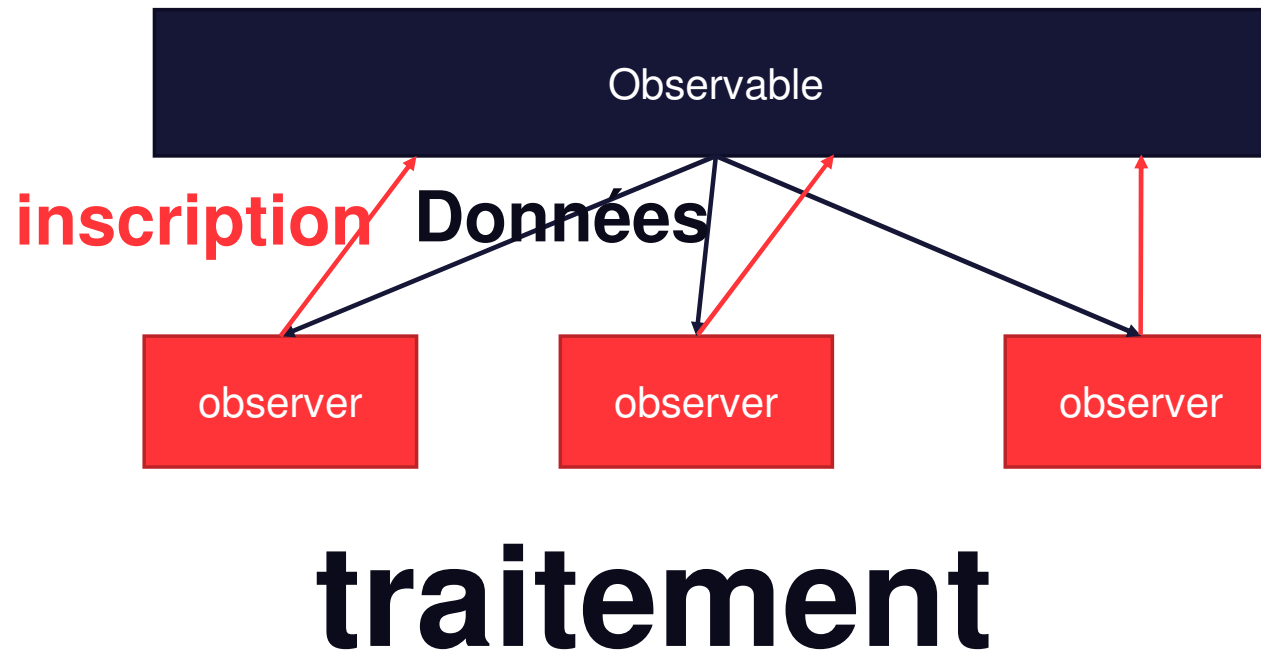


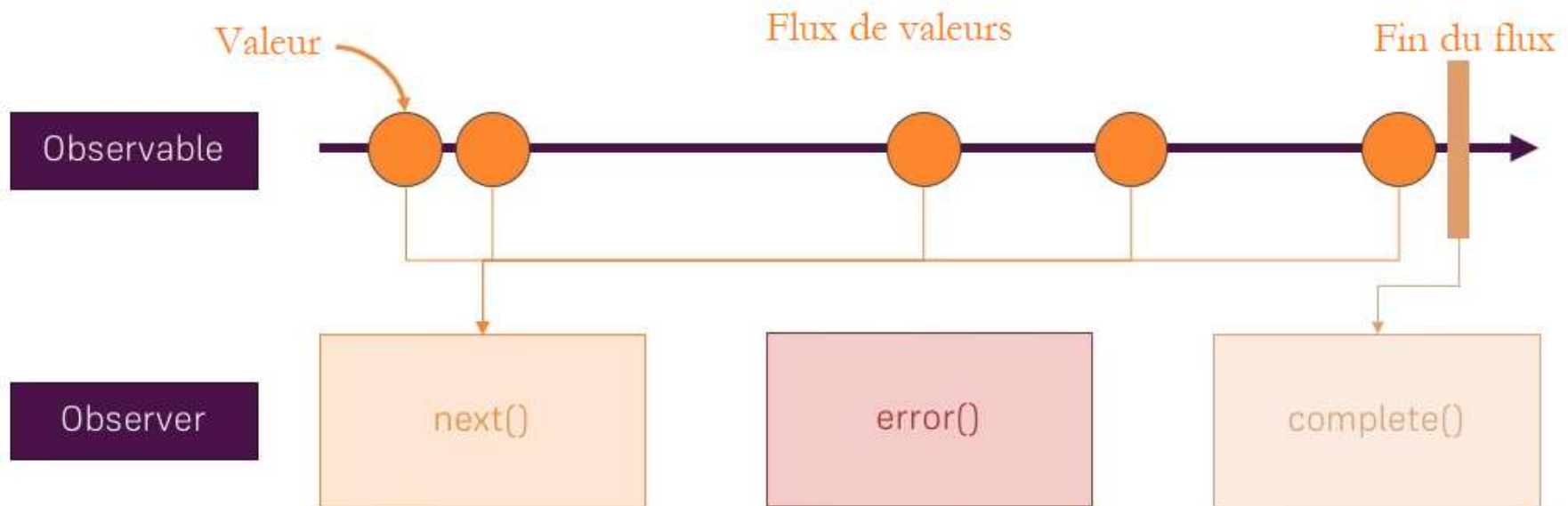
Le pattern « Observer »

- Le patron de conception **Observateur** permet à un objet de garder la trace d'autres objets, intéressés par l'état de ce dernier.
- Il définit une relation entre objets de type un-à-plusieurs.
- Lorsque l'état de cet objet **change**, il **notifie** ces **observateurs**.



Observables, Observers et subscriptions





S'inscrire à un observable

- Afin de **montrer votre intérêt à un flux Observable**, vous pouvez utiliser la méthode **subscribe** qu'il vous offre.
- En vous inscrivant à cet observable, vous pouvez **passer à cette méthode** un **objet qui a 3 fonctions** :
 - **next**, qui sera **appelé à chaque fois qu'une nouvelle valeur arrive dans le flux**. Elle prend en **paramètre cette valeur** et vous permet d'implémenter ce que vous voulez faire avec.
 - **Error, déclenché une fois en cas d'erreur** qui vous permet d'implémenter le comportement en réaction à cette erreur
 - **Complete, déclenché dès la fin du flux** qui vous permet d'implémenter le comportement en réaction à la fin du flux.



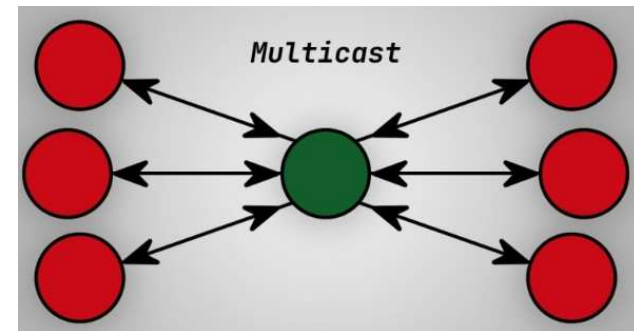
S'inscrire à un observable

```
myObservable$.subscribe({  
  next: (data) => {  
    //TODO: implementez le comportement quand vous recevez les données  
  },  
  error: (error) => {  
    //TODO: implementez le comportement quand vous avez une erreur  
  },  
  complete: () => {  
    //TODO: implementez le comportement à la fin du flux  
  },  
});
```



Hot Vs Cold Observable

- Les *Cold Observables* commencent à émettre des valeurs uniquement quand on s'y inscrit. Les *Hot observables*, par contre émettent toujours.
- Les *Cold Observables* diffusent un flux par inscrit, ils sont *unicast*. Chaque nouvelle inscription crée un *nouveau contexte d'exécution*.
- Les *Hot observables*, sont *multicast*, le *même flux est partagé par tous les inscrits*.
- Dans les *Cold Observables*, la *source de données est à l'intérieur* de l'observable.
- Dans les *HotObservables*, la *source de données est à l'extérieur* de l'observable.



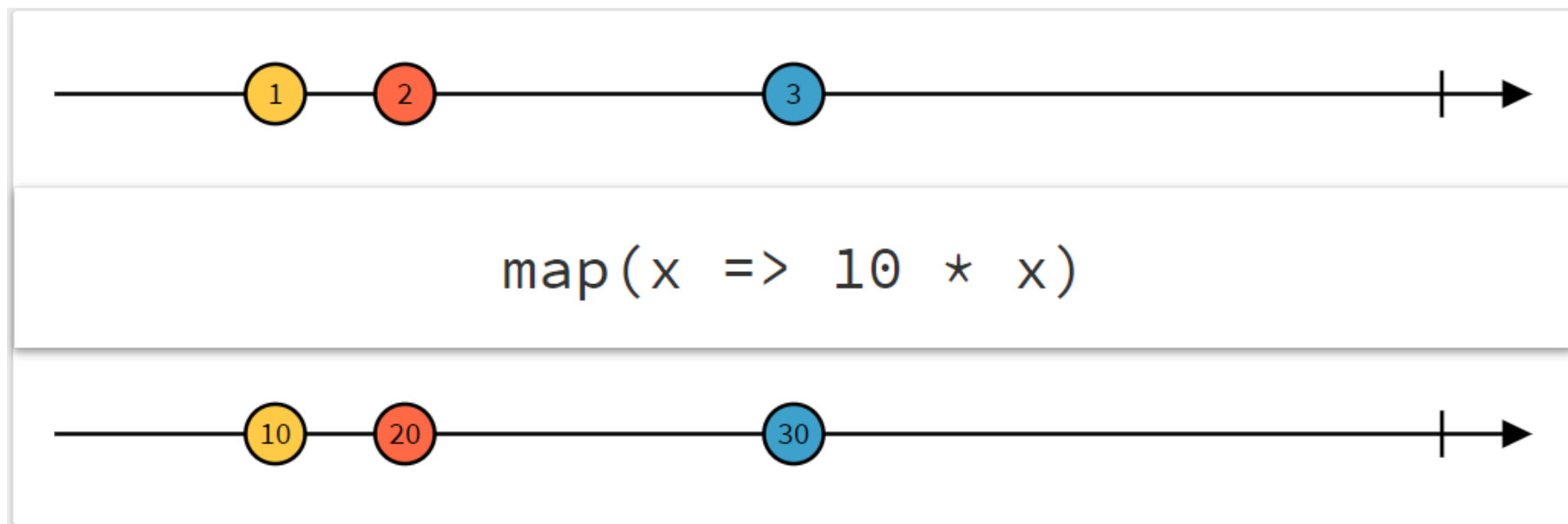
Les opérateurs de l'observable

- Les opérateurs sont des fonctions. Il y a deux types d'opérateurs :
- **Un opérateur pipeable** est une fonction qui prend un observable comme entrée et renvoie un autre observable. C'est une opération pure : le précédent Observable reste inchangé.
 - Syntaxe : `monObservable.pipe(opertaeur1(), operateur2(), ...)`.
- **Les opérateurs de création** sont l'autre type d'opérateur, qui peut être appelé comme fonctions autonomes pour créer un nouvel Observable. Par exemple : `of(1, 2, 3)` crée un observable qui va émettre 1, 2, et 3, l'un après l'autre.



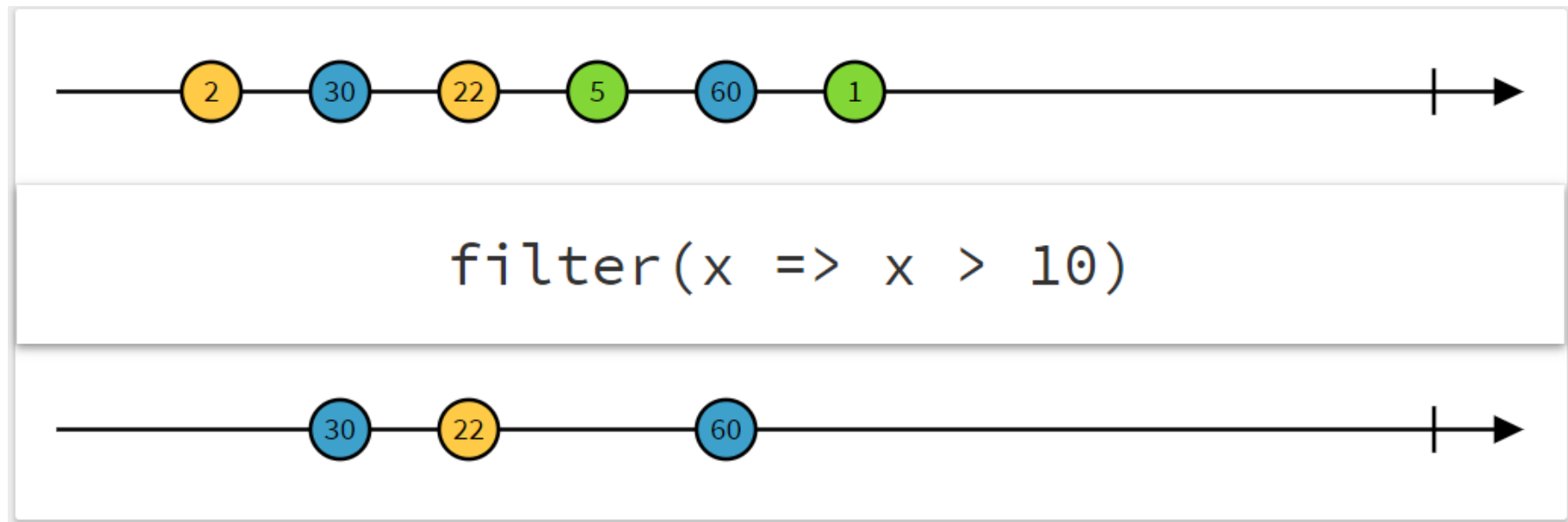
Quelques opérateurs utiles de l'Observable

map



Quelques opérateurs utiles de l'Observable

filter





Les opérateurs de création of

- l'opérateur `of` est utilisé afin de **convertir un argument en un observable**
- il ne fait **aucun aplatissement ou conversion** et **émet** chaque argument sous le **même type qu'il reçoit**
- `of` est couramment utilisé lorsque vous avez simplement **besoin de renvoyer une valeur là où un observable est attendu** ou de **démarrer une chaîne observable**.

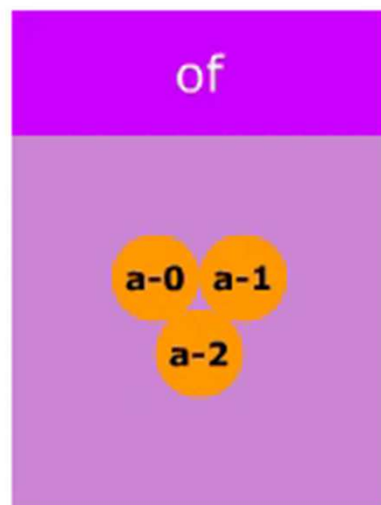
```
of([1, 2, 3], new Date(), {  
  name: 'sellaouti',  
  firstname: 'aymen',  
}).subscribe({  
  next: (data) => console.log('[of]', data),  
})
```

```
[of] ▶ (3) [1, 2, 3]  
[of] Tue Jan 03 2023 13:46:43 GMT+0100 (West Africa Standard Time)  
[of] ▶ {name: 'sellaouti', firstname: 'aymen'}  
[of] complete
```





Les opérateurs de création of





Les opérateurs RxJs

Gestion des erreurs, `catchError`

- L'opérateur `catchError` de RxJs permet de **capturer les erreurs qui se produisent dans un observable** et de les traiter de manière appropriée.
- Il permet de **continuer à émettre des valeurs depuis un observable** même si une erreur se produit, plutôt que de stopper complètement l'émission de valeurs.
- Il prend en argument une fonction qui prend en entrée l'erreur et **retourne un observable** pour gérer cette erreur.
- Si vous voulez retourner l'erreur dans votre flux utilisez l'opérateur `throwError`.





Les opérateurs RxJs

Gestion des erreurs, catchError

```
this.cvService.getCvById(param['id']).pipe(  
  catchError((e) => {  
    console.log(e);  
    this.router.navigate([APP_ROUTES.cv]);  
    return throwError(() => new Error(e));  
  })  
);
```





Les opérateurs RxJs

Gestion des erreurs, catchError

- Vous pouvez utiliser l'opérateur **EMPTY** de rxjs pour spécifier que vous **n'émettez rien** et qui émet ensuite une notification de **complétion** du flux.
- Vous pouvez aussi émettre ce que vous voulez afin de **continuer le flux**

```
catchError((e) => {  
    return EMPTY;  
}),
```

```
catchError((e) => {  
    return of(this.myService.getFakeData());  
}),
```





Quelques opérateurs utiles de l'Observable

<https://angular.io/guide/rx-library>

<http://reactivex.io/rxjs/manual/overview.html#operators>

<http://rxmarbles.com/>



Interagir avec une API Get Request

- Afin d'exécuter une requête `get` le module `http` nous offre une méthode `get`.
- Cette méthode retourne un `Observable`.
- Inscrivez vous à cet observable et définissez le comportement en cas de succès, erreur ou complétion

```
this.http.get(API_URL).subscribe({  
  next: (response: Response) => {  
    //ToDo with DATA  
  },  
  error: (err: Error) => {  
    //ToDo with error  
  },  
  complete: () => {  
    console.log('Data transmission complete');  
  }  
});
```



Interagir avec une API POST Request

- Afin d'exécuter une requête POST le module http nous offre une méthode post.
- Cette méthode retourne un Observable.
- Diffère de la méthode get avec un attribut supplémentaire : body

```
this.http.post(API_URL, dataToSend) .subscribe ({  
  next: (response: Response) => {  
    //ToDo with DATA  
  },  
  error: (err: Error) => {  
    //ToDo with error  
  },  
  complete: () => {  
    console.log('Data transmission complete');  
  }  
});
```



- Reprenez les deux fonctionnalités d'affichage de la liste des cvs et la page détailCv en utilisant l'api fourni par ce lien :

<https://apilb.tridevs.net/explorer/>



Sécuriser vos routes

- Dans vos applications, certaines routes ne doivent être accessibles que si vous êtes authentifié. Ce cas d'utilisation se répète souvent et s'est un sous cas de la sécurisation de vos routes.
- Angular a pris en considération ce cas en fournissant un mécanisme via l'utilisation des **Guard**.
- Ce sont des classes qui permettent de gérer l'accès à vos routes.
- Un guard informe sur la validité ou non de la continuation du process de navigation en retournant un booléen, une promesse d'un booléen ou un observable d'un booléen.
- Le routeur supporte plusieurs types de guards, par exemple :
 - **CanActivate** permettre ou non l'accès à une route.
 - **CanActivateChild** permettre ou non l'accès aux routes filles.
 - **CanDeactivate** permettre ou non la sortie de la route.





Guard

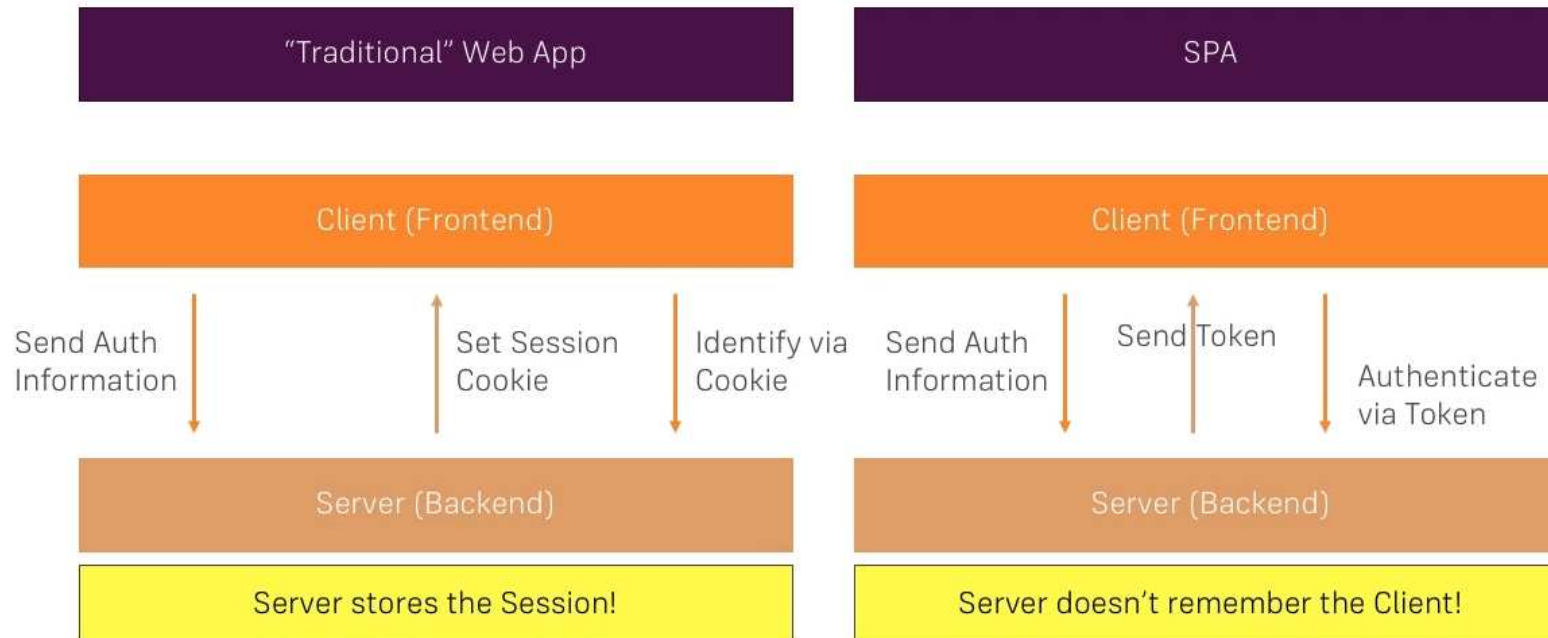
- A partir d'Angular 14 et la possibilité d'utiliser la fonction inject dans tous les contextes d'injection, Angular préconise les fonctionnels Guards.
- Pour créer un guard utilisez la commande `ng g g pathDuGuard`
- Si vous retourner `true` ou une promise de true ou un Observable de true **la route sera activée**.
- Si c'est `false` la **navigation** vers la route sera **annulée**
- Appliquer le guard sur la route ciblée

```
export const myGuard: CanActivateFn = (route, state) => {  
  return true;  
};
```

```
{  
  path: 'lampe',  
  component: ColorComponent,  
  canActivate: [AuthGuard]  
},
```



How does Authentication work?





Les headers

- Afin d'ajouter des headers à vos requêtes, le HttpClient vous offre la classe HttpHeaders.
- Cette classe est une classe immutable (read Only).
<https://angular.io/guide/http#immutability>
- Elle propose une panoplie de méthode helpers permettant de la manipuler.
- [set\(clé,valeur\)](#) permet d'ajouter des headers. Elle écrase les anciennes valeurs.
- [append\(clé,valeur\)](#) concatène de nouveaux headers.

Toutes les méthodes de modification retournent un HttpHeaders permettant un chainage d'appel.

<https://angular.io/api/common/http/HttpHeaders>





Les paramètres

- Afin d'ajouter des paramètres à vos requêtes, le HttpClient vous offre la classe HttpParams.
- Cette classe est une classe immutable (read Only).
- Elle propose une panoplie de méthode helpers permettant de la manipuler.
- `set(clé,valeur)` permet d'ajouter des headers. Elle écrase les anciennes valeurs.
- `append(clé,valeur)` concatène de nouveaux headers.

Toutes les méthodes de modification retournent un HttpParams permettant un chainage d'appel.

<https://angular.io/api/common/http/HttpParams>





Ajouter le token dans la requête

- Si la ressource demandée est contrôlée avec un token, vous devez y insérer le token afin d'être authentifié au niveau du serveur.
- Pour ajouter un token vous pouvez le faire via un objet `HttpParams`. Cet objet possède une méthode `set` à laquelle on passe le nom du token 'access_token' suivi du token.
- Vous devez ensuite l'ajouter comme paramètre à votre requête.

```
const params = new HttpParams()  
  .set('access_token', localStorage.getItem('token'));  
return this.http.post(this.apiUrl, personne, {params});
```

<https://angular.io/guide/http#immutability>





Ajouter le token dans la requête

- Une seconde méthode consiste à ajouter dans le header de la requête avec comme name 'Authorization' et comme valeur 'bearer' à laquelle on concatène le Token.
- Pour se faire, créer un objet de type HttpHeaders.
- Utiliser sa méthode append afin d'y ajouter ses paramètres.
- Ajouter la à la requête.

```
const headers = new HttpHeaders();  
headers.append('Authorization', 'Bearer ${token}');  
return this.http.post(this.apiUrl, personne, {headers});
```





Exercice

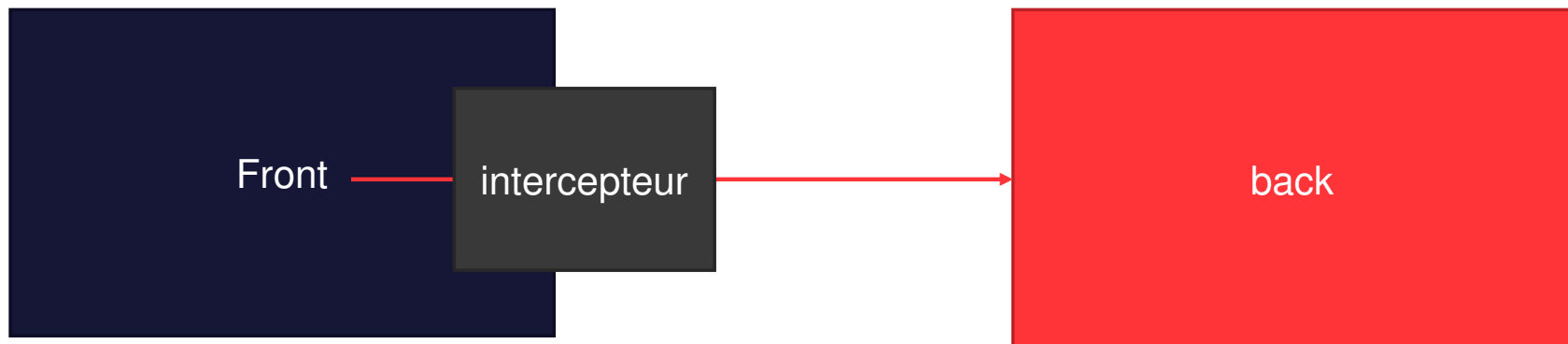
- Ajouter les guards nécessaires afin de sécuriser vos routes.
- Une personne déjà connectée ne peut pas accéder au composant de login.





Les intercepteurs

- A chaque fois que nous avons une requête à laquelle nous devons ajouter le token, nous devons refaire toujours le même travail.
- Pourquoi ne pas intercepter les requêtes HTTP et leur associer le token s'il est là à chaque fois ?
- Un intercepteur Angular (fournit par le client HTTP) va nous permettre **d'intercepter une requête à l'entrée et à la sortie de l'application.**
- Un intercepteur est soit une fonction (angular 17) ou une classe qui **implémente l'interface HttpInterceptor.**
- En implémentant cette interface, chaque intercepteur va devoir **implémenter** la méthode **intercept**.





Les intercepteurs

- Dans les anciennes versions d'angular on devait passer par une class qui implémente l'interface `HttpInterceptor`.

```
import { Injectable } from '@angular/core';
import { HttpRequest, HttpHandler, HttpEvent, HttpInterceptor, } from '@angular/common/http';
import { Observable } from 'rxjs';
@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  intercept( request: HttpRequest<unknown>, next: HttpHandler): Observable<HttpEvent<unknown>> {
    return next.handle(request);
  }
}
```





Les intercepteurs

- Dans les nouvelles versions d'angular nous passons par une **fonction**.
- Dans cette fonction vous récupérer comme premier paramètre l'objet **request** qui représente la **requête HTTP**.
- En deuxième paramètre, vous avez la fonction **next** qui vous permet de remettre la requête de continuer son parcours.

```
import { HttpInterceptorFn } from "@angular/common/http";  
export const newInterceptor: HttpInterceptorFn = (req, next) => {  
  return next(req);  
};
```





Les intercepteurs : changer la requête

- Par défaut la requête est immutable, on ne peut pas la changer.
- Solution : la cloner via sa méthode clone, changer les headers du clone et le renvoyer.

```
const cloneReq = request.clone({  
  setHeaders: {  
    'Authorization': token  
  } });  
// Chainer la nouvelle requete avec next.handle  
return next.handle(newReq);
```





Les intercepteurs

Appliquer l'intercepteur

- Afin d'appliquer l'intercepteur, vous devez le fournir au niveau du provider.
- Si vous utilisez un [intercepteur fonctionnel](#), vous utilisez la fonction [withInterceptors](#) qui est une des options de [provideHttpClient](#).
- Elle prend en paramètre un tableau d'intercepteur

```
bootstrapApplication(AppComponent, {providers: [  
  provideHttpClient(  
    withInterceptors([firstInterceptor, secondInterceptor]),  
  )  
]});
```





Angular Reactive Form





Reactive form

- Les Reactive Form sont une deuxième méthode de gérer vos formulaires avec Angular.
- Au contraire des Template Driven Form, ses formulaires sont **générés programmatiquement** dans la partie TS.
- Ceci permet **d'alléger le template** des validateurs.
- D'autre part ca permet d'avoir un **formulaire testable**
- Finalement ceci permet de plus facilement **générer des Validateurs personnalisés**.





Reactive form

Créer un formulaire via FormGroup

- Commencer par ajouter le module **ReactiveFormModule**.
- Afin de créer un formulaire avec l'approche réactive, vous devez créer un objet **FormGroup**.
- Un **FormGroup** prend en paramètre un objet décrivant le formulaire. Chaque champ de l'objet a comme **première propriété le nom et comme valeur un objet définissant les champs associés à ce formulaire**. Ce sont les **FormControl**.
- Chaque **FormControl** définit un **champ du formulaire**. Il prend en paramètre, **la valeur initiale**, un **Validator** ou un **tableau de Validator** et en troisième paramètre, un **AsyncValidator** ou un **tableau d'AsyncValidator**.

```
FormGroup.ctor({
  controls: {[p: string]: AbstractControl},
  validatorOrOpts?: ValidatorFn | ValidatorFn[] | AbstractControlOptions | null,
  asyncValidator?: AsyncValidatorFn | AsyncValidatorFn[] | null)
```

```
ngOnInit() {
  this.form = new FormGroup({
    name: new FormControl(null),
    firstname: new FormControl(null),
    age: new FormControl(null),
  });
}
```





Reactive form

Créer un formulaire

```
export declare interface AbstractControlOptions {  
  validators?: ValidatorFn | ValidatorFn[] | null;  
  asyncValidators?: AsyncValidatorFn | AsyncValidatorFn[] | null;  
  /**  
   * @description  
   * The event name for control to update upon.  
   */  
  updateOn?: 'change' | 'blur' | 'submit';  
}
```

```
form = new FormGroup({  
  name: new FormControl(null),  
  firstname: new FormControl("Aymen"),  
  age: new FormControl(0, {  
    asyncValidators: [],  
    validators: Validators.required,  
    updateOn: "blur",  
  }),  
});
```





Reactive form

Créer un formulaire via le service FormBuilder

- Vous pouvez aussi passer par le [service FormBuilder](#) et sa méthode [group](#).
- Elle prend en [paramètre un objet](#) avec comme [clé le nom](#) du FormControl et comme [valeur un tableau](#) avec comme [première propriété la valeur initiale du champ](#).
- Ceci est [moins verbeux](#) et plus simple à gérer pour les grands formulaires.

```
this.form = this.formBuilder.group({  
  email: [null],  
  username: [null],  
  userCategory: ['employee'],  
});
```

```
constructor(  
  private FormBuilder: FormBuilder  
) {}
```





Reactive form

Associer le FormGroup à votre form

- Une fois le formulaire définit, nous devons **l'associer au form de votre template**.
- Pour ce faire, vous devez ajouter la directive **formGroup** (qui se trouve dans le module **ReactiveFormsModule**) au niveau de la balise form et la binder à votre objet de type **FormGroup** au niveau de votre fichier TS.

```
reactive.component.ts | reactive.component.html
7  styleUrls: ['...']
8  })
9  export class Reactive
10
11  constructor(private
12  form: FormGroup;

1  <h3 class="text-center">User Details</h3>
2  <hr>
3  <form [formGroup]="form">
4  <div>
5    <label for="email">Email</label>
6    <input class="form-control" type="email">
```





Reactive form

Associer les FormControl à vos input

- Afin d'associer votre FormControl à votre champ input, utiliser la directive `formControlName` et associer le à l'identifiant du FormControl

```
this.form = new FormGroup( controls: {  
  email: new FormControl( formState: null),  
  username: new FormControl( formState: null),  
  age: new FormControl( formState: null),  
});
```

```
<div>  
  <label for="email">Email</label>  
  <input  
    class="form-control" type="email" id="email"  
    formControlName="email" placeholder="Enter email"/>  
</div>  
<div>  
  <label for="username">Username</label>  
  <input  
    class="form-control" type="text" id="username"  
    formControlName="username" placeholder="Username"/>  
</div>
```





Reactive form

Récupérer les FormControl dans le HTML

- Afin de récupérer les FormControl dans le HTML, utiliser la méthode `get` de votre form group et passer lui l'identifiant du FormControl à récupérer.

```
<button  
  class="btn btn-primary"  
  [disabled]="form.get('password').valid"  
  (click)="process()">  
  Submit  
</button>
```





Reactive form

Récupérer les erreurs du FormControl dans le HTML

- Afin de récupérer les erreurs de votre control dans le HTML, utiliser l'attribut `errors`.

```
<div
  *ngIf="form.get('name')?.errors && form.get('name')?.touched"
  class="alert alert-danger"
>
```





Reactive form Récupérer les FormControl dans le HTML

- Une deuxième méthode pour avoir un code moins verbeux est de créer des getters pour vos champs.

```
get name(): AbstractControl {  
    return this.form.get("name");  
}
```

```
<div *ngIf="name.errors && name.touched">  
    Ce champ est obligatoire  
</div>
```





Reactive form

Les Validateurs

- Un **validateur** est une **fonction** qui retourne **false** si un **champ est valide** selon une certaine condition **sinon elle retourne une information sur l'erreur**.
- Afin de valider votre formulaire, passer en deuxième paramètre de **FormControl** une **référence à une méthode de la classe Validators** ou un **tableau de ces méthodes**.

```
FormControl( formState: null, validatorOrOpts: [Validators.]),
  • email(control: AbstractControl)
  • required(control: AbstractControl)
  • compose(validators: null)
  • nullValidator(control: AbstractControl)
  • max(max: number)
  • composeAsync(validators: (AsyncValidatorFn | null)[])
  • maxLength(maxLength: number)
  • min(min: number)
  • minLength(minLength: number)
  • pattern(pattern: string | RegExp)
```





Reactive form

Les Validateurs

- **Validateurs synchrones** : Ce sont des fonctions qui contrôlent votre élément et retournent un **ensemble d'erreurs de validation ou null**. C'est ce qu'on passe en deuxième paramètre lors de l'instanciation d'un FormControl.
- **Validateurs asynchrones** : Ce sont des fonctions asynchrones qui contrôlent votre élément et retournent une **Promise ou un Observable** qui émettent un **ensemble d'erreurs de validation ou null**. C'est ce qu'on passe en troisième paramètre lors de l'instanciation d'un FormControl.
- Pour des raisons de **performance**, **Angular commence par les validateurs synchrones, s'ils passent il déclenche les validateurs asynchrones.**

```
formGroup: FormGroup = new FormGroup({  
  name: new FormControl(null, {  
    validators: [Validators.required, Validators.minLength(3)],  
    asyncValidators: [],  
    updateOn: "change"  
  }),  
  age: new FormControl(null),  
});
```





Reactive form

Manipulez les valeurs de votre form

- Afin de **mettre à jour la valeur** d'un **FormControl** ou d'un **FormGroup**, vous pouvez utiliser la méthode **setValue()** qui met à jour la valeur du contrôle de formulaire et valide la structure de la valeur fournie par rapport à la structure du contrôle.
- Vous pouvez utiliser la méthode **patchValue** si vous modifiez uniquement **une partie de votre formulaire**.

```
this.form.setValue({ name: "Sellaouti", firstname: "Aymen" });
```

```
this.form.patchValue({firstname: "Aymen" });
```





Reactive form

Suivre les modifications de vos FormControl et de vos FormGroup

- **FormControl** et **FormGroup**, vous fournissent deux **Observables** permettant le **suivi des changements de valeur et de statuts**.
- **ValueChanges** est un événement déclenché par les formulaires Angular **chaque fois que la valeur de FormControl, FormGroup change**. L'observable obtient la dernière valeur du contrôle. Il nous **permet de suivre les modifications apportées à la valeur en temps réel et d'y répondre**. Par exemple, nous pouvons l'utiliser pour valider la valeur, calculer les champs calculés, ...
- **statusChanges** est un événement déclenché par les formulaires Angular **chaque fois que Angular calcule le statut de validation** de FormControl, FormGroup. Il renvoie un observable afin que vous puissiez vous y abonner. L'observable obtient le **dernier état du contrôle**.



Exercice

- Afin de protéger les informations personnelles des mineurs, faites en sortes que lorsque l'âge de la personne possédant le Cv est inférieur à 18 ans, il ne puisse pas renseigner le path de l'image.



Exercice

- Etant donné que notre formulaire est assez volumineux et pour permettre une meilleure expérience utilisateur, nous voulons faire en sorte que si l'utilisateur saisit un formulaire valide et qu'il sorte de la page sans l'envoyer, il puisse le retrouver rempli lorsqu'il retourne à la page d'ajout d'un cv.





Reactive form Customiser vos validateurs

- Un custom validator est une fonction de type `ValidatorFn` qui prend en paramètre un control de type `AbstractControl` (qui contient une propriété `value` représentant la **valeur du champ à valider**) et **retourne null si la valeur est valide** ou un objet de type `ValidationErrors` s'il y a des **erreurs de validation**.

```
export declare interface ValidatorFn {  
  (control: AbstractControl): ValidationErrors | null;  
}
```

```
export function createPasswordStrengthValidator(): ValidatorFn {  
  return (control: AbstractControl): ValidationErrors | null => {  
    const value = control.value;  
    if (!value) {  
      return null;  
    }  
    const hasUpperCase = /[A-Z]+/.test(value);  
    const hasLowerCase = /[a-z]+/.test(value);  
    const hasNumeric = /[0-9]+/.test(value);  
    const passwordValid = hasUpperCase && hasLowerCase && hasNumeric;  
    return !passwordValid ? {passwordStrength: true} : null;  
  };  
}
```





Reactive form

Customiser vos validateurs

Validateurs Asynchrones

- Dans certains cas d'utilisation, la validation de votre champ ne se fait pas d'une façon **asynchrone**.
- Le cas le plus répandu est la **validation par votre backend**.
- Imaginez que vous avez un champ email et qu'il ne doit pas être redondant. La solution est d'avoir une API qui vérifie ça et qui vous retourne la réponse.
- Ce traitement étant Asynchrone, le Validateur synchrone ne fait plus affaire.
- Il faut donc créer un validateur **ASYNCHRONE**.
- Le Validator, renvoyée par la fonction de création de validateur, doit suivre ces règles :
- **Un seul argument** d'entrée est attendu, qui est de type **AbstractControl**. La fonction validateur peut obtenir la **valeur à valider** via la propriété **control.value**
- La fonction de validation **doit renvoyer une Promise<null>, un Observable<null>** si **aucune erreur** n'a été trouvée dans la valeur du champ, ce qui signifie que la **valeur est valide**
- Si des **erreurs de validation sont trouvées**, la fonction doit renvoyer **une Promise ou un Observable** d'un objet de type **ValidationErrors**.

```
export function userExistsValidator(authService: AuthService): AsyncValidatorFn {  
  return (control: AbstractControl) => {  
    return authService.findUserByEmail(control.value).pipe(  
      map(response => {  
        // Si pas d'erreur on retourne null  
        // S'il y a une erreur on retourne un objet décrivant l'erreur  
      })  
    );  
  };  
}
```





Exercice

- Le champ Cin étant unique, créer un validateur asynchrone permettant de gérer cette contrainte et tester le.





Reactive form

Customiser vos validateurs

Valider un form

- Afin de valider un form, vous devez faire la même chose que pour le validateur d'un FormControl.

```
export function createDateRangeValidator(): ValidatorFn {  
  return (form: AbstractControl): ValidationErrors | null => {  
    const start: Date = form.get('startAt').value;  
    const end: Date = form.get('endAt').value;  
    if (start && end) {  
      const isRangeValid = (end.getTime() - start.getTime() > 0);  
      return isRangeValid ? null : {dateRange: true};  
    }  
    return null;  
  };  
}
```

- Maintenant et pour l'appliquer à votre FormGroup ajouter à la création de votre FormGroup un second paramètre qui est un objet options et utiliser la propriété validators

```
this.form = new FormGroup(  
  {},  
  {  
    validators: VotreValideur  
  }  
);  
this.form = this.formBuilder.group(  
  {},  
  {  
    validators: VotreValideur  
  }  
);
```





Exercice

- Le champ Cin étant composé de 8 caractères numériques, il présente une corrélation entre l'âge de la personne et les deux premiers caractères.
- Si la personne a un âge ≥ 60 ans, les deux premiers caractères numériques doivent être entre 00 et 19.
- Sinon ça doit être supérieur à 19.
- Créer le validateur permettant de faire ça.



Angular Signaux et RxJs





Quand utiliser Les signaux

- Les signaux sont excellents pour **gérer l'état**
- Les signaux **ont toujours une valeur**.
- Tout ce qui est **synchrone** peut être modélisé avec des signaux.
- Les signaux **n'émettent rien** : un **consommateur doit extraire leur valeur en cas de besoin, et une valeur sera toujours renvoyée (de manière synchrone)**.





Quand utiliser RxJs

- Vous avez besoin d'un observable, et non d'un signal, lorsque :
 - Vous vous souciez du **moment** où la valeur sera émise ;
 - Vous **ne pouvez pas renvoyer une valeur instantanément** ;
 - Vous vous souciez de **la complétion** : un **observable** peut être **complet**, un **signal** est **permanent** ;
 - vous souhaitez **filtrer ou retarder le moment où vous émettez la valeur**.
- **Événements DOM**, requêtes **XHR**, **saisie utilisateur** : vous pouvez facilement les utiliser avec **debounceTime()**, **concatMap()**, **take(1)**, **skip(2)**, **forkJoin()**, et annuler les requêtes avec **switchMap()**.
- Si votre source de données ne peut pas répondre à la question « **Quelle est la valeur actuelle ?** » **sans délai**, elle **ne peut pas être présentée comme un signal**.
- Les signaux se distinguent par un autre domaine : **la représentation de l'état de l'application**.





signaux et rxJs

Interopérabilité

- Les signaux **ne sont pas la pour enlever RxJs** mais plutôt pour **limiter** son utilisation là où il est le **plus approprié** de l'utiliser.
- **RxJS** se marie mieux avec les **tâches asynchrones** alors que les **signaux** sont **synchrones**.
- Nous **pouvons utiliser des signaux et des observables ensemble**, et nous pouvons **les convertir l'un en l'autre**.
- **Deux fonctions**, pour faire cela, sont disponibles dans le tout package : **@angular/core/rxjs-interop**
 - **toObservable** qui convertit un signal en un observable
 - **toSignal** qui convertit un observable en un signal. Elle prend en premier paramètre l'observable et en second paramètre un objet **ToSignalOptions** permettant de définir **la valeur initiale**.





signaux et rxJs

ToSignalOptions

- `toSignal` permet de **convertir un observable en un Signal**
- `toSignal` s'inscrit à l'observable et crée un signal qu'il **modifie à chaque nouvelle valeur reçue du flux**.
- `toSignal` gère la désinscription de l'inscription. Il a donc besoin d'injecter l'injectable `DestroyRef` et donc d'un contexte d'injection.
- Le **deuxième paramètre de `toSignal`** et un objet d'option de type **`ToSignalOptions`**

```
interface ToSignalOptions<T> {  
  initialValue?: unknown;  
  requireSync?: boolean | undefined;  
  injector?: Injector | undefined;  
  manualCleanup?: boolean | undefined;  
  equal?: ValueEqualityFn<T> | undefined;  
}
```





signaux et rxJs toSignal

```
interface ToSignalOptions<T> {  
  initialValue?: unknown;  
  requireSync?: boolean | undefined;  
  injector?: Injector | undefined;  
  manualCleanup?: boolean | undefined;  
  equal?: ValueEqualityFn<T> | undefined;  
}
```

- **initialValue**: unknown : Définit une valeur initiale pour le Signal avant la première émission de l'Observable.
- **requireSync**: boolean : Exige que l'Observable émette immédiatement une valeur synchrone, sinon.
- **injector**: **Injector** : Spécifie l'injecteur pour gérer le nettoyage de l'abonnement à l'Observable.
- **manualCleanup**: boolean : Désactive le nettoyage automatique de l'abonnement, laissant la responsabilité à l'utilisateur.
- **equal**: ValueEqualityFn<T> : Fournit une fonction personnalisée pour déterminer si une nouvelle valeur justifie une mise à jour du Signal.

```
cvs = toSignal(  
  this.todoService.getTodos(),  
  { initialValue: [] }  
);
```





Signal et les appels HTTP

- Un **signal** est une opération **Synchrone**. Quand vous faites **un appel http** ceci revient à faire une **opération Asynchrone**.
- Afin de **briser le bridge entre ces deux mondes** et récupérer les informations concernant vos requêtes http pour **récupérer des données et les transformer en signals représentant votre State**, angular a introduit **3 API qui sont encore en Experimental** cad qu'elles peuvent ne pas devenir stables ou avoir des modifications significatives. Ces API's sont

Experimental

Might not become stable at all or have significant changes

Developer Preview

Fully functional but not ready to stabilize

Stable

Production ready with full support

Deprecated

Obsolete API, should be replaced

Removed

Feature has been completely removed

resource()

×

All Packages

All Statuses

Feature	Package	21	20	19	18	17	16	15	14
resource()	@angular/core								
rxResource()	@angular/core/rxjs-interop								
httpResource()	@angular/common/http			19.2					





Signal et les appels HTTP

➤ Si on se pose la question suivante : De quoi pouvons nous avoir besoins lorsqu'on fait des appels HTTP ?

- Résultat de l'appel en cas de succès.
- Message d'erreur en cas d'erreur
- Savoir est ce que l'appel est en cours de traitement
- Pouvoir relancer un appel quand les paramètres dont il dépend changent

Resource API



- Les nouveaux API Resource sont la pour faire le pont entre le monde asynchrone de RxJs et Synchrone des Signals et répondre à tous ces questions



Les ressources API

Basique	<code>resource()</code>	Se base sur les promesses	Utilisez le si vous préférez l'api fetch de ES pour récupérer vos données
Reactive	<code>rxResource()</code>	Utilises les Observables RxJs	Utilisez le si vous préférez travailler avec RxJs pour vos appels http ou que vous avez besoins d'utiliser les opérateurs de RxJs
Pragmatique	La plus simple et la plus flexible.	Le dernier de la série et le plus simple à implémenter.	C'est un Wrapper sur le HttpClient, il nécessite une url et les paramètres dont il dépend. Offre une deuxième version pour les requêtes les plus complexes



Les propriétés d'une ressource

- Tous les types de ressources étendent l'interface `Resource` :

```
interface Resource<T> {  
  /**  
   * The current value of the `Resource`, or `undefined` if there is no current value.*/  
  readonly value: Signal<T>;  
  /**  
   * The current status of the `Resource`, which describes what the resource is currently doing and  
   * what can be expected of its `value`.*/  
  readonly status: Signal<ResourceStatus>;  
  /**  
   * When in the `error` state, this returns the last known error from the `Resource`.*/  
  readonly error: Signal<unknown>;  
  /**  
   * Whether this resource is loading a new value (or reloading the existing one).*/  
  readonly isLoading: Signal<boolean>;  
  /**  
   * Whether this resource has a valid current value.  
   *  
   * This function is reactive. */  
  hasValue(): this is Resource<Exclude<T, undefined>>;  
  /**  
   * Instructs the resource to re-load any asynchronous dependency it may have.  
   * @returns true if a reload was initiated, false if a reload was unnecessary or unsupported  
   */  
  reload(): boolean;  
}
```





Signal et les appels HTTP

httpResource

- Dans **Angular 19.2** elle pouvait récupérer une **chaîne, une fonction ou un objet de type `HttpRequest`** permettant de décrire l'appel http à effectuer.
- A partir de la **version 20** on ne peut utiliser **qu'une fonction qui retourne une chaîne ou un objet de type `HttpRequest`**.
- L'introduction de `httpResource` rend la gestion des requêtes HTTP **plus déclarative et réactive**.
- **Au contraire du `HttpClient`, la `httpResource` déclenche son appel http dès sa création**. Ensuite elle **se recharge automatiquement** et effectue une nouvelle requête à **chaque changement d'un signal source** dont dépend la fonction.
- Dans cet exemple, à chaque fois que le **signal `search` change un nouvel appel est exécuté**

```
userHttpResource = httpResource(() => API.users + `?name_like=${this.search()}`);
```





Signal et les appels HTTP

httpResource

- Si vous avez besoin d'un **contrôle plus précis sur la requête**, vous pouvez transmettre une fonction qui renvoie un objet **HttpRequest**.
- Outre la propriété **URL**, elle peut spécifier et réagir à des options telles que la **méthode**, les **paramètres** ou **les en-têtes**.

```
userHttpRequest = httpRequest(() => ({
  url: API.users,
  params: new HttpParams().set('name_like', `^${this.search()}` ),
  method: 'GET'
}));
```



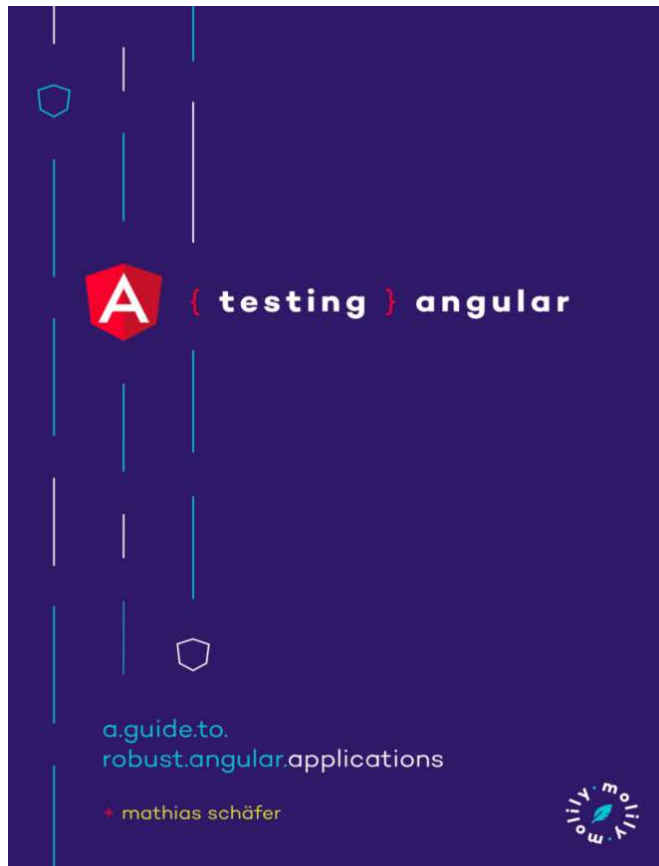


Jasmine

Angular
Tests Unitaires



Références





Tests unitaires : Introduction

- La **plus petite unité de test possible**
- Couvre une **petite fonctionnalité** et ne s'occupe pas de comment les différentes unités testées travaillent ensemble.
- Il est **isolé** et ne doit **pas dépendre d'autres tests**.
- Rapide, fiable et pointe directement sur le bug en question.
- Rend votre **code plus robuste**, le bug sera identifié plus tôt
- Vous permet de **formaliser et de documenter** vos besoins
- Un bon test décrit clairement comment le code d'implémentation doit se comporter
- Un bon test doit couvrir **les scénarios les plus importants**
- Les tests rendent le changement sûr en **empêchant les régressions**





Tests unitaires : Partout ?

- Alors devriez-vous écrire des tests automatisés pour tous les cas possibles afin de garantir l'exactitude ?
- Non, disent les principes de l'ISTQB : "Les tests exhaustifs sont impossibles".
- Il n'est **ni techniquement faisable ni utile** d'écrire des tests pour toutes les entrées et conditions possibles.
- Au lieu de cela, vous devez **évaluer les risques** d'un certain cas et rédiger d'abord des **tests pour les cas à haut risque**.
- Même s'il était viable de couvrir tous les cas, cela vous donnerait un **faux sentiment de sécurité**.





Angular et les tests unitaires

- Angular avec sa structuration et ses différentes couches se marie parfaitement avec les tests unitaires.
- Chaque couche ayant un rôle unique et une tâche bien spécifique, les tests unitaires seront donc propres à chaque partie, composant, pipe, service, directive, ...
- L'utilisation de l'injection de dépendance implique le couplage faible et donc des tests isolés.
- Les providers qui permettent de fournir des classes fictives facilitent aussi cette notion d'isolation.



Jasmin et Karma

- **Jasmin** est un framework permettant de faciliter la création de tests. Il contient un ensemble de fonctionnalités permettant d'écrire plusieurs types de test.



Jasmine



karma est un task runner pour vos tests. Il utilise un fichier de configuration afin de gérer le process de test en identifiant les fichiers de chargement, le framework de test, le navigateur à lancer...

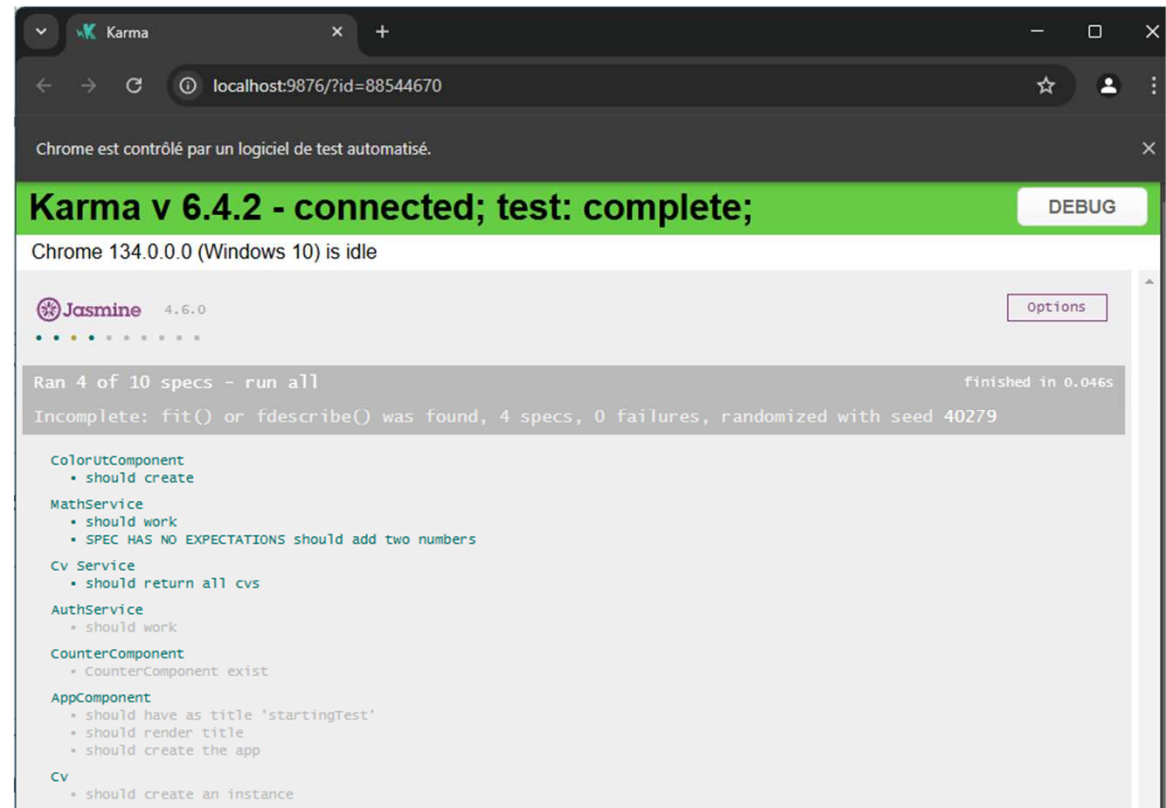




Lancement d'un test

- Afin de lancer un test, vous avez juste besoin d'une seule commande et Karma fait le reste. Il exécutera les tests, ouvrira le navigateur, et affichera un rapport sur l'ensemble des tests.

`ng test`





Concepts de base de jasmine describe (suite)

- Pour ce qui est de Jasmine, un test se compose d'une ou plusieurs suites. Une suite est déclarée avec un bloc describe :

```
describe('Suite description', () => {  
  /* ... */  
});
```

- Chaque suite décrit un morceau de code, le code à tester.

<https://testing-angular.com/>





Concepts de base de jasmine Specification (**it**)

- Chaque **describe** se compose d'une ou plusieurs **spécifications**.
- Une **spécification** est déclarée avec un bloc **it** :

```
describe('description de la suite', () => {  
  it('description de la spécification', () => {  
    /* ... */  
  });  
});
```

- **it** est une **fonction** qui prend deux paramètres.
 - Le premier paramètre est une **chaîne** avec une **description lisible**
 - Le second paramètre est une **fonction** contenant **le code de votre test**





Concepts de base de jasmin Specification (**it**)

- Pour écrire le titre de votre test, le **it**..., demandez vous ce que doit faire le code que vous testez.
- Pour une LampeComponent par exemple, il doit allumer et éteindre la lampe, on aura donc :

```
it('switch on the lamp', () => {  
  /* ... */  
});  
it('switch off the lamp', () => {  
  /* ... */  
})
```

- Après it, un verbe suit généralement, comme switch on, une deuxième famille de testeur préfère suivre le it par **should**





Concepts de base de jasmin

Specification (it)

- À l'intérieur du bloc **it** se trouve le code de test réel.
- Indépendamment du framework de test, le code de test se compose généralement de trois phases :
 - **Arrange**
 - **Act**
 - **Assert.**
- **Arrange** est la **phase de préparation** et de mise en place. Par exemple, la classe testée est instanciée. Les dépendances sont mises en place. Des espions (spy) et des faux sont créés.
- **Act** est la **phase où l'interaction** avec le code testé. Par exemple, une méthode est appelée ou un élément HTML du DOM est cliqué.
- **Assert** est la **phase où le comportement du code est contrôlé** et vérifié. Par exemple, la sortie réelle est comparée à la sortie attendue.





Concepts de base de jasmine Specification (it)

- Imaginons que nous voulons tester un service qui permet de compter les caractères distincts d'une chaîne
- Nous commençons par tester l'addition.
 - **Arrange**
 - Nous devons créer une instance du service et de ses dépendances s'ils existent
 - **Act**
 - Appeler la fonction `countDistinctCar` avec ses paramètres
 - **Assert.**
 - Vérifier que la fonction retourne le bon résultat

```
describe("Utils Service", () => {  
  it('should count distinct car', () => {  
    //Arrange  
    const utilsS = new UtilsService();  
  
    //Act  
    const result = utilsS.countDistinctCar('aymen sellouti');  
  
    // Assert  
    const expectedResult = 11;  
    expect(result).toEqual(expectedResult);  
  });  
});
```





Concepts de base de jasmin

Attente (Expectation)

- Dans la phase **d'affirmation** (**Assert**), le test **compare** la **sortie** ou la **valeur de retour réelle** à la **sortie ou à la valeur de retour attendue**. S'ils sont **identiques**, le test **réussit**. S'ils **diffèrent**, le test **échoue**.
- Afin de gérer ça, jasmine nous offre la fonction **expect**.
- Cette **fonction** est **associée** à un ensemble de **matchers** permettant de **faciliter la validation** de vos **attentes ou expectations**.

```
expect(actualValue).toBe(expectedValue);
```





Jasmin matchers

- Les **matchers** de Jasmine sont des **fonctions** qui permettent de **tester si une valeur donnée correspond à une condition spécifique**. Ils permettent donc de vérifier que les fonctionnalités de l'application se comportent comme prévu.
- **toBe()** : vérifie si deux valeurs sont strictement égales (utilisant l'opérateur "===")
- **toEqual()** : vérifie si deux objets ont les mêmes propriétés et les mêmes valeurs
- **toMatch()** : vérifie si une chaîne de caractères correspond à une expression régulière
- **toBeDefined()** : vérifie si une variable est définie
- **toBeUndefined()** : vérifie si une variable n'est pas définie
- **toBeNull()** : vérifie si une variable est null
- **toBeTruthy()** : vérifie si une expression est vraie
- **toBeFalsy()** : vérifie si une expression est fausse
- **toContain()** : vérifie si un tableau ou une chaîne de caractères contient un élément spécifié
- **toBeLessThan()** : vérifie si une valeur est inférieure à une autre
- **toBeGreaterThan()** : vérifie si une valeur est supérieure à une autre
- ...

<https://jasmine.github.io/api/edge/matchers.html>





Concepts de base de jasmin

- **describe (string, function)** : fonction qui prend en paramètre un titre et une ensemble de test individuel.
- **it (string, function)** : fonction représentant un **test** individuel qui prend en paramètre un titre et une fonction définissant un test individuel.
- **expect** : fonction qui retourne un booléen et évalue une expectation un besoin à valider par le test unitaire.

Exemple **expect**(etatActuel).**toBe**(etatExpecté)

- **les matchers** : sont des helpers prédéfinis permettant différentes validations.





Concepts de base de jasmin

- `xit` permet d'exclure un test individuel
- `xdescribe` permet d'exclure tout le bloc
- `fit` permet de spécifier le test individuel à exécuter
- `fdescribe` permet de spécifier le bloc à exécuter.



Exercice

- Récupérer le Répo suivant :
<https://github.com/aymensellaouti/startingTest>
- Créer les tests nécessaires pour le service MathService





Concepts de base de jasmine

- Lorsque vous écrivez plusieurs spécifications dans une suite, vous réalisez rapidement que **la phase d'arrangement (Arrange) est similaire**, voire identique, dans toutes ces spécifications.
- Par exemple, lors du test du MathService, la phase Arrange consiste toujours à créer une instance de MathService.
- Afin de centraliser ces traitements réplétifs, Jasmine propose quatre fonctions : **beforeEach**, **afterEach**, **beforeAll** et **afterAll**. Ils sont **appelés à l'intérieur d'un bloc describe**.
- Ils attendent un **paramètre**, une **fonction** qui est appelée **aux étapes données**.





Concepts de base de jasmine

Jasmin offre des handlers permettant de répéter certaines fonctionnalités.

- **beforeEach** : prend en paramètre une **callback** et la **répète** avant chaque spec **it**.
- **afterEach** : prend en paramètre une **callback** et la **répète** après chaque spec **it**.
- **beforeAll** : prend en paramètre une **callback** et la **répète** avant chaque suite **describe**.
- **afterAll** : prend en paramètre une **callback** et la **répète** après chaque suite **describe**.





Angular
Playwright





Introduction à Playwright



- Framework open-source de Microsoft pour le test E2E.
- Permet de tester des applications sur tous les navigateurs modernes (Chromium, Firefox, WebKit/Safari) avec une seule API.
- Conçu pour le web moderne (Single Page Applications comme Angular) où les éléments sont dynamiques.



<https://learn.microsoft.com/>



Pourquoi Playwright

- **API unifiée** : Playwright **fonctionne sur tous les moteurs de navigateur modernes** (Chromium, WebKit, Firefox) et prend en charge **l'émulation d'appareil pour la couverture mobile**. Il comprend à la fois les options de navigateur avec interface graphique et sans interface graphique, permettant aux développeurs de choisir entre la commodité de **débogage et l'exécution en CI/Cloud**.
- **Test résilient** : Playwright implémente « **l'attente automatique** » (aucun délai d'attente artificiel) et « **réessai automatique** » (assertions web), éliminant les causes clés des tests instables.
- **Isolation des tests** : **chaque test s'exécute dans son propre *BrowserContext***, indépendamment des autres tests exécutés en même temps. Les tests sont exécutés en parallèle (pour l'optimisation) et un échec de test n'affecte pas les autres (pour la fiabilité).
- **Outils puissants** : Playwright offre une interface CLI ou l'extension Visual Studio Code.

<https://learn.microsoft.com/>





Playwright Introduction

- Afin d'installer playwright passez par la commande suivante

`npm init playwright`

```
TS playwright.config.ts U X
TS playwright.config.ts > [e] default > [u] use > [u] baseURL
12  * See https://playwright.dev/docs/test-configuration
13  */
14  export default defineConfig({
15    testDir: './tests',
16    /* Run tests in files in parallel */
17    fullyParallel: true,
18    /* Fail the build on CI if you accidentally left test files in src.
19    forbidOnly: !!process.env.CI,
20    /* Retry on CI only */
21    retries: process.env.CI ? 2 : 0,
22    /* Opt out of parallel tests on CI. */
23    workers: process.env.CI ? 1 : undefined,
24    /* Reporter to use. See https://playwright.dev/docs/test-reporters
25    reporter: 'html',
26    /* Shared settings for all the projects below. See
27    use: {
28      /* Base URL to use in actions like `await page.goto('/')`
29      baseURL: 'http://localhost:4200',
30
31      /* Collect trace when retrying the failed test. See
32      trace: 'on-first-retry',
33    },
34
35    /* Configure projects for major browsers */
36    projects: [
37      {
38        name: 'chromium',
39        use: { ...devices['Desktop Chrome'] },
40      },
41    ],
42  });
```

```
webServer: {
  command: 'npm run start',
  url: 'http://localhost:4200',
  reuseExistingServer: !process.env.CI,
},
```

```
"devDependencies": {
  "@angular/build": "^20.3.15",
  "@angular/cli": "^20.3.15",
  "@angular/compiler-cli": "^20.3.0",
  "@faker-js/faker": "^10.3.0",
  "@playwright/test": "^1.58.2",
  "@types/jasmine": "~5.1.0",
  "@types/node": "^25.3.3",
  "commander": "^14.0.3",
  "jasmine-core": "~5.9.0",
  "karma": "~6.4.0",
  "karma-chrome-launcher": "~3.2.0",
  "karma-coverage": "~2.2.0",
  "karma-jasmine": "~5.1.0",
  "karma-jasmine-html-reporter": "~2.1.0",
  "ts-node": "^10.9.2",
  "typescript": "~5.9.2"
}
```





Qu'est ce qu'un test playwright

- Un test Playwright est un script asynchrone qui ressemble beaucoup à ce qu'on trouve dans d'autres frameworks (comme Jest ou Jasmine en Angular), mais avec une puissance de contrôle du navigateur en plus.
- Playwright vous propose la **fonction test qui définit un bloc de test**. Elle prend en paramètre une **chaîne décrivant le test** et la **fonction de test asynchrone** qui permet de **recupérer la page dans un objet page** et de la tester.

```
import { test, expect } from '@playwright/test';

test('has title', async ({ page }) => {
  await page.goto('https://playwright.dev/');

  // Expect a title "to contain" a substring.
  await expect(page).toHaveTitle(/Playwright/);
});
```



Lancer le premier test

- Pour manipuler Playwright, vous avez besoin de ces trois commandes :
- `npx playwright test` : Lance tous les tests en arrière-plan (mode "Headless", très rapide).
- `npx playwright test --ui` : Ouvre l'interface interactive (le meilleur moyen pour déboguer).
- `npx playwright show-report` : Affiche un rapport HTML détaillé après les tests.
- Je vous conseille d'ajouter les commandes dans la partie script de package.json

```
"e2e:test": "playwright test",  
"e2e:testui": "playwright test --ui",  
"e2e:report": "playwright show-report"
```

L'objet page

L'objet page via son api vous permet de donner des ordres au navigateur.

- **Navigation** : `page.goto('/login')` "Ouvre cette page".
- **Interaction** : `page.click()`, `page.fill()` "Agis sur le contenu".
- **Observation** : `page.screenshot()`, `page.url()` "Regarde ce qu'il se passe".
- **Attente** : `page.waitForSelector()` "Patiente jusqu'à ce que...".



Les 3 piliers de l'API de Playwright

Pour tester une application Angular, Playwright repose sur trois concepts simples :

- **Locators (Sélecteurs)** : "Où se trouve l'élément ?"
- **Actions** : "Que fait l'utilisateur ?" (cliquer, taper, cocher).
- **Assertions** : "Quel est le résultat attendu ?"

Trouver les éléments (Locators)

- Il existe deux familles de sélecteurs :
 - Les locators « *User-Centric* »
 - Les sélecteurs CSS
- Afin d'identifier lequel choisir, vous pouvez suivre cette démarche :
 1. **Le Rôle 'User-Centric'** (getByRole) Pourquoi ? Car si le test échoue, c'est souvent que votre application n'est pas accessible (ex: une image sans texte alternatif). On teste comme un humain.
 2. **Les sélecteur CSS** avec un attribut de test pour les éléments dynamiques ou sans rôle sémantique.
 - Il est déconseillé d'utiliser les sélecteurs susceptibles d'être modifiés fréquemment comme les **classes** ou les **ids** c'est un **Anti-Pattern**
 - La bonne pratique est d'utiliser des **attributs** avec ce **pattern data-*** permettant de donner un contexte à vos sélecteurs et de les **isoler des changements css et js**.
- Exemple data-testid, data-test, data-playwright

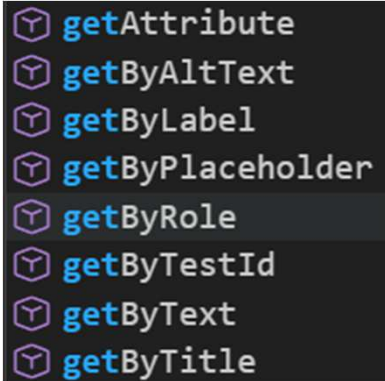
<https://playwright.dev/docs/locators>



Trouver les éléments (Locators)

Exemples

- `page.locator('[data-testid=login-btn]')`
- `page.getByRole('button', { name: 'Connexion' })` : Cible un bouton par son texte
- `page.getByText('Bienvenue')` : Trouve un texte spécifique à l'écran.
- `page.getByLabel('Nom d'utilisateur')` : Cible un champ de formulaire via son label.
- `page.getByPlaceholder('votre@email.com')` : Cible par l'indice dans le champ.
- `page.getByPlaceholder('votreTestId')` : Cible par l'attribut data-testid



```
getAttribute  
getByAltText  
getByLabel  
getByPlaceholder  
getByRole  
getByTestId  
getByText  
getTitle
```

<https://playwright.dev/docs/locators>



Quels rôles utiliser ? (Les plus courants)

Élément HTML	Rôle Playwright	Exemple
<button>	button	page.getByRole('button', { name: 'Envoyer' })
<a>	link	page.getByRole('link', { name: 'En savoir plus' })
<input type="text">	textbox	page.getByRole('textbox', { name: 'Email' })
<h1> à <h6>	heading	page.getByRole('heading', { name: 'Mon Profil' })
<input type="checkbox">	checkbox	page.getByRole('checkbox', { name: 'Accepter' })

```
page.getByRole('alert', {  
  checked?  
  disabled?  
  exact?  
  expanded?  
  includeHidden?  
  // level?  
  name?  
  ex pressed?  
  /* selected?
```

```
getByRole(role:  
  "alert"|"alertdialog"|"application"|"article"|"banner"|"blockquote"|"button"|"caption"|"cell"|"checkbox"|"code"|"columnheader"|"combobox"|"complementary"|"contentinfo"|"definition"|"deletion"|"dialog"|"directory"|"document"|"emphasis"|"feed"|"figure"|"form"|"generic"|"grid"|"gridcell"|"group"|"heading"|"img"|"insertion"|"link"|"list"|"listbox"|"listitem"|"log"|"main"|"marquee"|"math"|"meter"|"menu"|"menubar"|"menuitem"|"menuitemcheckbox"|"menuitemradio"|"navigation"|"none"|"note"|"option"|"paragraph"|"presentation"|"progressbar"|"radio"|"radiogroup"|"region"|"row"|"rowgroup"|"rowheader"|"scrollbar"|"search"|"searchbox"|"separator"|"slider"|"spinbutton"|"status"|"strong"|"subscript"|"superscript"|"switch"|"tab"|"table"|"tablist"|"tabpanel"|"term"|"textbox"|"time"|"timer"|"toolbar"|"tooltip"|"tree"|"treegrid"|"treeitem", options?: {
```

<https://playwright.dev/docs/locators>



Interagir (Actions)

- Les **actions** sont là pour vous permettre de **déclencher des actions** que peut faire l'utilisateur de votre page. C'est généralement les événements du DOM.
- Toutes les actions sont **asynchrones** (await) et incluent l'**auto-waiting** (Playwright attend que l'élément soit visible et cliquable).

```
// Cliquer sur un bouton
await page.getByRole('button', { name: 'Valider' }).click();
// Cliquer sur un bouton invisible en utilisant le DOM
await detailBtn.dispatchEvent('click')

// Saisir du texte
await page.getByPlaceholder('Rechercher...').fill('Angular');

// Cocher une case ou choisir une option
await page.getByLabel("J'accepte les conditions").check();
await page.locator('select').selectOption('Premium');
```

<https://playwright.dev/docs/input>





Vérifier (Assertions)

- Les assertions (vérification) de vos tests sont effectuées via la fonction `expect`.
- Une assertion est associée à un matcher. C'est la fonction qui définit ce que nous voulons avoir.
- La liste est très longue et illustrée dans la documentation.

```
// Vérifier qu'un message de succès apparaît
await expect(page.getByText('Données enregistrées')).toBeVisible();

// Vérifier l'état d'un bouton
await expect(page.getByRole('button')).toBeDisabled();

// Vérifier l'URL actuelle
await expect(page).toHaveURL(/.*dashboard/);
```

Auto-retrying assertions
Non-retrying assertions
Asymmetric matchers
Negating matchers

<https://playwright.dev/docs/test-assertions>





Vérifier (Assertions les plus utilisées)

Visibilité et existence

```
// toBeVisible() - L'élément est visible dans le DOM
await expect(page.locator('[data-testid=card]')).toBeVisible();

// toBeHidden() - L'élément est caché (display: none, visibility: hidden)
await expect(page.locator('[data-testid=modal]')).toBeHidden();

// toHaveCount(n) - Vérifier le nombre d'éléments
await expect(page.locator('li')).toHaveCount(8);

// toBeDefined() - L'élément existe
await expect(page.locator('[data-testid=cv$List]')).toBeDefined();
```

Contenu et texte

```
// toHaveText(text) - Texte exact
await expect(page.locator('h1')).toHaveText('My CV');

// toContainText(text) - Contient le texte
await expect(page.locator('button')).toContainText('Submit');

// toHaveTitle(regex) - Titre de la page
await expect(page).toHaveTitle(/Docapost/);

// toHaveValue(value) - Valeur d'un input
await expect(page.locator('input[name=email]')).toHaveValue('test@example.com');
```

État de l'élément

```
// toBeEnabled() - Le bouton/input est activé
await expect(page.locator('button')).toBeEnabled();

// toBeDisabled() - Le bouton/input est désactivé
await expect(page.locator('button[type=submit]')).toBeDisabled();

// toBeChecked() - Le checkbox est coché
await expect(page.locator('input[type=checkbox]')).toBeChecked();

// toHaveAttribute(attr, value) - Vérifier un attribut HTML
await expect(page.locator('a')).toHaveAttribute('href', '/cv');
```





Vérifier (Assertions les plus utilisées)

Localisation et interaction

```
// toHaveFocus() - L'élément a le focus
await expect(page.locator('input')).toHaveFocus();

// toBeInViewport() - L'élément est visible dans la viewport
await expect(page.locator('[data-testid=card]')).toBeInViewport();

// toHaveURL(url) - L'URL actuelle
await expect(page).toHaveURL('http://localhost:4200/cv');
```

CSS et classes

```
// toHaveClass(className) - Possède cette classe CSS
await expect(page.locator('[data-testid=card]')).toHaveClass('active');

// toHaveCSS(prop, value) - Propriété CSS
await expect(page.locator('div')).toHaveCSS('background-color', 'rgb(0, 0, 0)');

// toHaveStyle(style) - Style inline
await expect(page.locator('span')).toHaveStyle('color: red');
```

Avec négation (not)

```
// NOT visible
await expect(page.locator('[data-testid=card]')).not.toBeVisible();

// NOT contient le texte
await expect(page.locator('button')).not.toContainText('Delete');

// NOT a la classe active
await expect(page.locator('li')).not.toHaveClass('active');

// NOT coché
await expect(page.locator('input[type=checkbox]')).not.toBeChecked();
```

<https://playwright.dev/docs/test-assertions>



Gérer le Réseau (Mocking API)

Dans une application Angular, le service HttpClient envoie des requêtes vers un backend. Playwright peut "intercepter" ces appels pour renvoyer des données fictives (**Mocks**).

Pourquoi simuler les appels API ?

- **Rapidité** : Pas besoin d'attendre une réponse réelle du serveur.
- **Fiabilité** : Vos tests ne dépendent pas de l'état de la base de données.
- **Cas limites** : Facile de tester une erreur 500 ou un chargement très long.

<https://playwright.dev/docs/api/class-route>



Gérer le Réseau (Mocking API)

- Playwright utilise `page.route()` pour capturer des URL et y répondre avec un JSON personnalisé par exemple.
- La signature de `page.route` est la suivante : `page.route(url, handler[, options])`
- `url` : l'url à intercepter
- `handler` la méthode à exécuter qui reçoit en paramètre la route et peut en faire ce qu'il veut.
- Parmi les apis de la `route` vous avez la méthode `fulfill` qui permet de **retourner une réponse**

```
await page.route('urlPattern', async route => {  
  const json = cvs;  
  await route.fulfill({  
    json,  
  }); // On renvoie notre faux JSON  
});
```

```
await route.fulfill({  
  json,  
  body?  
}); contentType?  
await headers?  
  path?  
// E response?  
await status?
```

<https://playwright.dev/docs/api/class-route>





Annexe





Angular

Nouveautés ES6 et Typescript



ES6 : Les Fonctions fléchées

- La notation des fonctions fléchées (arrow functions) est l'exemple typique de syntaxe visant à optimiser la compacité du code.
- Cette notation astucieuse, utilisant les caractères `=>`, est même finalement assez lisible.
- Si la fonction attend un seul paramètre, la syntaxe générique est :

```
p => { /* Instructions */};
```

- Si la fonction attend plusieurs paramètres, les parenthèses sont obligatoires :

```
(p1, ..., pN) => { /* Instructions */};
```

- Nous pouvons réécrire notre fonction de mise à jour du titre avec :

```
const saySomething = message => { console.log(message); };
```



ES6 : Les Fonctions fléchées

Si la fonction n'a pas de paramètre, on utilise les parenthèses **vides** :

```
() => { /* Instructions */};
```

Si l'instruction de la fonction retourne une expression, on **écrit** :

```
(p1, ..., pN) => { return expression };
```

La syntaxe peut être encore réduite à :

```
(p1, ..., pN) => expression
```



ES6 : Les Fonctions fléchées

- L'un des avantages majeurs de l'utilisation des fonctions fléchées est la notion de `this` lexical et non local.

```
function Voiture() {  
  this.kilometres = 100;  
  console.log(this.kilometres);  
  setTimeout(function () {  
    console.log(this);  
    this.kilometres += 10;  
    console.log(this.kilometres);  
  }, 1000);  
}
```

```
new Voiture();
```

```
$ node fonctionArrow.js  
100  
Timeout {  
  _idleTimeout: 1000,  
  _idlePrev: null,  
  _idleNext: null,  
  _idleStart: 46,  
  _onTimeout: [Function (anonymous)],  
  _timerArgs: undefined,  
  _repeat: null,  
  _destroyed: false,  
  [Symbol(refed)]: true,  
  [Symbol(kHasPrimitive)]: false,  
  [Symbol(refed)]: true,  
  [Symbol(kHasPrimitive)]: false,  
  [Symbol(asyncId)]: 5,  
  [Symbol(triggerId)]: 1  
}  
NaN
```



ES6 : Les Fonctions fléchées

- Les fonctions fléchées en es6 utilisent un this lexical (celui du contexte ou elles sont définies).

```
function Voiture() {  
  this.kilometres = 100;  
  console.log(this.kilometres);  
  setTimeout(() => {  
    console.log("this", this);  
    this.kilometres += 10;  
    console.log(this.kilometres);  
  }, 1000);  
}  
new Voiture();
```

```
100  
this Voiture { kilometres: 100 }  
110
```



ES6 déstructuration (destructuring)

- La déstructuration permet de définir des variables depuis un tableau ou un objet en utilisant de la mise en correspondance (matching) et plus particulièrement de la correspondance de motifs (pattern matching)

```
var array = ["un", "deux", "trois", "quatre"];
var [a, b, c, d] = array;

console.log(a, b, c, d);
// => un, deux, trois, quatre

// => Il est possible d'ignorer un élément
var [un, , trois, quatre] = ["α", "β", "γ", "δ"];

console.log(un, trois, quatre);
// => α, γ, δ
```



ES6 déstructuration (destructuring)

- Pour les objets on accède via les noms des propriétés.
- Ces noms peuvent être renommés.

```
//Pour les objets on accède avec les noms de propriétés  
var newObject = { a: 1, b: 2, c: 3};  
var { a, b, c} = newObject;  
console.log(a, b, c);  
  
// On peut nommer les paramètres récupérés  
var newObject = { a: 1, b: 2, c: 3};  
var { a: un, b: deux, c: trois} = newObject;  
console.log(un, deux, trois);
```

- En cas d'erreur la variable aura comme valeur *undefined*

```
var [a, b, c] = [1, 2];  
console.log(a, b, c);  
// => 1, 2, undefined  
var [a] = [];  
console.log(a);  
// => undefined
```



ES6 décomposition (spread)

- La **syntaxe de décomposition** permet d'étendre un itérable (tableau, chaîne de caractères ou objet) en lieu et place de plusieurs arguments (pour les appels de fonctions) ou de plusieurs éléments (pour les littéraux de tableaux) ou de paires clés-valeurs (pour les littéraux d'objets).

- Pour l'utilisation de la décomposition dans les appels de fonction :

```
f(...objetIterable);
```

- Pour les littéraux de tableaux :

```
[...objetIterable, 4, 5, 6]
```

- Pour les littéraux objets (nouvelle fonctionnalité pour ECMAScript, actuellement en proposition de niveau 4, finalisée) :

```
let objClone = { ...obj };
```



ES6 décomposition (spread)

- La **syntaxe de décomposition** permet d'étendre un itérable (tableau, chaîne de caractères ou objet) en lieu et place de plusieurs arguments (pour les appels de fonctions) ou de plusieurs éléments (pour les littéraux de tableaux) ou de paires clés-valeurs (pour les littéraux d'objets).

- Pour l'utilisation de la décomposition dans les appels de fonction :

```
f(...objetIterable);
```

- Pour les littéraux de tableaux :

```
[...objetIterable, 4, 5, 6]
```

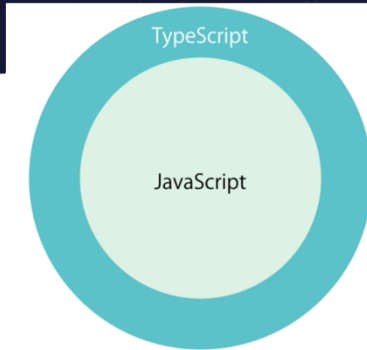
- Pour les littéraux objets (nouvelle fonctionnalité pour ECMAScript, actuellement en proposition de niveau 4, finalisée) :

```
let objClone = { ...obj };
```





Typescript, c'est quoi ?



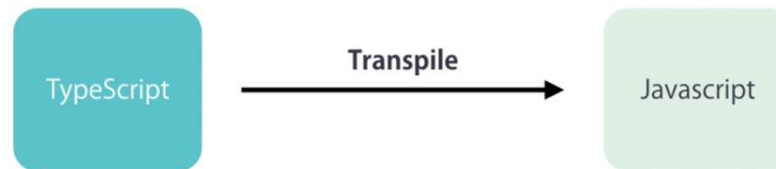
- Typescript est un sur-ensemble (superset) de Javascript.
- Ne peut pas être intégré directement à une page web.
- Permet de tirer partie des fonctionnalités modernes de ES6 comme les modules, les classes, les interfaces, fonctions fléchées, etc..
- Open source.





Typescript, c'est quoi ?

- Doit être transcompilé en Javascript.



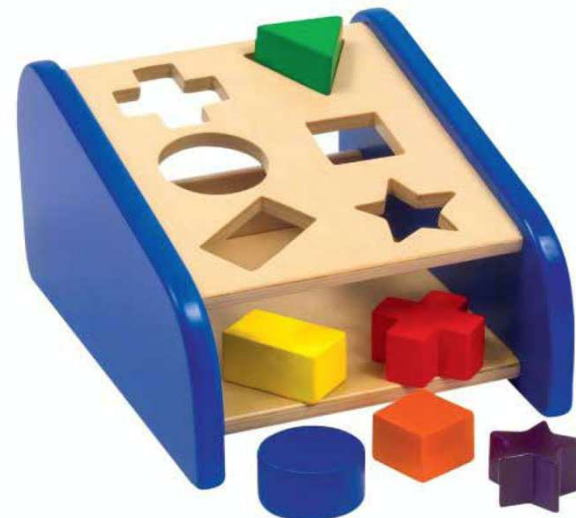
- Un langage transcompilé est un langage pouvant être compilé en un autre langage.
- Autrement dit, on peut parler d'un langage en surcouche d'un langage existant.
- Amène une phase de compilation aux langages interprétés.





Typage

- Typescript permet le **typage implicite** et le **typage explicite**. Vous n'êtes donc pas obligés de "typer" vos variables. Toutefois, c'est fortement recommandé.
- Le typage en TS ne sert qu'au compilateur pour prévenir les erreurs.
- Types proposés par Typescript :
 - Boolean
 - Number
 - String
 - Array
 - Enum
 - Any





Typage

- Une variable déclarée selon un type précis ne peut plus recevoir des valeurs d'un autre type.

```
// legal
let isReady : boolean = false;
let decimal : number = 20;
let name : string = "Dev.to";
let numbers : number[] = [1,2, 3];
```

```
// illegal
let isReady : boolean = 10;
let decimal : number = "not a number";
let name : string = true;
let numbers : number[] = "not an array of numbers";
```

- En cas de besoin, on peut utiliser le pipe (|) pour déclarer les différents types de valeurs qui peuvent être stockés dans une variable.

```
let multiple : boolean | number = 10;
multiple = true; // still valid
```

- Ou encore le type any :

```
let unknown : any = 30;
unknown = "what is this variable?";
```





Type assertions

- Typescript peut avoir des confusions concernant les types de variables.
- Ceci peut poser des problèmes pour le développeur.

```
let msg = 'abc';  
let test = msg.  
  endsWith (method) String.endsWith  
  fixed  
  fontcolor  
  fontsize  
  includes  
  indexOf  
  italics  
  lastIndexOf  
  length  
  link  
  localeCompare  
  match
```

```
let msg;  
msg = 'abc';  
let test = msg.
```

IntelliSense ne propose rien car il ne reconnaît pas msg comme un String





Conversion de types

- Pour contourner ces problèmes, Typescript donne la possibilité de traiter une variable selon un type bien précis. Ce qu'on appelle plus communément le "cast".
- Ainsi, si une variable est déclarée avec le type any, et qu'à un certain moment de votre programme, vous avez besoin d'utiliser certaines méthodes dédiées aux chaînes de caractères (String), vous pouvez utiliser <string> comme dans l'exemple ci-dessous :

```
let unknown : any;  
let unknown = "hello";  
  
console.log("length is : ", (<string>unknown).length);
```





Typage avec les fonctions

- On peut également typer les paramètres et la valeur de retour d'une fonction, toujours avec la syntaxe des deux points:

```
function creerCours(nb: number, nom: string): Cours {  
    var cours = new Cours();  
    cours.nb = nb;  
    cours.nom = nom;  
    return Cours;  
}
```

- Là aussi, le typage des valeurs de retour des fonctions peut être d'une grande utilité et est donc fortement recommandé.

```
// illegal  
function test(name: string, surname: string): string {  
    return name.length; // will return a number here which is not expected  
}
```





Tuples et Enums

- ▶ TypeScript introduit les Tuples (comme dans Python) qui permettent de créer des tableaux où les types des éléments (fixe) sont connus. Ces types peuvent être différents.

```
let x: [boolean, number];  
x = [true, 10]; // initialization  
x = [10, "hello"]; // illegal initialization
```

- ▶ Autre différence avec Javascript, c'est l'arrivée des Enums, très connu en Java. Un Enum est un ensemble de constantes. Il existe trois types d'Enums :
 - ▶ Enum Numériques
 - ▶ Enum Strings
 - ▶ Enum hétérogène

```
enum Direction {  
    Up = 1,  
    Down,  
    Left,  
    Right,  
}  
movePlayer(Direction.Up);
```





Classes

- JavaScript s'appuie sur les fonctions pour la création des composants
- JavaScript s'appuie sur les prototypes pour spécialiser ces composants
- JavaScript ne propose pas de notion de classe jusqu'à ES 2015.
- TypeScript propose d'utiliser les notions de programmation orientée objet tel que :
 - Le constructeur
 - L'encapsulation
 - Les accesseurs (ES5)
 - L'héritage
 - Les classes et interfaces

```
class Personne {  
    name : string;  
  
    constructor(name : string) {  
        this.name = name;  
    }  
  
    sayHello() {  
        console.log("Hello, i am " + this.name);  
    }  
}
```

```
let p : Personne;  
p = new Personne("Nidhal");  
p.sayHello();
```



Constructor

- ▶ Comme en Java, le constructeur est responsable de la création des objets à partir d'une classe.
- ▶ Contrairement au Java, une classe ne peut implémenter qu'un seul constructeur. D'où la possibilité de déclarer des arguments optionnels avec "?".
- ▶ Regardez bien la différence entre les 2 bouts de code. Alors ?

```
class Point {  
  private x : number;  
  private y : number;  
  constructor(x:number, y?:number) {  
    this.x = x;  
    this.y = y;  
  }  
  
  showPoint() {  
    console.log(`X: ${this.x} Y: ${this.y}`);  
  }  
}
```



```
class Point {  
  constructor(private x:number, private y:number) {  
  }  
  
  showPoint() {  
    console.log(`X: ${this.x} Y: ${this.y}`);  
  }  
}
```





Accesseurs

- Comme en Java ou en C#, le principe d'encapsulation adopté par Typescript nous propose 3 modes d'accès:
 - Public
 - Private
 - Protected
- Contrairement à Java, le mode par défaut est Public.





Propriétés

- Pour accéder en lecture ou en écriture à un attribut de classe, nous avons l'habitude d'utiliser des accesseurs (getters + setters).

```
class Point {  
  constructor(private x:number, private y:number) {  
  }  
  
  showPoint() {  
    console.log(`X: ${this.x} Y: ${this.y}`);  
  }  
  
  getX() {  
    return this.x;  
  }  
  
  getY() {  
    return this.y;  
  }  
}
```

```
class Point {  
  constructor(private x:number, private y:number) {  
  }  
  
  showPoint() {  
    console.log(`X: ${this.x} Y: ${this.y}`);  
  }  
  
  get X() {  
    return this.x;  
  }  
  
  set X(newValue) {  
    this.x = newValue;  
  }  
}
```

Testons ces bouts de code...



Interfaces

- Les Interfaces nous permettent d'établir des contraintes par rapport aux objets et aux classes qu'on implémente.
- Peuvent être utilisés pour définir des types bien définies sans passer par les classes.
- Il est important de noter que les interfaces ne sont pas compilées en Javascript.
- Ils servent juste à la validation de nos variables ou autre objet par le compilateur de Typescript.





Generics

- Generics en Typescript donne la possibilité de passer plusieurs types à un composant, en ajoutant une couche supplémentaire d'abstraction et de réutilisation à notre code.
- Les génériques peuvent être appliqués aux fonctions, interfaces et classes dans Typescript.

```
function identity<T>(arg: number): number {  
  return arg;  
}
```



```
function identity<T>(arg: T): T {  
  return arg;  
}
```





Modules

- Les modules servent à partager du code entre divers fichiers sources. Utilisation des mots-clés **Export** et **Import**.
- 2 types d'export : Export nommée et Export par défaut.
- On ne peut avoir qu'une seul Export par défaut par fichier.
- Possibilité d'utiliser des alias.

```
export default class Etudiant {  
  id : number;  
  nom : string;  
  prenom : string;  
}
```

```
export function testFct() {  
  console.log("This is a text");  
}
```

```
import Etudiant, { testFct } from './component';
```

```
import Etudiant, { testFct as tf } from './component';
```





Décorateurs

- Les décorateurs (appelés également annotations) TypeScript permettent d'ajouter des informations sur nos classes, pour indiquer par exemple que telle classe est un composant de l'application, ou telle autre un service. On utilise @ comme syntaxe :

```
@Pipe({  
  name: 'filter',  
  pure : false  
})  
export class FilterPipe {
```

```
@Directive({  
  selector: '[appCustomDir]'  
})  
export class CustomDirDirective {
```

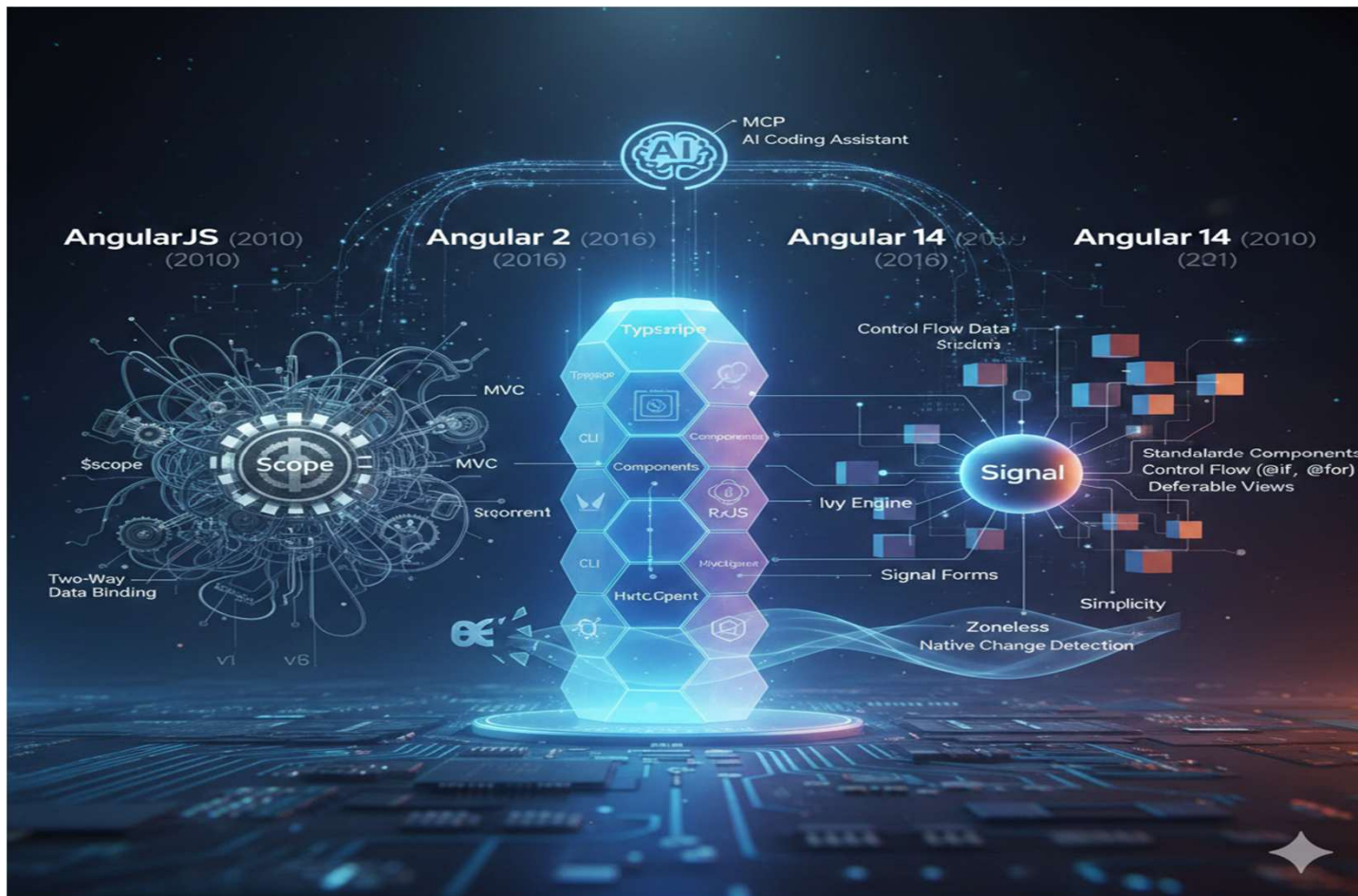
```
@Component({  
  selector: 'app-card',  
  templateUrl: './card.component.html',  
  styleUrls: ['./card.component.css']  
})  
export class CardComponent {
```





AngularJs, Angular, Angular la renaissance

L'évolution d'angular





AngularJs, Angular, Angular la renaissance

L'évolution d'angular

- L'évolution d'Angular est l'une des plus fascinantes du monde JavaScript. On est passé d'un framework "magique" mais lourd (AngularJS) à une plateforme ultra-performante, modulaire et désormais orientée vers la réactivité fine (Signals).

L'ère de la rupture : AngularJS vs Angular 2

- **AngularJS (v1.x - 2010)** : Basé sur le pattern MVC, le \$scope, et le "Two-way data binding" automatique. Très puissant mais difficile à optimiser pour les grosses applications.
- **Angular 2 (2016) : Réécriture totale.**
 - Passage au **TypeScript**.
 - Architecture **basée sur les composants** (fin du \$scope).
 - Introduction de la CLI (Command Line Interface).
 - Utilisation de **RxJS** pour l'asynchronisme.





AngularJs, Angular, Angular la renaissance

La stabilisation et l'optimisation (v4 à v11)

- v4 - v8 : Introduction du HttpClient (v4), du "Differential Loading" (v8) pour servir du code plus léger aux navigateurs modernes.
- v9 (2020) - L'arrivée d'Ivy : Le nouveau moteur de rendu Ivy devient le standard. Il permet des bundles plus petits et une compilation beaucoup plus rapide.
- v11 : Suppression du support d'Internet Explorer 11.





AngularJs, Angular, Angular la renaissance

L'ère "Standalone" et la Renaissance (v14 à v17)

- C'est ici que le framework a commencé à se simplifier radicalement pour redevenir compétitif face à React ou Vue.
- **v14 & v15 (2022) : Arrivée des Standalone Components.** Les NgModules deviennent optionnels, ce qui réduit considérablement le "boilerplate" (code répétitif).
- **v16 (2023) - La Révolution des Signals :** Introduction des **Signals** en version preview. C'est une nouvelle façon de gérer l'état de l'application de manière beaucoup plus précise, sans dépendre entièrement de Zone.js.
- **v17 (Novembre 2023) :** * Nouveau **Control Flow** dans les templates (remplacement des *ngIf et *ngFor par @if et @for).
 - **Deferrable Views** (@defer) pour charger des parties de la page uniquement quand elles sont visibles.





AngularJs, Angular, Angular la renaissance

Vers le "Zoneless" et l'IA (v18 à v21)

- Les versions récentes visent la performance brute et l'intégration des nouveaux standards web

Version	Changement Majeur	Impact Développeur
v18	Zoneless (Experimental)	Possibilité de se passer de zone.js, rendant l'app plus légère et rapide.
v19	Standalone par défaut	Plus besoin d'écrire standalone: true. Arrivée de l'API resource() pour simplifier le fetch de données.
v20 (Mai 2025)	Zoneless stabilisé	La détection de changement devient native et ultra-performante.
v21 (Nov 2025)	Experimental Signal Forms	Les formulaires entament le chemin réactif via les Signals.

